You can access this page also inside the Remote Desktop by using the icons on the desktop

- Score
- Questions and Answers
- Exam Tips

Expand All

CNPE Simulator Kubernetes 1.35

https://killer.sh

Each question needs to be solved on a specific instance other than your main `candidate@terminal`. You'll need to connect to the correct instance via ssh, the command is provided before each question. To connect to a different instance you always need to return first to your main terminal by running the `exit` command, from there you can connect to a different one.

In the real exam each question will be solved on a different instance whereas in the simulator multiple questions will be solved on same instances.

Use `sudo -i` to become root on any node in case necessary.

▼ Question 1 | Operator Pattern, CRD, Kustomize, Git

Solve this question on: `ssh cnpe7683`

A custom operator is in development. The `TeamMonitoring` *CRD* has been installed in the cluster from `/course/1/team-monitoring`, which is a local Git repository with Kustomize configuration.

1. Create a new version `v1alpha2` of the *CRD*, where property `target` is an object with two string properties: `namespace` and `service`
2. Deploy the updated *CRD* to the cluster using Kustomize
3. Commit your change to Git in local branch `main`
4. Create a *TeamMonitoring* resource in *Namespace* `pacific`, named `general` . Field `target.namespace` should be `test-ns` and `target.service` should be `test-svc`

Answer:

Investigate

Let's have a look at the given scenario:

```
➜ ssh cnpe7683

➜ candidate@cnpe7683:~$ cd /course/1/team-monitoring

➜ candidate@cnpe7683:/course/1/team-monitoring$ git status
```

```
On branch main
nothing to commit, working tree clean

➜ candidate@cnpe7683:/course/1/team-monitoring$ git log
commit e68c24d04df2f883e6034027015e8418d421d423 (HEAD -> main)
Author: root <root@cnpe7683>
Date:   Thu Jun 5 20:19:58 2025 +1000

    init of project

➜ candidate@cnpe7683:/course/1/team-monitoring$ ls
README.md  crd.yaml  kustomization.yaml
```

We can see the Kustomize code. The question says it contains a *CRD* named `TeamMonitoring` and that is has already been installed in the cluster:

```
➜ candidate@cnpe7683:/course/1/team-monitoring$ k get crd
NAME                                  CREATED AT
teammonitorings.monitoring.killer.sh  2025-11-05T19:03:17Z

➜ candidate@cnpe7683:/course/1/team-monitoring$ k get TeamMonitoring -A
No resources found
```

Looks like the *CRD* exists, just no resources yet for it.

Step 1: Update *CRD*

The question requires us to update the *CRD*. If the *CRD* is already in use by others, meaning there are resources of that *CRD*, then a good way is to create a new version to not break things.

```
➜ candidate@cnpe7683:/course/1/team-monitoring$ vim crd.yaml
```

```yaml
# cnpe7683:/course/1/team-monitoring/crd.yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: teammonitorings.monitoring.killer.sh
spec:
  group: monitoring.killer.sh
  scope: Namespaced
  names:
    kind: TeamMonitoring
    plural: teammonitorings
    singular: teammonitoring
    shortNames:
      - tmon
  versions:
    - name: v1alpha1
      served: true
      storage: false      # only one version can be storage, set this to false
      schema:
        openAPIV3Schema:
          type: object
          description: TeamMonitoring defines monitoring configuration per team
          properties:
            apiVersion:
              type: string
            kind:
              type: string
```

```
            metadata:
              type: object
            spec:
              type: object
              properties:
                target:
                  type: string
                  description: Target service to monitor
            status:
              type: object
# NEW CODE FROM HERE DOWN
    - name: v1alpha2      # define the new version
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          description: TeamMonitoring defines monitoring configuration per team
          properties:
            apiVersion:
              type: string
            kind:
              type: string
            metadata:
              type: object
            spec:
              type: object
              properties:
                target:
                  type: object     # target is now type object
                  properties:
                    namespace:     # first property
                      type: string
                    service:       # second property
                      type: string
            status:
              type: object
```

Step 2: Deploy using Kustomize

We can build the whole manifest using Kustomize and should see our additions. This will just output the YAML without any changes in the cluster:

```
➜ candidate@cnpe7683:/course/1/team-monitoring$ k kustomize .
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: teammonitorings.monitoring.killer.sh
spec:
  group: monitoring.killer.sh
  names:
    kind: TeamMonitoring
...
  - name: v1alpha2
    schema:
      openAPIV3Schema:
        description: TeamMonitoring defines monitoring configuration per team
        properties:
          apiVersion:
            type: string
          kind:
```

```
            type: string
          metadata:
            type: object
          spec:
            properties:
              target:
                properties:
                  namespace:
                    type: string
                  service:
                    type: string
                type: object
            type: object
          status:
            type: object
        type: object
    served: true
    storage: true
```

Next we better diff before we apply. We could do `k kustomize . | k diff -f -` which builds the manifest and pipes it to `kubectl diff`. But instead we can also simply use:

```
➜ candidate@cnpe7683:/course/1/team-monitoring$ k diff -k .
...
   name: teammonitorings.monitoring.killer.sh
   resourceVersion: "29348"
   uid: 398c63fe-4e77-498f-a50f-c1368e565046
@@ -43,6 +43,32 @@
             type: object
         type: object
     served: true
+    storage: false
+  - name: v1alpha2
+    schema:
+      openAPIV3Schema:
+        description: TeamMonitoring defines monitoring configuration per team
+        properties:
+          apiVersion:
+            type: string
+          kind:
+            type: string
+          metadata:
+            type: object
+          spec:
+            properties:
+              target:
+                properties:
+                  namespace:
+                    type: string
+                  service:
+                    type: string
+                type: object
+            type: object
+          status:
+            type: object
+        type: object
+    served: true
     storage: true
 status:
```

```
   acceptedNames:
@@ -65,3 +91,4 @@
     type: Established
   storedVersions:
   - v1alpha1
+  - v1alpha2
```

Once we're satisfied we apply:

```
➜ candidate@cnpe7683:/course/1/team-monitoring$ k apply -k .
customresourcedefinition.apiextensions.k8s.io/teammonitorings.monitoring.killer.sh configured

➜ candidate@cnpe7683:/course/1/team-monitoring$ k get crd
NAME                                  CREATED AT
teammonitorings.monitoring.killer.sh   2025-11-05T19:43:31Z

➜ candidate@cnpe7683:/course/1/team-monitoring$ k describe crd teammonitorings.monitoring.killer.sh
Name:         teammonitorings.monitoring.killer.sh
Namespace:
Labels:       <none>
Annotations:  <none>
API Version:  apiextensions.k8s.io/v1
Kind:         CustomResourceDefinition
Metadata:
  Creation Timestamp:  2025-11-05T19:43:31Z
  Generation:          2
  Resource Version:    29804
  UID:                 398c63fe-4e77-498f-a50f-c1368e565046
Spec:
  Conversion:
    Strategy:  None
  Group:       monitoring.killer.sh
  Names:
    Kind:       TeamMonitoring
    List Kind:  TeamMonitoringList
    Plural:     teammonitorings
    Short Names:
      tmon
    Singular:  teammonitoring
  Scope:       Namespaced
  Versions:
    Name:  v1alpha1
    Schema:
      openAPIV3Schema:
...
    Name:          v1alpha2
    Schema:
      openAPIV3Schema:
...
Status:
...
  Stored Versions:
    v1alpha1
    v1alpha2
```

Our change is live.

Step 3: Git Commit

In good old GitOps fashion we store all changes in Git. We confirm that we're in the required branch `main` and query the status of changes:

```
➜ candidate@cnpe7683:/course/1/team-monitoring$ git branch
* main

➜ candidate@cnpe7683:/course/1/team-monitoring$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   crd.yaml

no changes added to commit (use "git add" and/or "git commit -a")
```

With `git status` we can see the changes we did compared to the last commit. Now we can `git add` and `git commit`:

```
➜ candidate@cnpe7683:/course/1/team-monitoring$ git add .

➜ candidate@cnpe7683:/course/1/team-monitoring$ git commit -m 'update'
[main 23d075a] update
 1 file changed, 28 insertions(+), 1 deletion(-)

➜ candidate@cnpe7683:/course/1/team-monitoring$ git log
commit 23d075aad747d476ca040a95ac3129b228f68bfc (HEAD -> main)
Author: CNPE User <cnpe-user@simulator>
Date:   Wed Nov 5 19:54:44 2025 +0000

    update

commit c0fa8a530ada7008dc5a7c83e1a908e36a776362
Author: root <root@cnpe7683>
Date:   Thu Jun 5 20:19:58 2025 +1000

    init of project
```

No way to hide anything any longer, our change is immortalised in Git history. Actually, it is still possible to alter local Git history that has not been pushed, but let's not get into that now!

Step 4: Create resource

Now we can finally use our updated *CRD* in version `v1alpha2`. For this we create a file, but it's not needed to do this in the Git directory, so we do it in our home dir.

> ℹ️ We name the file `1.yaml` because it belongs to the first question. In this simulator multiple questions will be solved on the same servers, so it makes sense to name accordingly

```
➜ candidate@cnpe7683:~$ vim 1.yaml
```

```yaml
# cnpe7683:/home/candidate/1.yaml
apiVersion: monitoring.killer.sh/v1alpha2   # new version
kind: TeamMonitoring
metadata:
  name: general
  namespace: pacific
spec:
  target:
    namespace: test-ns   # first field
    service: test-svc    # second field
```

That's it, looks kinda clean and simple. Does it work though?

```
➜ candidate@cnpe7683:~$ k apply -f 1.yaml
teammonitoring.monitoring.killer.sh/general created

➜ candidate@cnpe7683:~$ k get TeamMonitoring -A
NAMESPACE   NAME      AGE
pacific     general   5s
```

It works. Though in this scenario there is actually no one that will do anything with that *TeamMonitoring* resource `general`, apart from the simulator scoring system hopefully!

But the idea would be to have a custom controller, like a *Deployment*, which watches for created/updated/deleted *TeamMonitoring* resources by communicating with the K8s Api. It then creates *Pods* or anything else according to the information in the resource. Like in this case the controller could even create a custom Prometheus alert that watches the *Service* `test-sec` in *Namespace* `test-ns`.

This combination of custom *CRDs* and controllers that manage them can be called the **Operator pattern**. It allows for highly customisable and extensible applications on top of Kubernetes.

▼ Question 2 | Prometheus Monitoring

Solve this question on: `ssh cnpe4328`

Your team is evaluating a minimal Prometheus installation as a potential platform service offering. Prometheus is installed in *Namespace* `prometheus` and can be accessed at `http://cnpe4328:30020`.

Currently only *Pods* in *Namespace* `kariba` with the labels `app=frontend` and `app=backend` are being scraped.

1. Extend the existing scrape configuration `minimal` in the *ConfigMap* `prometheus-server` so that *Pods* with the label `app=proxy` are also scraped. Make sure Prometheus uses the updated configuration
2. Afterwards, run a query to calculate the **sum** of `http_requests_per_minute{}` for each *Deployment*. Identify the one with the highest sum and scale it to `2` replicas

ℹ️ After restarting Prometheus, it may take 10-20 seconds for newly scraped metrics to become available in query results

Answer:

Step 1: Adjust scrape config

We need to alter the existing Prometheus installation, first we check how it's installed:

```
➜ ssh cnpe4328

➜ candidate@cnpe4328:~$ k -n prometheus get all
NAME                         READY   STATUS    RESTARTS   AGE
pod/prometheus-server-0      1/1     Running   0          5m36s

NAME                           TYPE        ...   PORT(S)          AGE
service/prometheus-server      NodePort    ...   9090:30020/TCP   5m36s

NAME                                    READY   AGE
statefulset.apps/prometheus-server      1/1     5m36s

➜ candidate@cnpe4328:~$ k -n prometheus get cm
NAME                 DATA   AGE
kube-root-ca.crt     1      5m49s
prometheus-server    2      5m49s
```

Looks like it has been deployed via a *StatefulSet* and we also see the *ConfigMap* in question.

```
➜ candidate@cnpe4328:~$ k -n prometheus edit cm prometheus-server
```

```
# kubectl -n prometheus edit cm prometheus-server
apiVersion: v1
data:
  prometheus.rules: 'groups: []'
  prometheus.yml: |
    global:
      scrape_interval: 10s
      evaluation_interval: 10s
    rule_files:
      - /etc/prometheus/prometheus.rules
    alerting:
      alertmanagers: []
    scrape_configs:
      - job_name: 'minimal'
        kubernetes_sd_configs:
          - role: pod
            namespaces:
              names: ['kariba']
        relabel_configs:
          - source_labels: [__meta_kubernetes_pod_label_app]
            regex: (frontend|backend|proxy)                          # UPDATE
            action: keep
          - source_labels: [__meta_kubernetes_pod_ip]
            target_label: __address__
            replacement: $1:8080
          - target_label: __metrics_path__
            replacement: /metrics
...
```

The *ConfigMap* contains two files, `prometheus.rules` and `prometheus.yml`. These are most probably mounted into the *StatefulSet*. We only have to do a small change in the `regex:` line to add the additional label.

We should be comfortable making smaller changes to service config like the above. Even if we have never worked with a Prometheus scrape config, from the question text alone we should be able to figure out the change.

After we applied the change we need to ensure that the Prometheus *Pod* was restarted to work with the latest version.

```
→ candidate@cnpe4328:~$ k -n prometheus get pod
NAME                 READY   STATUS    RESTARTS   AGE
prometheus-server-0   1/1     Running   0          15m

→ candidate@cnpe4328:~$ k -n prometheus rollout restart sts prometheus-server
statefulset.apps/prometheus-server restarted

→ candidate@cnpe4328:~$ k -n prometheus get pod
NAME                 READY   STATUS    RESTARTS   AGE
prometheus-server-0   1/1     Running   0          4s
```

Extra Info: Prometheus config reload without restart

Instead of restarting the *StatefulSet* we could also initiate a config reload like this:

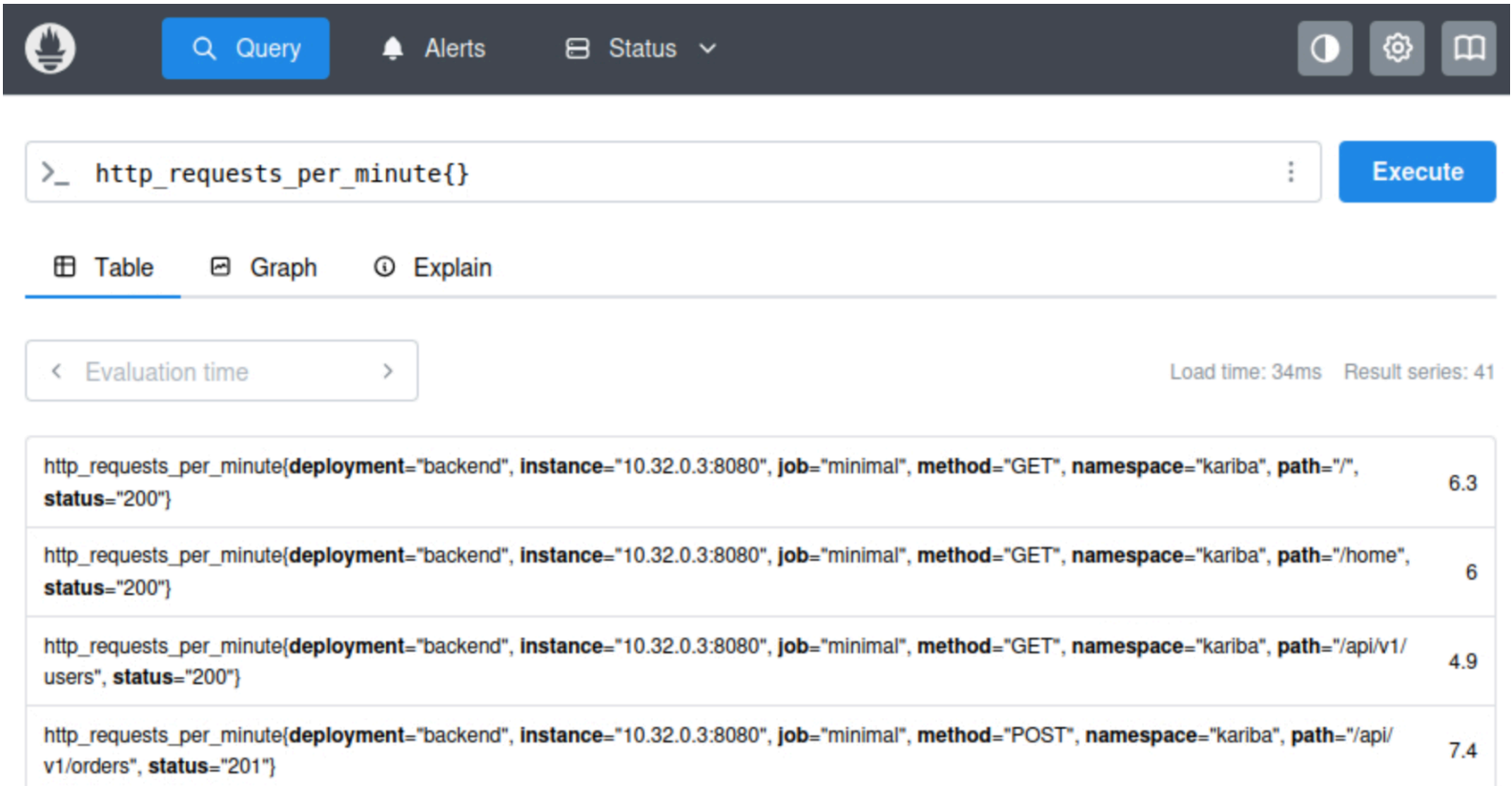
```
→ candidate@cnpe4328:~$ curl -X POST http://cnpe4328:30020/-/reload
```

This is possible because Prometheus is running with argument `--web.enable-lifecycle`.

Step 2: Scale up *Deployment* with highest requests per minute

Now we access the Prometheus web interface on `http://cnpe4328:30020`. We should see quite some entries if we run:

```
http_requests_per_minute{}
```



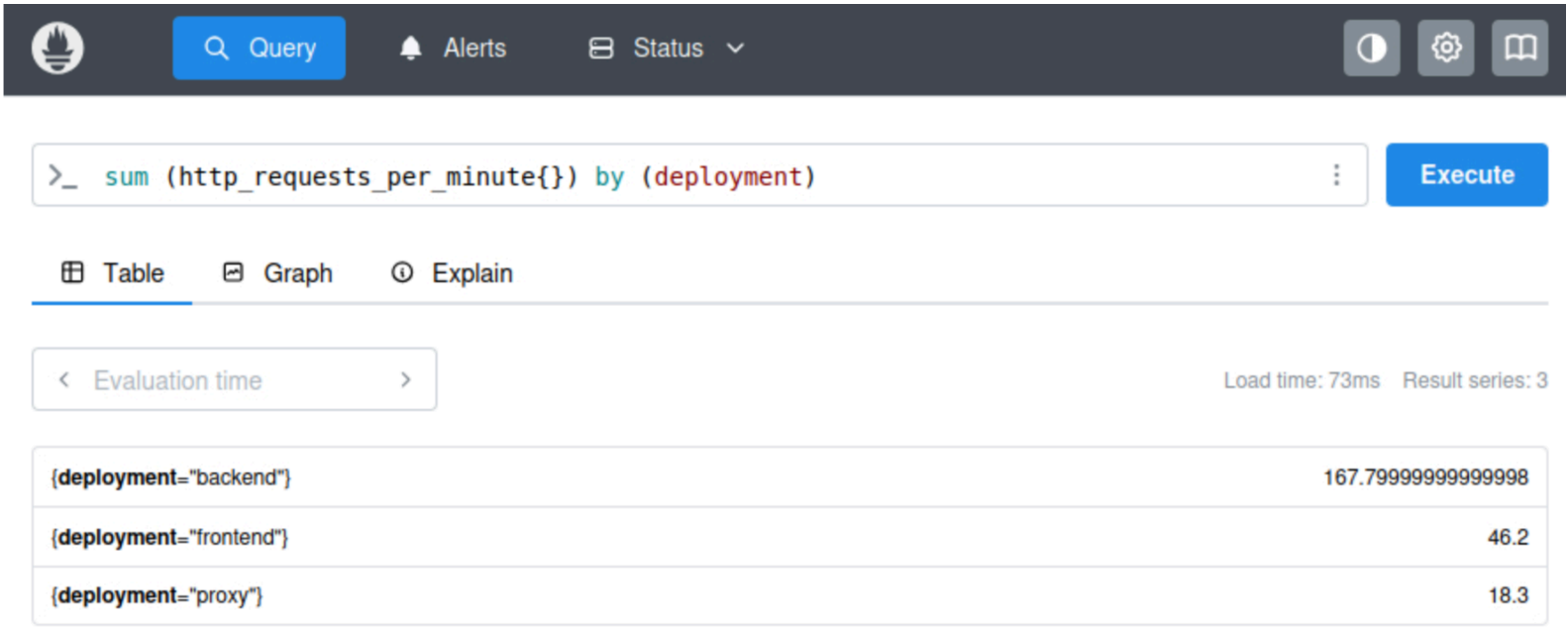
Here are some results exported:

```
http_requests_per_minute{deployment="backend", instance="10.32.0.3:8080", job="minimal", method="GET",
namespace="kariba", path="/", status="200"} 6.3
http_requests_per_minute{deployment="backend", instance="10.32.0.3:8080", job="minimal", method="GET",
namespace="kariba", path="/home", status="200"}     6
http_requests_per_minute{deployment="backend", instance="10.32.0.3:8080", job="minimal", method="GET",
namespace="kariba", path="/api/v1/users", status="200"}     4.9
...
http_requests_per_minute{deployment="frontend", instance="10.32.0.2:8080", job="minimal", method="GET",
namespace="kariba", path="/api/v1/health", status="200"}   10
http_requests_per_minute{deployment="frontend", instance="10.32.0.2:8080", job="minimal", method="GET",
namespace="kariba", path="/api/v1/cart", status="200"}     3.8
http_requests_per_minute{deployment="frontend", instance="10.32.0.2:8080", job="minimal", method="POST",
namespace="kariba", path="/api/v1/cart/add", status="201"}
...
http_requests_per_minute{deployment="proxy", instance="10.32.0.6:8080", job="minimal", method="GET",
namespace="kariba", path="/api/v1/internal/payments/status", status="200"}    2.6
http_requests_per_minute{deployment="proxy", instance="10.32.0.6:8080", job="minimal", method="DELETE",
namespace="kariba", path="/api/v1/internal/sessions/cleanup", status="204"}        0.3
http_requests_per_minute{deployment="proxy", instance="10.32.0.6:8080", job="minimal", method="GET",
namespace="kariba", path="/api/v1/internal/metrics/aggregate", status="200"}
...
```

Because of the change in step 1 we should see labels `deployment="frontend"`, `deployment="backend"` and `deployment="proxy"` for metric `http_requests_per_minute`.

We can "sum by label" to find the *Deployment* with the highest amount of total requests:

```
sum (http_requests_per_minute{}) by (deployment)
```



The same query can also be written in an equivalent form:

```
sum by (deployment) (http_requests_per_minute{})
```

Or we could simply filter by the `deployment` label and run these 3 queries, but if there are more *Deployments* or we don't know them all this would be not ideal:

```
sum(http_requests_per_minute{deployment="frontend"})

sum(http_requests_per_minute{deployment="backend"})

sum(http_requests_per_minute{deployment="proxy"})
```

Using our amazing query power we should see that *Deployment* `backend` has the most requests per minute, hence we scale it up:

```
➜ candidate@cnpe4328:~$ k -n kariba get deploy
NAME       READY   UP-TO-DATE   AVAILABLE   AGE
backend    1/1     1            1           7m4s
frontend   1/1     1            1           2d
proxy      1/1     1            1           6m43s

➜ candidate@cnpe4328:~$ k -n kariba scale deploy backend --replicas 2
deployment.apps/backend scaled

➜ candidate@cnpe4328:~$ k -n kariba get deploy
NAME       READY   UP-TO-DATE   AVAILABLE   AGE
backend    2/2     2            2           7m20s
frontend   1/1     1            1           2d
proxy      1/1     1            1           6m59s
```

Done here.

▼ Question 3 | Argo CD

Solve this question on: `ssh cnpe3849`

Argo CD is installed with the web interface reachable at `http://cnpe3849:30030` and the `argocd` command installed. Use user `admin` with password `admin` where needed.

1. The existing Argo CD application `web-client` is connected to the Git repository cloned at `/course/3/web-client`. Commit, push and sync these changes:
   - The *Pod* label `version` should be `v2`
   - The content the Nginx server returns should be `Lagoon Web Client v2`
2. In `/course/3/web-client` create new Git branch `testing`:
   - Set the *Pod* label `version` should be `v3`
   - The content the Nginx server returns should be `Lagoon Web Client v3`
   - Push the change
3. Create new Argo CD application `web-client-testing`. It should have the same settings from `web-client` with two differences:
   - The Git branch to use as source should be `testing`
   - The destination K8s *Namespace* should be `lagoon-testing`

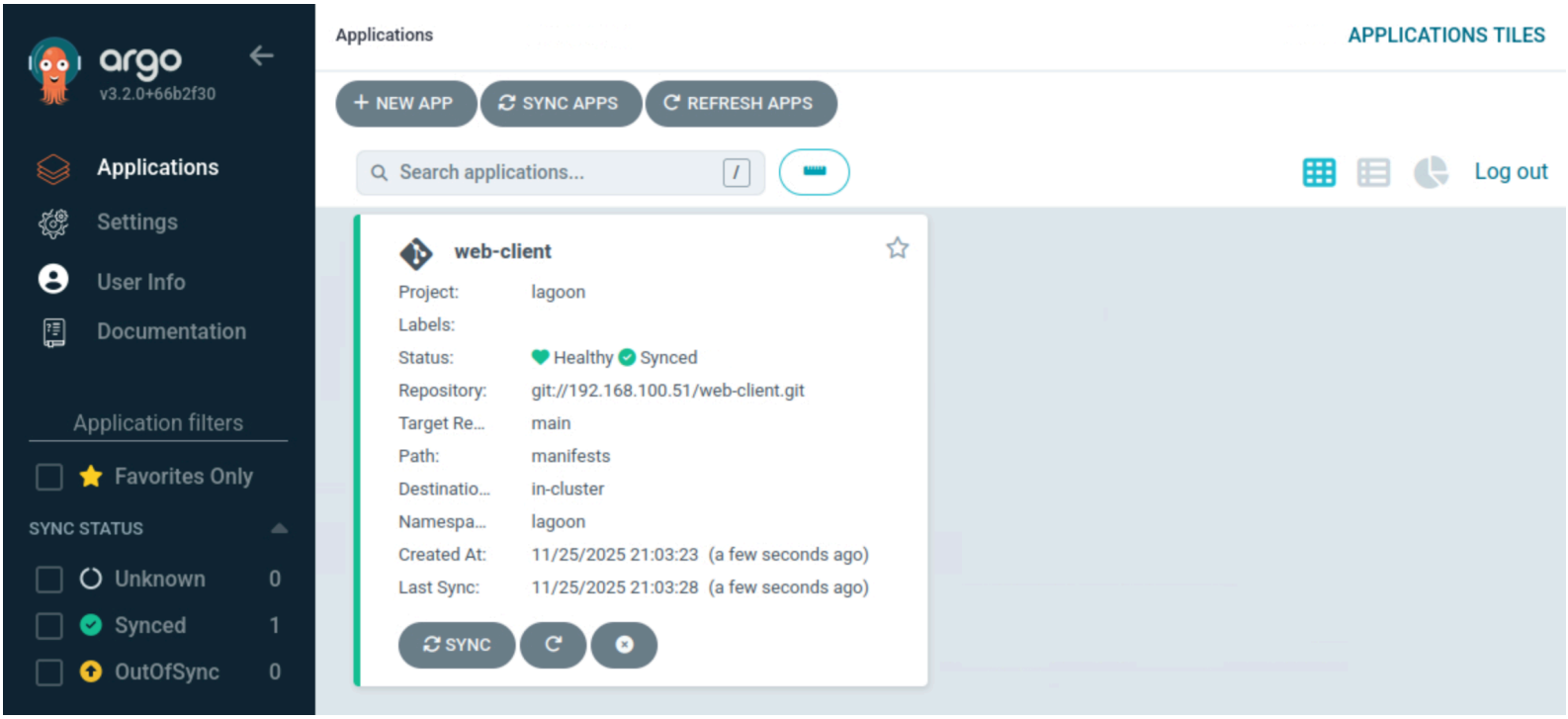
   Ensure the new Argo CD application is applied without errors.

Answer:

Argo CD is a Git-driven Kubernetes deployment controller that watches a Git repository for Kubernetes YAML files and applies these to a Kubernetes cluster. Argo CD can install applications to the same cluster where it runs in or to others if they have been configured.

Step 1: Update application and deploy

Once we log into the Argo CD web interface we can see the existing application:



The status says *Healthy* and *Synced*, which means Argo CD successfully reached the Git repository, rendered the manifests, applied them to the cluster, and confirmed that the Kubernetes resources now match the desired state stored in Git.

This means we should also see the K8s resources:

```
→ candidate@terminal:~$ ssh cnpe3849

→ candidate@cnpe3849:~$ k -n lagoon get cm,deploy,pod,svc
NAME                           DATA   AGE
configmap/kube-root-ca.crt     1      79m
configmap/web-client           1      13m

NAME                           READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/web-client     1/1     1            1           13m

NAME                                READY   STATUS    RESTARTS   AGE
pod/web-client-6744c9fdd-s2gmf      1/1     Running   0          13m

NAME                 TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)   AGE
service/web-client   ClusterIP   10.109.220.198   <none>        80/TCP    13m
```

Now we perform the change:

```
→ candidate@cnpe3849:~$ cd /course/3/web-client

→ candidate@cnpe3849:/course/3/web-client$ vim manifests/web-client.yaml
```

```
# cnpe3849:/course/3/web-client/manifests/web-client.yaml
apiVersion: v1
```

```yaml
kind: ConfigMap
metadata:
  name: web-client
data:
  nginx.conf: |
    events {}
    http {
      server {
        listen 80;
        location / {
          return 200 'Lagoon Web Client v2';    # CHANGE
        }
      }
    }
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-client
spec:
  replicas: 1
  selector:
    matchLabels:
      app: web-client
  template:
    metadata:
      labels:
        app: web-client
        version: v2     # CHANGE
    spec:
      containers:
        - name: nginx
          image: nginx:1-alpine
          volumeMounts:
            - name: web-client
              mountPath: /etc/nginx/nginx.conf
              subPath: nginx.conf
      volumes:
        - name: web-client
          configMap:
            name: web-client
...
```

Now we can see the change in Git and need to commit and push:

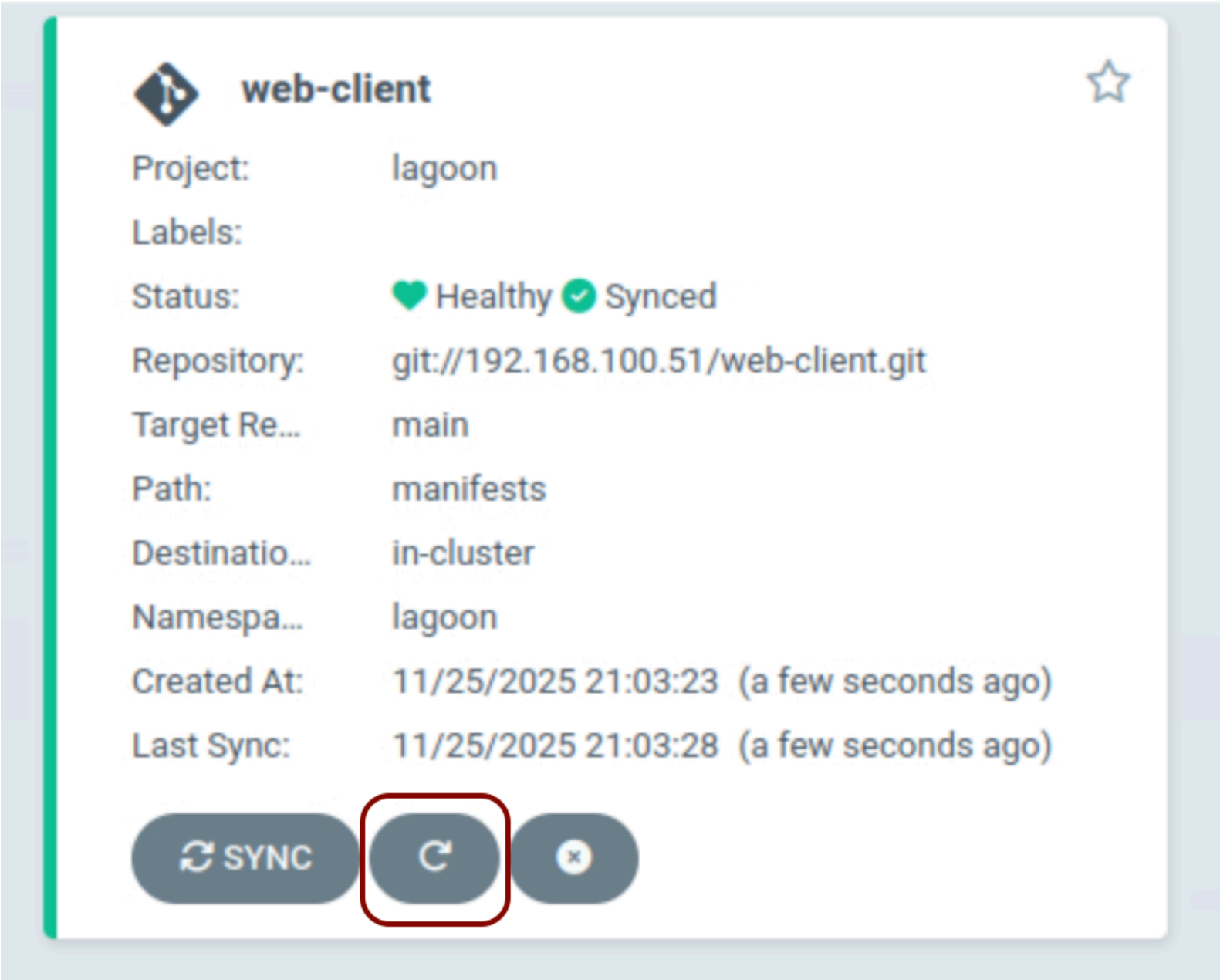
```
→ candidate@cnpe3849:/course/3/web-client$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   manifests/web-client.yaml

no changes added to commit (use "git add" and/or "git commit -a")

→ candidate@cnpe3849:/course/3/web-client$ git add manifests/web-client.yaml

→ candidate@cnpe3849:/course/3/web-client$ git commit -m 'update'
[main 5745cff] update
 1 file changed, 2 insertions(+), 1 deletion(-)

→ candidate@cnpe3849:/course/3/web-client$ git status
```

```
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

So far all changes only happened in our local clone of the Git repository. But we still need to push the changes to the remote:

```
→ candidate@cnpe3849:/course/3/web-client$ git remote -v
origin  git://192.168.100.51/web-client.git (fetch)
origin  git://192.168.100.51/web-client.git (push)

→ candidate@cnpe3849:/course/3/web-client$ git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 385 bytes | 192.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0
To git://192.168.100.51/web-client.git
   3a7e62c..5745cff  main -> main
```

Argo CD will check the Git source very few minutes, we can speed this up by clicking the the refresh button:



The refresh will look a the current Git state, see there is a difference and start a sync. We could also force a sync via:

```
argocd app sync web-client
```

After deployment we should see that a new *Pod* was created and that it has the new label:

```
→ candidate@cnpe3849:/course/3/web-client$ k -n lagoon get pod --show-labels
NAME                          ...   STATUS   ...   LABELS
web-client-7845db4c66-4hs84 ...   Running ...   app=web-client,version=v2...
```

The text Nginx returns should also be the new one:

```
→ candidate@cnpe3849:/course/3/web-client$ k -n lagoon exec -it web-client-7845db4c66-4hs84 -- curl
localhost:80
Lagoon Web Client v2
```

Step 2: Create and push new Git branch with file changes

We should create a new Git branch `testing` and perform a smaller change in it:

```
→ candidate@cnpe3849:/course/3/web-client$ git checkout -b testing
Switched to a new branch 'testing'

→ candidate@cnpe3849:/course/3/web-client$ git branch
  main
* testing

→ candidate@cnpe3849:/course/3/web-client$ vim manifests/web-client.yaml
```

```yaml
# cnpe3849:/course/3/web-client/manifests/web-client.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: web-client
data:
  nginx.conf: |
    events {}
    http {
      server {
        listen 80;
        location / {
          return 200 'Lagoon Web Client v3';   # CHANGE
        }
      }
    }
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-client
spec:
  replicas: 1
  selector:
    matchLabels:
      app: web-client
  template:
    metadata:
      labels:
        app: web-client
        version: v3   # CHANGE
```

```
    spec:
...
```

Next we `git add` and `git push`:

```
→ candidate@cnpe3849:/course/3/web-client$ git status
On branch testing
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   manifests/web-client.yaml

no changes added to commit (use "git add" and/or "git commit -a")

→ candidate@cnpe3849:/course/3/web-client$ git add manifests/web-client.yaml

→ candidate@cnpe3849:/course/3/web-client$ git commit -m 'update'
[testing 245a7c7] update
 1 file changed, 2 insertions(+), 2 deletions(-)

→ candidate@cnpe3849:/course/3/web-client$ git push origin testing
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 371 bytes | 123.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0
To git://192.168.100.51/web-client.git
   5745cff..245a7c7  testing -> testing
```

So far nothing should've been changed in Argo CD, because there is only one application which is checking branch `main` and not branch `testing`.

Step 3: Create a new Argo CD application

Now we create a second Argo CD application `web-client-testing` which checks Git branch `testing` and deploys to K8s *Namespace* `lagoon-testing`. We could use the Argo CD web interface to create the new application and copy over most settings. Or we do this by using the `applications.argoproj.io` resource:

```
→ candidate@cnpe3849:~$ k get crd
NAME                          CREATED AT
applications.argoproj.io      2025-11-25T16:04:37Z
applicationsets.argoproj.io   2025-11-25T16:04:39Z
appprojects.argoproj.io       2025-11-25T16:04:40Z

→ candidate@cnpe3849:~$ k get applications -A
NAMESPACE   NAME         SYNC STATUS   HEALTH STATUS
argocd      web-client   Synced        Healthy

→ candidate@cnpe3849:~$ k -n argocd get application web-client -oyaml > s3_web-client.yaml

→ candidate@cnpe3849:~$ cp s3_web-client.yaml s3_web-client-testing.yaml

→ candidate@cnpe3849:~$ vim s3_web-client-testing.yaml
```

```
# cnpe3849:/home/candidate/s3_web-client-testing.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
```

```
metadata:
  name: web-client-testing      # CHANGE
  namespace: argocd
spec:
  destination:
    namespace: lagoon-testing   # CHANGE
    server: https://kubernetes.default.svc
  project: lagoon
  source:
    path: manifests
    repoURL: git://192.168.100.51/web-client.git
    targetRevision: testing     # CHANGE
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

ℹ️ We removed quite a bit from the copied *Application* manifest, like the whole `status` and some `metadata` fields that will be added by the K8s API or Argo CD controllers. When copying exported manifests, always sanitize read-only fields to avoid validation errors like "unknown field spec.history"

Then we applied the required changes and can create it, but it's always better to diff at first. Because we want a new resource to be created and not replace the existing one because we forgot to change the name for example:

```
➜ candidate@cnpe3849:~$ k -f s3_web-client-testing.yaml diff
diff -u -N /tmp/LIVE-1260665185/argoproj.io.v1alpha1.Application.argocd.web-client-testing /tmp/MERGED-
1252977625/argoproj.io.v1alpha1.Application.argocd.web-client-testing
--- /tmp/LIVE-1260665185/argoproj.io.v1alpha1.Application.argocd.web-client-testing     2025-11-25
20:26:24.722455261 +0000
+++ /tmp/MERGED-1252977625/argoproj.io.v1alpha1.Application.argocd.web-client-testing   2025-11-25
20:26:24.722455261 +0000
@@ -0,0 +1,21 @@
+apiVersion: argoproj.io/v1alpha1
+kind: Application
+metadata:
+  creationTimestamp: "2025-11-25T20:26:24Z"
+  generation: 1
+  name: web-client-testing
+  namespace: argocd
+  uid: 14d4b26f-b9b3-49c5-9f8c-9464622e64c3
+spec:
+  destination:
+    namespace: lagoon-testing
+    server: https://kubernetes.default.svc
+  project: lagoon
+  source:
+    path: manifests
+    repoURL: git://192.168.100.51/web-client.git
+    targetRevision: testing
+  syncPolicy:
+    automated:
+      prune: true
+      selfHeal: true

➜ candidate@cnpe3849:~$ k -f s3_web-client-testing.yaml apply
application.argoproj.io/web-client-testing created
```

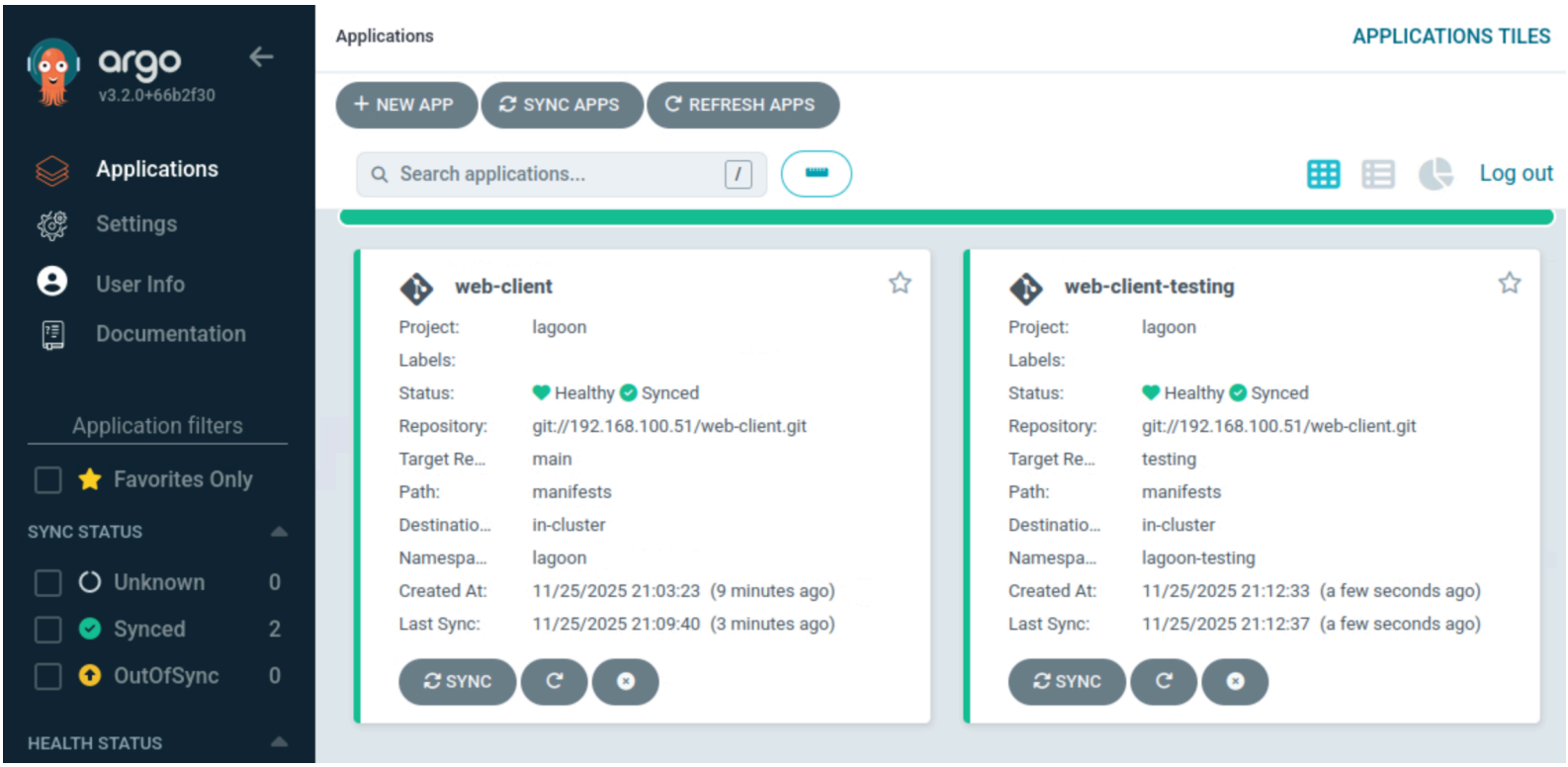
Once create the resource we should see the new application in the web interface. This is because Argo CD internal handles all applications as *CRD Application* resources itself. So it's the same if created via the web interface or manually via for example kubectl.

First it might still be in `OutOfSync` state:

```
➜ candidate@cnpe3849:~$ k get application -A
NAMESPACE   NAME                 SYNC STATUS   HEALTH STATUS
argocd      web-client           Synced        Healthy
argocd      web-client-testing   OutOfSync     Missing
```

But soon, if everything is correctly configured, we should see:

```
➜ candidate@cnpe3849:~$ k get application -A
NAMESPACE   NAME                 SYNC STATUS   HEALTH STATUS
argocd      web-client           Synced        Healthy
argocd      web-client-testing   Synced        Healthy
```



And the result:

```
➜ candidate@cnpe3849:~$ k -n lagoon-testing get pod --show-labels
NAME                          ... STATUS   ...  LABELS
web-client-64cb74749-7gvrt    ... Running  ...  app=web-client,version=v3,...

➜ candidate@cnpe3849:~$ k -n lagoon-testing exec -it web-client-64cb74749-7gvrt -- curl localhost:80
Lagoon Web Client v3
```

The previous application should remain unchanged because it uses Git branch `main`. Here we still see label `v2`:

```
➜ candidate@cnpe3849:~$ k -n lagoon get pod --show-labels
NAME                          ... STATUS   ...  LABELS
web-client-7845db4c66-4hs84   ... Running  ...  app=web-client,version=v2,...
```

▼ Question 4 | Flagger for Blue/Green Deployments

Solve this question on: `ssh cnpe0720`

Flagger is used for two apps In *Namespace* `malawi`. It's configured to perform automated Blue/Green deployments without any service mesh or metrics provider installed.

You can reach `app1` at `http://cnpe0720:30041` and `app2` at `http://cnpe0720:30042`.

1. For *Deployment* `app1`:
    - Increase the patch number of the semantic version in env variable `APP_VERSION` by `1`
    - Write the events triggered on the *Canary* resource into `/course/4/app1.log`
2. For *Deployment* `app2` change the *Canary* resource analysis:
    1. Add a basic `pre-rollout` webhook
    2. It should check if the new *Pods* respond via HTTP
    3. It should use the canary *Service* for this check
    4. You can use the template below

    Once done, trigger a new rollout by setting the `APP_VERSION` to `1.0.1`

```
analysis:
  interval: 5s
  iterations: 2
  metrics: []
  webhooks:
    - name: "basic-http-test"
      type: pre-rollout
      url: http://TODO # DNS to canary service
      timeout: 5s
      metadata:
        type: "http"
        method: "GET"
        expectedStatus: "200"
```

Answer:

Flagger is a progressive delivery controller that automates canary and blue-green deployments by analyzing new versions before promoting them.

Overview

Let's have a look at all the resources in *Namespace* `malawi`:

```
➜ candidate@terminal:~$ ssh cnpe0720

➜ candidate@cnpe0720:~$ k -n malawi get all
NAME                                  READY   STATUS    RESTARTS   AGE
pod/app1-primary-5b5cfc9548-szpmh     1/1     Running   0          28m
pod/app2-primary-6ddfff7b64-72hfl     1/1     Running   0          27m

NAME                  TYPE        ...   PORT(S)        AGE
service/app1          ClusterIP   ...   80/TCP         28m
```

```
service/app1-canary    ClusterIP   ...   80/TCP          28m
service/app1-expose    NodePort    ...   80:30041/TCP    28m
service/app1-primary   ClusterIP   ...   80/TCP          28m
service/app2           ClusterIP   ...   80/TCP          27m
service/app2-canary    ClusterIP   ...   80/TCP          27m
service/app2-expose    NodePort    ...   80:30042/TCP    28m
service/app2-primary   ClusterIP   ...   80/TCP          27m

NAME                           READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/app1           0/0     0            0           28m
deployment.apps/app1-primary   1/1     1            1           28m
deployment.apps/app2           0/0     0            0           28m
deployment.apps/app2-primary   1/1     1            1           27m

NAME                        STATUS         ...
canary.flagger.app/app1     Initialized    ...
canary.flagger.app/app2     Initialized    ...
```

There are two *Deployments* that we manage directly: `app1` and `app2`. Same for the two nodePort *Services* `app1-expose` and `app2-expose` which allow us to access the applications:

```
➜ candidate@cnpe0720:~$ curl http://cnpe0720:30041
app1 version 5.3.8

➜ candidate@cnpe0720:~$ curl http://cnpe0720:30042
app2 version 1.0.0
```

Now because both applications have Flagger *Canary* resources that control them, Flagger will create additional resources:

- The *Deployments* `app1-primary` and `app2-primary`
- The ClusterIP services (`app1`, `app1-primary`, `app1-canary` and the same for `app2`)

In this case we see no `*-canary` *Deployments*, because these are not needed during Blue/Green deployments. But we do see the `*-canary` *Services* because they route to the new *Pods* created by the `app1` *Deployment* during a rollout.

Step 1

We need to change the `APP_VERSION` by increasing the patch version by 1:

```
➜ candidate@terminal:~$ ssh cnpe0720

➜ candidate@cnpe0720:~$ k -n malawi edit deploy app1
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
...
  labels:
    app: app1
  name: app1
  namespace: malawi
spec:
...
  template:
    metadata:
      labels:
        app: app1
    spec:
```

```
      containers:
      - args:
        - |
          echo "app1 version ${APP_VERSION}" > /usr/local/apache2/htdocs/index.html;
          httpd-foreground
        command:
        - /bin/sh
        - -c
        env:
        - name: APP_VERSION
          value: 5.3.9          # UPDATE
        image: httpd:2-alpine
...
```

The same change can be done with:

```
k -n malawi set env deploy/app1 APP_VERSION=5.3.9
```

The correct new version will be `5.3.9`, since semantic versioning follows the `MAJOR.MINOR.PATCH` format.

Before the update we can see that only `app1-primary` has replicas:

```
→ candidate@cnpe0720:~$ k -n malawi get deploy | grep app1
app1           0/0     0            0           49m
app1-primary   1/1     1            1           48m
```

After the update (give it a few seconds) we can see the rollout going on and both have replicas:

```
→ candidate@cnpe0720:~$ k -n malawi get deploy | grep app1
app1           1/1     1            1           50m
app1-primary   1/1     1            1           50m

→ candidate@cnpe0720:~$ k -n malawi get canary
NAME   STATUS        WEIGHT   ...
app1   Progressing   0        ...
app2   Initialized   0        ...
```

And after promotion only `app1-primary` has replicas again and we see the updated new *Pod*:

```
→ candidate@cnpe0720:~$ k -n malawi get deploy | grep app1
app1           0/0     0            0           51m
app1-primary   1/1     1            1           50m

→ candidate@cnpe0720:~$ k -n malawi get pod | grep app1
app1-primary-658c44fbb7-bcq6x   1/1     Running   0          1m32s
```

The result is that the new version is returned and that the status of the `app1` *Canary* changed to `Succeeded`:

```
→ candidate@cnpe0720:~$ curl http://cnpe0720:30041
app1 version 5.3.9

→ candidate@cnpe0720:~$ k -n malawi get canary
NAME   STATUS        WEIGHT   ...
app1   Succeeded     0        ...
app2   Initialized   0        ...
```

The relevant Flagger events for this rollout are:

```
➜ candidate@cnpe0720:~$ k -n malawi describe canary app1
Name:         app1
Namespace:    malawi
...
Status:
  Canary Weight:  0
  Conditions:
    Last Transition Time:  2025-12-01T19:05:36Z
    Last Update Time:      2025-12-01T19:05:36Z
    Message:               Canary analysis completed successfully, promotion finished.
    Reason:                Succeeded
    Status:                True
    Type:                  Promoted
  Failed Checks:           0
  Iterations:              0
  Last Applied Spec:       67969d67
  Last Promoted Spec:      67969d67
  Last Transition Time:    2025-12-01T19:05:36Z
  Phase:                   Succeeded
  Tracked Configs:
Events:
  Type     Reason  Age                From     Message
  ----     ------  ----               ----     -------
  Warning  Synced  58m                flagger  app1-primary.malawi not ready: waiting for rollout to finish:
observed deployment generation less than desired generation
  Normal   Synced  57m (x2 over 58m)  flagger  all the metrics providers are available!
  Normal   Synced  57m                flagger  Initialization done! app1.malawi
  Normal   Synced  8m18s              flagger  New revision detected! Scaling up app1.malawi
  Normal   Synced  8m8s               flagger  Starting canary analysis for app1.malawi
  Normal   Synced  8m8s               flagger  Advance app1.malawi canary iteration 1/2
  Normal   Synced  7m58s              flagger  Advance app1.malawi canary iteration 2/2
  Normal   Synced  7m38s              flagger  Copying app1.malawi template spec to app1-primary.malawi
  Normal   Synced  7m18s              flagger  Promotion completed! Scaling down app1.malawi
```

We copy the events and write them to `/course/4/app1.log`:

```
➜ candidate@cnpe0720:~$ vim /course/4/app1.log
```

```
# cnpe0720:/course/4/app1.log
Normal   Synced  8m18s  flagger  New revision detected! Scaling up app1.malawi
Normal   Synced  8m8s   flagger  Starting canary analysis for app1.malawi
Normal   Synced  8m8s   flagger  Advance app1.malawi canary iteration 1/2
Normal   Synced  7m58s  flagger  Advance app1.malawi canary iteration 2/2
Normal   Synced  7m38s  flagger  Copying app1.malawi template spec to app1-primary.malawi
Normal   Synced  7m18s  flagger  Promotion completed! Scaling down
```

We could get the events also like this:

```
kubectl -n malawi get events | grep canary | grep app1
```

Or:

```
kubectl -n malawi get events --field-selector involvedObject.kind=Canary,involvedObject.name=app1
```

(Optional) Further Investigation

If we check the events then we see these:

1. `New revision detected! Scaling up app1.malawi`
2. `Starting canary analysis for app1.malawi`
3. `Advance app1.malawi canary iteration 1/2`
4. `Advance app1.malawi canary iteration 2/2`
5. `Copying app1.malawi template spec to app1-primary.malawi`
6. `Promotion completed! Scaling down`

These make more sense if we check the *Canary* resource:

```
→ candidate@cnpe0720:~$ k -n malawi get canary app1 -oyaml
```

```yaml
apiVersion: flagger.app/v1beta1
kind: Canary
metadata:
...
  name: app1
  namespace: malawi
spec:
  analysis:              # canary analysis
    interval: 5s         # wait time between checks
    iterations: 2        # two iterations
    metrics: []
    threshold: 10
  provider: kubernetes   # no Linkerd, Istio or another mesh is used
  service:
    port: 80
    portDiscovery: true
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
```

We can see for example the `1/2` and `2/2` of the configured `iterations: 2`.

In this case the only analysis that Flagger does is ensuring that the new canary *Pods* start successfully and remain healthy during the configured iterations.

Using `provider: kubernetes` means Flagger runs in mesh-less Blue/Green mode, which limits it to basic *Pod*-health checks without traffic shifting, ingress-level routing, or built-in metrics analysis.

Step 2

We're required to update the analysis section of the `app2` *Canary* resource to actually call the *Service->Pods* via HTTP. As it is configured right now all Flagger does is checking the *Pod's* health.

Before editing K8s resources in plane we could make a backup:

```
→ candidate@cnpe0720:~$ k -n malawi get canary app2 -oyaml > 4_canary_app2.yaml

→ candidate@cnpe0720:~$ k -n malawi edit canary app2
```

```yaml
# kubectl -n malawi edit canary app2
apiVersion: flagger.app/v1beta1
kind: Canary
metadata:
```

```
...
  name: app2
  namespace: malawi
spec:
  analysis:
    interval: 5s
    iterations: 2
    metrics: []
    threshold: 10
    webhooks:                       # ADD from here
    - name: "basic-http-test"
      type: pre-rollout
      url: http://app2-canary.malawi   # set DNS to canary service
      timeout: 5s
      metadata:
        type: "http"
        method: "GET"
        expectedStatus: "200"       # ADD until here
  provider: kubernetes
  service:
    port: 80
    portDiscovery: true
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: app2
status:
...
```

Using `url: http://app2-canary.malawi` is enough in this case, but if another port should be checked we can specify it like `url: http://app2-canary.malawi:1234`.

Now we need to rollout:

```
→ candidate@cnpe0720:~$ k -n malawi set env deploy/app2 APP_VERSION=1.0.1
deployment.apps/app2 env updated

→ candidate@cnpe0720:~$ k -n malawi get canary,pod
NAME                       STATUS        ...
canary.flagger.app/app1    Succeeded     ...
canary.flagger.app/app2    Progressing   ...

NAME                                 READY   STATUS    RESTARTS   AGE
pod/app1-primary-fd97d8fb-xfhh4      1/1     Running   0          7m49s
pod/app2-59b9b7b55b-swz6h            1/1     Running   0          39s
pod/app2-primary-7b5bbb897b-vmkvd    1/1     Running   0          9m49s
```

We can see the new `app2-59b9b7b55b-swz6h` *Pod* running. After the analysis and promotion:

```
→ candidate@cnpe0720:~$ k -n malawi get canary,pod
NAME                       STATUS      WEIGHT   LASTTRANSITIONTIME
canary.flagger.app/app1    Succeeded   0        2025-12-01T21:54:56Z
canary.flagger.app/app2    Succeeded   0        2025-12-01T22:02:46Z

NAME                                 READY   STATUS    RESTARTS   AGE
pod/app1-primary-fd97d8fb-xfhh4      1/1     Running   0          8m17s
pod/app2-primary-6976c67bf8-7swgv    1/1     Running   0          27s

→ candidate@cnpe0720:~$ curl http://cnpe0720:30042
app2 version 1.0.1
```

Looking great, and we will also see this in the events:

```
➜ candidate@cnpe0720:~$ k -n malawi describe canary app2
Name:         app2
Namespace:    malawi
...
Events:
  Type     Reason  Age                From     Message
  ----     ------  ----               ----     -------
  Normal   Synced  4m26s   flagger  New revision detected! Scaling up app2.malawi
  Normal   Synced  4m16s   flagger  Starting canary analysis for app2.malawi
  Normal   Synced  4m16s   flagger  Pre-rollout check basic-http-test passed
  Normal   Synced  4m16s   flagger  Advance app2.malawi canary iteration 1/2
  Normal   Synced  4m6s    flagger  Advance app2.malawi canary iteration 2/2
  Normal   Synced  3m46s   flagger  Copying app2.malawi template spec to app2-primary.malawi
  Normal   Synced  3m26s   flagger  Promotion completed! Scaling down app2.malawi
```

Here we can see in `Pre-rollout check basic-http-test passed` that our custom webhook worked.

(Optional) Force an error during analysis

Let's see what happens during an error:

```
➜ candidate@cnpe0720:~$ k -n malawi edit canary app2
```

```yaml
# kubectl -n malawi edit canary app2
apiVersion: flagger.app/v1beta1
kind: Canary
metadata:
...
  name: app2
  namespace: malawi
spec:
  analysis:
    interval: 5s
    iterations: 2
    metrics: []
    threshold: 10
    webhooks:
    - name: "basic-http-test"
      type: pre-rollout
      url: http://does-not-exist.malawi    # SET WRONG URL
      timeout: 5s
      metadata:
        type: "http"
        method: "GET"
        expectedStatus: "200"
...
```

```
→ candidate@cnpe0720:~$ k -n malawi set env deploy/app2 APP_VERSION=1.0.2
deployment.apps/app2 env updated

→ candidate@cnpe0720:~$ k -n malawi get canary,pod
NAME                      STATUS        ...
canary.flagger.app/app1   Succeeded     ...
canary.flagger.app/app2   Progressing   ...

NAME                                 READY   STATUS    RESTARTS   AGE
pod/app1-primary-fd97d8fb-xfhh4      1/1     Running   0          14m
pod/app2-69cb8dcfdc-x85pf            1/1     Running   0          21s
pod/app2-primary-6976c67bf8-7swgv    1/1     Running   0          6m41s
```

We can see errors in the events:

```
→ candidate@cnpe0720:~$ k -n malawi describe canary app2
Name:         app2
Namespace:    malawi
...
Events:
  Type     Reason  Age                  From     Message
  ----     ------  ----                 ----     -------
...
  Normal   Synced  50s (x2 over 7m50s)  flagger  New revision detected! Scaling up app2.malawi
  Normal   Synced  0s (x6 over 7m40s)   flagger  Starting canary analysis for app2.malawi
  Warning  Synced  0s (x5 over 39s)     flagger  Halt app2.malawi advancement pre-rollout check basic-http-
test failed POST http://does-not-exist.malawi giving up after 1 attempt(s): Post "http://does-not-
exist.malawi": dial tcp: lookup does-not-exist.malawi on 10.96.0.10:53: no such host
```

And:

```
→ candidate@cnpe0720:~$ k -n malawi get canary,pod
NAME                      STATUS      ...
canary.flagger.app/app1   Succeeded   ...
canary.flagger.app/app2   Failed      ...

NAME                                 READY   STATUS    RESTARTS   AGE
pod/app1-primary-fd97d8fb-xfhh4      1/1     Running   0          16m
pod/app2-primary-6976c67bf8-7swgv    1/1     Running   0          8m17s

→ candidate@cnpe0720:~$ curl http://cnpe0720:30042
app2 version 1.0.1
```

We're still on the old version and that the *Canary* `app2` has status Failed. The events will show that it was tried 10 times because of the configured `threshold: 10`.

To fix this again, first correct the *Canary* resource (fix the webhook URL) and then trigger a new change in the `app2` *Deployment*. For example by bumping `APP_VERSION` or running `kubectl -n malawi rollout restart deploy app2`.

▼ Question 5 | OPA Gatekeeper, Helm

Solve this question on: `ssh cnpe7683`

OPA (Open Policy Agent) Gatekeeper is installed and should be used to validate applications in *Namespace* `planet-apps`.

1. In `/course/5/infra-opa`, complete and create the *ConstraintTemplate*:
    - *Pods* must include label `planet` with any value
    - *Deployments* must define at least `2` replicas
    - Replace the `TODO` placeholders in the violation messages with appropriate text
2. In the same directory, update the *PlanetAppConstraint* so that it applies only to the `planet-apps` *Namespace*, then create it
3. For Helm chart in `/course/5/app-saturn`:
    - Update the *Deployment* manifest to meet the minimal OPA requirements without using Helm values
    - Set the chart version to `1.0.2`
    - Deploy the chart as `app-saturn` in *Namespace* `planet-apps`

Answer:

Helm Chart: Kubernetes YAML template-files combined into a single package, *Values* allow customisation

Helm Release: Installed instance of a *Chart*

Helm Values: Allow to customise the YAML template-files in a *Chart* when creating a *Release*

OPA Constraint Template: A reusable policy with logic and schema for creating specific constraints

OPA Constraint: A concrete instance of a constraint template that applies the defined policy with given parameters to selected Kubernetes resources

OPA Rego: Policy language used by OPA to define and enforce rules over structured data

OPA Gatekeeper: enforces OPA policies in Kubernetes by validating and mutating resources through admission controls

Overview

This question combines OPA and Helm, which can be a little confusing at first, so let's get an overview:

```
➜ ssh cnpe7683

➜ candidate@cnpe7683:~$ find /course/5/ | grep infra
/course/5/infra-opa
/course/5/infra-opa/constraint_template.yaml
/course/5/infra-opa/constraint.yaml
```

These are the OPA resources we need to work with. They are normal YAML files that we can create or delete using kubectl.

```
➜ candidate@cnpe7683:~$ find /course/5/ | grep app
/course/5/app-earth
/course/5/app-earth/Chart.yaml
/course/5/app-earth/values.yaml
/course/5/app-earth/templates
/course/5/app-earth/templates/app.yaml

/course/5/app-venus
/course/5/app-venus/Chart.yaml
/course/5/app-venus/values.yaml
/course/5/app-venus/templates
/course/5/app-venus/templates/app.yaml
```

```
/course/5/app-saturn
/course/5/app-saturn/Chart.yaml
/course/5/app-saturn/values.yaml
/course/5/app-saturn/templates
/course/5/app-saturn/templates/app.yaml
```

Then we have three different, but very similar, apps that are configured as Helm charts. Helm charts can be used or pulled from a remote Helm registry, or like in this case, they can simply be stored in a local folder.

These three Helm charts have already been installed, which we can see if we list all Helm releases in all *Namespaces*:

```
➜ candidate@cnpe7683:~$ helm ls -A
NAME            NAMESPACE       ...   CHART
app-earth       planet-apps     ...   app-earth-chart-1.10.4
app-saturn      planet-apps     ...   app-saturn-chart-1.0.1
app-venus       planet-apps     ...   app-venus-chart-1.10.2
```

We will first change and install the OPA resources, and afterwards adjust and deploy a Helm chart, so that it won't violate the new rules.

Step 1: *ConstraintTemplate*

In this step we need to finish the existing *ConstraintTemplate*:

```
➜ candidate@cnpe7683:~$ vim /course/5/infra-opa/constraint_template.yaml
```

```
# cnpe7683:/course/5/infra-opa/constraint_template.yaml
...
package planetappconstraint

violation[{"msg": msg}] {
  input.review.kind.kind == "Pod"
  not input.review.object.metadata.labels.TODO    # CHANGE
  msg := "Pod is missing required label: TODO"    # CHANGE
}

violation[{"msg": msg}] {
  input.review.kind.kind == "Deployment"
  replicas := input.review.object.spec.replicas
  replicas < 10                                   # CHANGE replicas and "TODO" text
  msg := sprintf("Deployment requires at least TODO replicas, found %v", [replicas])
}
```

Above we only look at the Rego section in the file where we need to do four changes. Which results in:

```
# cnpe7683:/course/5/infra-opa/constraint_template.yaml
apiVersion: templates.gatekeeper.sh/v1
kind: ConstraintTemplate
metadata:
  name: planetappconstraint
spec:
  crd:
    spec:
      names:
        kind: PlanetAppConstraint
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package planetappconstraint
```

```
        violation[{"msg": msg}] {
          input.review.kind.kind == "Pod"
          not input.review.object.metadata.labels.planet
          msg := "Pod is missing required label: planet"
        }

        violation[{"msg": msg}] {
          input.review.kind.kind == "Deployment"
          replicas := input.review.object.spec.replicas
          replicas < 2
          msg := sprintf("Deployment requires at least two replicas, found %v", [replicas])
        }
```

We need to understand the Rego language, which should be possible for anyone who has some experience with programming languages.

The first violation checks if the resource is a *Pod*. Rego can access the *Pod* resource via `input.review.object` and then checks if the label `planet` is available. The label value is ignored.

The second violation checks if the resource is a *Deployment*. Now Rego can access the *Deployment* resource via `input.review.object` and this allows to check the replicas.

```
➜ candidate@cnpe7683:~$ k apply -f /course/5/infra-opa/constraint_template.yaml
constrainttemplate.templates.gatekeeper.sh/planetappconstraint created
```

Once we create the *ConstraintTemplate*, OPA Gatekeeper will create a *CRD* for us:

```
➜ candidate@cnpe7683:~$ k get crd
NAME                                                  CREATED AT
assign.mutations.gatekeeper.sh                        2025-11-19T12:25:03Z
assignimage.mutations.gatekeeper.sh                   2025-11-19T12:25:03Z
assignmetadata.mutations.gatekeeper.sh                2025-11-19T12:25:04Z
configpodstatuses.status.gatekeeper.sh                2025-11-19T12:25:04Z
configs.config.gatekeeper.sh                          2025-11-19T12:25:05Z
connectionpodstatuses.status.gatekeeper.sh            2025-11-19T12:25:05Z
connections.connection.gatekeeper.sh                  2025-11-19T12:25:05Z
constraintpodstatuses.status.gatekeeper.sh            2025-11-19T12:25:05Z
constrainttemplatepodstatuses.status.gatekeeper.sh    2025-11-19T12:25:05Z
constrainttemplates.templates.gatekeeper.sh           2025-11-19T12:25:05Z
expansiontemplate.expansion.gatekeeper.sh             2025-11-19T12:25:05Z
expansiontemplatepodstatuses.status.gatekeeper.sh     2025-11-19T12:25:06Z
modifyset.mutations.gatekeeper.sh                     2025-11-19T12:25:06Z
mutatorpodstatuses.status.gatekeeper.sh               2025-11-19T12:25:07Z
planetappconstraint.constraints.gatekeeper.sh         2025-11-19T14:36:00Z
providers.externaldata.gatekeeper.sh                  2025-11-19T12:25:07Z
syncsets.syncset.gatekeeper.sh                        2025-11-19T12:25:07Z

➜ candidate@cnpe7683:~$ k get planetappconstraint
No resources found
```

Above we see quite some *CRDs* that OPA Gatekeeper provides with its default installation. But we also see our custom one for `planetappconstraint` (or `PlanetAppConstraint`), just as it was defined in `constraint_template.yaml`.

So far there are no resources yet for `planetappconstraint`, which we'll create in the next step.

Step 2: Constraint

Now we create a new resource of *PlanetAppConstraint*, but first we need to make a small change:

```
➜ candidate@cnpe7683:~$ vim /course/5/infra-opa/constraint.yaml
```

```yaml
# cnpe7683:/course/5/infra-opa/constraint.yaml
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: PlanetAppConstraint
metadata:
  name: planet-app-constraint
spec:
  match:
    namespaces:
      - planet-apps         # SET NAMESPACE
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
      - apiGroups: ["apps"]
        kinds: ["Deployment"]
```

A constraint is a resource of the *CRD*, that was created by a *ConstraintTemplate*. Here we can now define where and to which resources the policies/rules should be applied. In this case, it applies to all *Pods* and *Deployments* in *Namespace* `planet-apps`.

If we left the `namespaces:` section out completely, the rules would be applied **cluster-wide**, which can cause serious interruptions if not properly tested.

We apply the resource:

```
➜ candidate@cnpe7683:~$ k apply -f /course/5/infra-opa/constraint.yaml
planetappconstraint.constraints.gatekeeper.sh/planet-app-constraint created

➜ candidate@cnpe7683:~$ k get planetappconstraint
NAME                    ENFORCEMENT-ACTION   TOTAL-VIOLATIONS
planet-app-constraint   deny
```

(Optional) View existing violations

If we give OPA a few seconds, we should see some violations displayed for running resources, *Pods* or *Deployments* in our case.

```
    ➜ candidate@cnpe7683:~$ k get planetappconstraint
NAME                    ENFORCEMENT-ACTION   TOTAL-VIOLATIONS
planet-app-constraint   deny                 2
```

It's **important** to understand that running resources will **not** be affected by newly created or updated OPA constraints. But if we for example want to rollout restart a *Deployment* that now violates implemented OPA policies, new *Pods* will **not** be created. We can view the violation details:

```
➜ candidate@cnpe7683:~$ k describe planetappconstraint planet-app-constraint
Name:         planet-app-constraint
Namespace:
Labels:       <none>
Annotations:  <none>
API Version:  constraints.gatekeeper.sh/v1beta1
Kind:         PlanetAppConstraint
...
Status:
...
  Total Violations:  2
  Violations:
    Enforcement Action:  deny
```

```
    Group:                apps
    Kind:                 Deployment
    Message:              Deployment requires at least two replicas, found 1
    Name:                 app-saturn
    Namespace:            planet-apps
    Version:              v1
    Enforcement Action:   deny
    Group:
    Kind:                 Pod
    Message:              Pod is missing required label: planet
    Name:                 app-saturn-58759b57d-fzs5g
    Namespace:            planet-apps
    Version:              v1
```

Above we can see that there is one violation for *Deployment* `app-saturn` because of the minimum replicas requirement. The other violation is for a *Pod* of the *Deployment* because it is missing the `planet` label.

We can also see our adjusted violation messages work.

(Optional) Test rules manually

Creating a *Pod* without label `planet` fails:

```
➜ candidate@cnpe7683:~$ candidate@cnpe7683:~$ k -n planet-apps run test --image=nginx:alpine
Error from server (Forbidden): admission webhook "validation.gatekeeper.sh" denied the request: [planet-app-constraint] Pod is missing required label: planet

➜ candidate@cnpe7683:~$ k -n planet-apps run test --image=nginx:alpine --labels planet=test
pod/test created

➜ candidate@cnpe7683:~$ k -n planet-apps delete pod test
pod "test" deleted from planet-apps namespace
```

Creating a *Deployment* without required minimum of replicas fails:

```
➜ candidate@cnpe7683:~$ k -n planet-apps create deploy test --image=nginx:alpine
error: failed to create deployment: admission webhook "validation.gatekeeper.sh" denied the request: [planet-app-constraint] Deployment requires at least two replicas, found 1

➜ candidate@cnpe7683:~$ k -n planet-apps create deploy test --image=nginx:alpine --replicas 2
deployment.apps/test created
```

Above we can see that the *Deployment* was created once we set the required amount of replicas, **but** the *Pods* that it would create don't have the required labels. Hence the *Deployment* can't create *Pods* till this is fix or the OPA rule is changed:

```
➜ candidate@cnpe7683:~$ k -n planet-apps get deploy test
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
test   0/2     0            0           26s

➜ candidate@cnpe7683:~$ k -n planet-apps describe deploy test
Name:                   test
Namespace:              planet-apps
...
Conditions:
  Type            Status  Reason
  ----            ------  ------
  Progressing     True    NewReplicaSetCreated
```

```
  Available        False   MinimumReplicasUnavailable
  ReplicaFailure   True    FailedCreate
OldReplicaSets:    <none>
NewReplicaSet:     test-6fbdb6cff (0/2 replicas created)
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ----  ----                   -------
  Normal  ScalingReplicaSet  34s   deployment-controller  Scaled up replica set test-6fbdb6cff from 0 to 2

➜ candidate@cnpe7683:~$ k -n planet-apps delete deploy test
deployment.apps "test" deleted from planet-apps namespace
```

The above behavior is what we want to see from our implemented constraint and template!

Step 3: Adjust and deploy Helm chart

We see existing violations for the *Deployment* `app-saturn`:

```
➜ candidate@cnpe7683:~$ k describe planetappconstraint planet-app-constraint
...
  Total Violations:  2
  Violations:
    Enforcement Action:  deny
    Group:               apps
    Kind:                Deployment
    Message:             Deployment requires at least two replicas, found 1
    Name:                app-saturn
    Namespace:           planet-apps
    Version:             v1
    Enforcement Action:  deny
    Group:
    Kind:                Pod
    Message:             Pod is missing required label: planet
    Name:                app-saturn-58759b57d-fzs5g
    Namespace:           planet-apps
    Version:             v1
```

This *Deployment* is created by the Helm chart in `/course/5/app-saturn`, which we are required to fix according to this question.

First we'll update the Helm chart version:

```
➜ candidate@cnpe7683:~$ vim /course/5/app-saturn/Chart.yaml
```

```
# cnpe7683:/course/5/app-saturn/Chart.yaml
apiVersion: v2
name: app-saturn-chart
description: Minimal application chart
type: application
version: 1.0.2    # UPDATE
```

Next we raise the *Deployment* replicas and add the *Pod* labels:

```
➜ candidate@cnpe7683:~$ vim /course/5/app-saturn/templates/app.yaml
```

```
# cnpe7683:/course/5/app-saturn/templates/app.yaml
apiVersion: apps/v1
kind: Deployment
```

```
metadata:
  name: app-saturn
spec:
  replicas: 2            # CHANGE
  selector:
    matchLabels:
      app: app-saturn
  template:
    metadata:
      labels:
        planet: saturn   # ADD
        app: app-saturn
    spec:
      containers:
        - image: nginx:1-alpine
          name: app
          resources:
            requests:
              cpu: 20m
              memory: 20Mi
...
```

Finally we need to install our updated via Helm chart:

```
➜ candidate@cnpe7683:~$ cd /course/5/app-saturn

➜ candidate@cnpe7683:/course/5/app-saturn$ helm -n planet-apps ls
NAME            NAMESPACE       ...   CHART
app-earth       planet-apps     ...   app-earth-chart-1.10.4
app-saturn      planet-apps     ...   app-saturn-chart-1.0.1
app-venus       planet-apps     ...   app-venus-chart-1.10.2
```

Above we see Helm release `app-saturn` in version `1.0.1`. Now to upgrade:

```
➜ candidate@cnpe7683:/course/5/app-saturn$ helm upgrade -n planet-apps app-saturn .
Release "app-saturn" has been upgraded. Happy Helming!
NAME: app-saturn
LAST DEPLOYED: Wed Nov 19 15:30:45 2025
NAMESPACE: planet-apps
STATUS: deployed
REVISION: 2
DESCRIPTION: Upgrade complete
TEST SUITE: None

➜ candidate@cnpe7683:/course/5/app-saturn$ helm -n planet-apps ls
NAME            NAMESPACE       ...   CHART
app-earth       planet-apps     ...   app-earth-chart-1.10.4
app-saturn      planet-apps     ...   app-saturn-chart-1.0.2
app-venus       planet-apps     ...   app-venus-chart-1.10.2
```

We can now see the Helm release in version `1.0.2`, awesome!

```
➜ candidate@cnpe7683:~$ k get planetappconstraint
NAME                     ENFORCEMENT-ACTION   TOTAL-VIOLATIONS
planet-app-constraint    deny                 0

➜ candidate@cnpe7683:~$ k -n planet-apps get deploy
NAME         READY   UP-TO-DATE   AVAILABLE   AGE
app-earth    2/2     2            2           149m
app-saturn   2/2     2            2           148m
app-venus    2/2     2            2           148m
```

Also no more violations and all *Deployments* have their replicas running, great!

OPA Gatekeeper uses the **Operator pattern**, which is a combination of custom *CRDs* and controllers that manage them. It can be quite complex but it allows for highly customisable and extensible applications on top of Kubernetes.

▼ Question 6 | OpenTofu, Terraform

Solve this question on: `ssh cnpe4328`

Your platform team uses OpenTofu/Terraform to manage Kubernetes resources.

Perform the following, command `tofu` ready to be used:

1. For `/course/6/service-black-bean` create a human-readable diff output of the changes that would be applied and store it at `/course/6/service-black-bean/diff.txt`
2. For `/course/6/service-green-curry` raise the replicas for the deployment resource `green-curry` to `2` and apply the change
3. Update `/course/6/service-red-velvet/main.tf`:
    - Add a new `NodePort` *Service* named `cake`, nodePort `30060`
    - The OpenTofu/Terraform resource name should also be named `cake`
    - It should point to the existing *Deployment* `red-velvet`

Answer:

OpenTofu is an infrastructure as code tool under the Linux Foundation's stewardship that serves as a drop-in replacement for Terraform. It's not too common to use it for managing everyday Kubernetes resources like in this scenario, but it's possible. Much more common to use Helm or Kustomize instead.

Introduction

OpenTofu/Terraform manages resources, Kubernetes resources in this case, by comparing the desired state to the current state. If we look into one of the directories we can see some files:

```
➜ ssh cnpe4328

➜ candidate@cnpe4328:~$ cd /course/6/service-black-bean

➜ candidate@cnpe4328:/course/6/service-black-bean$ ls
main.tf  terraform.tfstate  terraform.tfstate.backup
```

File `main.tf` contains the OpenTofu/Terraform configuration that defines the resources to manage. It's common to have multiple `.tf` files in the same directory for better separation and readability. All `.tf` files in the directory are loaded together and treated as a single configuration during `tofu plan` and `tofu apply`.

File `terraform.tfstate` stores the current known state of the managed infrastructure so Terraform/OpenTofu knows what exists.

File `terraform.tfstate.backup` contains the previous state file kept automatically as a backup before the last change.

State management

OpenTofu/Terraform allows the state to be remote as well, stored in backends such as S3, Consul, or other supported storage systems instead of the local filesystem. The state contains sensitive data like tokens and passwords. This is why remote storage that handles security, locking, and consistency is highly encouraged.

Helm is another stateful manager of Kubernetes resources which stores release information (functionally acting as its state) in the cluster (as *Secrets* or *ConfigMaps*). Helm knows which resources it created because of the release information.

Kustomize on the other hand does not track the state of applied resources, which is by design. This means that Kustomize cannot delete resources on its own, because it does not know which resources it created or whether they may have been created by something else.

Step 1

We are asked to generate a diff of changes that would be applied:

```
→ candidate@cnpe4328:/course/6/service-black-bean$ tofu plan
kubernetes_service.test-service: Refreshing state... [id=baikal/test-service]
kubernetes_deployment.black-bean: Refreshing state... [id=baikal/black-bean]

OpenTofu used the selected providers to generate the following execution plan. Resource actions are
indicated with the following symbols:
  + create
  - destroy

OpenTofu will perform the following actions:

  # kubernetes_service.black-bean will be created
  + resource "kubernetes_service" "black-bean" {
      + id                     = (known after apply)
      + status                 = (known after apply)
      + wait_for_load_balancer = true

      + metadata {
          + generation       = (known after apply)
          + labels           = {
              + "app" = "black-bean"
            }
          + name             = "black-bean"
          + namespace        = "baikal"
          + resource_version = (known after apply)
          + uid              = (known after apply)
        }

      + spec {
          + allocate_load_balancer_node_ports = true
          + cluster_ip                        = (known after apply)
          + cluster_ips                       = (known after apply)
          + external_traffic_policy           = (known after apply)
```

```
      + health_check_node_port            = (known after apply)
      + internal_traffic_policy           = (known after apply)
      + ip_families                       = (known after apply)
      + ip_family_policy                  = (known after apply)
      + publish_not_ready_addresses       = false
      + selector                          = {
          + "app" = "black-bean"
        }
      + session_affinity                  = "None"
      + type                              = "ClusterIP"

      + port {
          + node_port   = (known after apply)
          + port        = 80
          + protocol    = "TCP"
          + target_port = "80"
        }

      + session_affinity_config (known after apply)
    }
  }

# kubernetes_service.test-service will be destroyed
# (because kubernetes_service.test-service is not in configuration)
- resource "kubernetes_service" "test-service" {
    - id                     = "baikal/test-service" -> null
    - status                 = [
        - {
            - load_balancer = [
                - {
                    - ingress = []
                  },
              ]
          },
      ] -> null
    - wait_for_load_balancer = true -> null

    - metadata {
        - annotations      = {} -> null
        - generation       = 0 -> null
        - labels           = {
            - "app" = "test"
          } -> null
        - name             = "test-service" -> null
        - namespace        = "baikal" -> null
        - resource_version = "37151" -> null
        - uid              = "9cb9c20b-8069-43a6-8fc9-35e381623711" -> null
      }

    - spec {
        - allocate_load_balancer_node_ports = true -> null
        - cluster_ip                        = "10.106.37.84" -> null
        - cluster_ips                       = [
            - "10.106.37.84",
          ] -> null
        - external_ips                      = [] -> null
        - health_check_node_port            = 0 -> null
        - internal_traffic_policy           = "Cluster" -> null
        - ip_families                       = [
```

```
            - "IPv4",
          ] -> null
        - ip_family_policy                 = "SingleStack" -> null
        - load_balancer_source_ranges      = [] -> null
        - publish_not_ready_addresses      = false -> null
        - selector                         = {
            - "app" = "test"
          } -> null
        - session_affinity                 = "None" -> null
        - type                             = "ClusterIP" -> null

        - port {
            - node_port   = 0 -> null
            - port        = 8080 -> null
            - protocol    = "TCP" -> null
            - target_port = "8080" -> null
          }
      }
  }

Plan: 1 to add, 0 to change, 1 to destroy.

─────────────────────────────────────────────────────────────────────────────
─────────────────────────────────────────────────

Note: You didn't use the -out option to save this plan, so OpenTofu can't guarantee to take exactly these
actions if you run "tofu apply" now.
```

This shows all the actions, in short:

```
Plan: 1 to add, 0 to change, 1 to destroy.
```

We can see that one *Service* would be created and one would be deleted. Now to solve the step:

```
→ candidate@cnpe4328:/course/6/service-black-bean$ tofu plan > diff.txt
```

Using `tofu plan -out=diff.txt` would be wrong here because it writes a binary plan file, not the human-readable text diff that the question requires.

Step 2

Here we need to change the replicas and apply. If we look at the current situation then we see:

```
→ candidate@cnpe4328:~$ cd /course/6/service-green-curry

→ candidate@cnpe4328:/course/6/service-green-curry$ vim main.tf
```

```
...
locals {
  namespace = "baikal"
}

resource "kubernetes_deployment" "green-curry" {
  metadata {
    name      = "green-curry"
    namespace = local.namespace
```

```
    labels = { app = "green-curry" }
  }

  spec {
    replicas = 2    # UPDATE

    selector {
      match_labels = { app = "green-curry" }
    }

    template {
      metadata {
        labels = { app = "green-curry" }
      }

      spec {
        container {
          name  = "nginx"
          image = "nginx:1-alpine"

          port {
            container_port = 80
          }
        }
      }
    }
  }
}
...
```

Looking at the content we can see that resources are managed in *Namespace* `baikal`. Let's see what happens before and after we apply:

```
→ candidate@cnpe4328:/course/6/service-green-curry$ k -n baikal get deploy
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
black-bean    1/1     1            1           132m
green-curry   0/0     0            0           132m
red-velvet    1/1     1            1           132m

→ candidate@cnpe4328:/course/6/service-green-curry$ tofu apply
kubernetes_service.green-curry: Refreshing state... [id=baikal/green-curry]
kubernetes_deployment.green-curry: Refreshing state... [id=baikal/green-curry]

OpenTofu used the selected providers to generate the following execution plan. Resource actions are
indicated with the following symbols:
  ~ update in-place

OpenTofu will perform the following actions:

  # kubernetes_deployment.green-curry will be updated in-place
  ~ resource "kubernetes_deployment" "green-curry" {
        id               = "baikal/green-curry"
        # (1 unchanged attribute hidden)

      ~ spec {
          ~ replicas                  = "0" -> "2"
            # (4 unchanged attributes hidden)

            # (3 unchanged blocks hidden)
```

```
        }

        # (1 unchanged block hidden)
    }

Plan: 0 to add, 1 to change, 0 to destroy.

Do you want to perform these actions?
  OpenTofu will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

kubernetes_deployment.green-curry: Modifying... [id=baikal/green-curry]
kubernetes_deployment.green-curry: Modifications complete after 8s [id=baikal/green-curry]

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

→ candidate@cnpe4328:/course/6/service-green-curry$ k -n baikal get deploy
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
black-bean    1/1     1            1           135m
green-curry   2/2     2            2           135m
red-velvet    1/1     1            1           135m
```

The plan clearly shows the change from `0` to `2` replicas, and kubectl confirms the *Deployment* was updated.

Step 3

Here we need to do a slightly larger change. The idea is that we copy the code from the existing service section and just change what's needed:

```
→ candidate@cnpe4328:~$ cd /course/6/service-red-velvet

→ candidate@cnpe4328:/course/6/service-red-velvet$ vim main.tf
```

```
...
resource "kubernetes_service" "red-velvet" {
  metadata {
    name      = "red-velvet"
    namespace = local.namespace
    labels = { app = "red-velvet" }
  }

  spec {
    selector = { app = "red-velvet" }

    port {
      port        = 80
      target_port = 80
      protocol    = "TCP"
    }

    type = "ClusterIP"
  }
}

# COPIED kubernetes_service resource from above
```

```
resource "kubernetes_service" "cake" {       # USE resource name "cake"
  metadata {
    name      = "cake"                       # update
    namespace = local.namespace

    # The Service has its own label app="cake" but it selects the red-velvet Pods via selector app="red-
velvet" further down
    labels = { app = "cake" }                # update
  }

  spec {
    selector = { app = "red-velvet" }        # still selects the Deployment

    port {
      port        = 80
      target_port = 80
      node_port   = 30060                    # add
      protocol    = "TCP"
    }

    type = "NodePort"                        # update
  }
}
```

Now we should see and be able to apply the changes:

```
➜ candidate@cnpe4328:/course/6/service-red-velvet$ tofu apply
kubernetes_service.red-velvet: Refreshing state... [id=baikal/red-velvet]
kubernetes_deployment.red-velvet: Refreshing state... [id=baikal/red-velvet]

OpenTofu used the selected providers to generate the following execution plan. Resource actions are
indicated with the following symbols:
  + create

OpenTofu will perform the following actions:

  # kubernetes_service.cake will be created
  + resource "kubernetes_service" "cake" {
      + id                     = (known after apply)
      + status                 = (known after apply)
      + wait_for_load_balancer = true

      + metadata {
          + generation       = (known after apply)
          + labels           = {
              + "app" = "cake"
            }
          + name             = "cake"
          + namespace        = "baikal"
          + resource_version = (known after apply)
          + uid              = (known after apply)
        }

      + spec {
          + allocate_load_balancer_node_ports = true
          + cluster_ip                        = (known after apply)
          + cluster_ips                       = (known after apply)
          + external_traffic_policy           = (known after apply)
          + health_check_node_port            = (known after apply)
```

```
          + internal_traffic_policy           = (known after apply)
          + ip_families                       = (known after apply)
          + ip_family_policy                  = (known after apply)
          + publish_not_ready_addresses       = false
          + selector                          = {
              + "app" = "red-velvet"
            }
          + session_affinity                  = "None"
          + type                              = "NodePort"

          + port {
              + node_port   = 30060
              + port        = 80
              + protocol    = "TCP"
              + target_port = "80"
            }

          + session_affinity_config (known after apply)
        }
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  OpenTofu will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

kubernetes_service.cake: Creating...
kubernetes_service.cake: Creation complete after 0s [id=baikal/cake]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

And the result:

```
→ candidate@cnpe4328:/course/6/service-red-velvet$ k -n baikal get svc
NAME           TYPE        ...   PORT(S)        AGE
cake           NodePort    ...   80:30060/TCP   29s
green-curry    ClusterIP   ...   80/TCP         147m
red-velvet     ClusterIP   ...   80/TCP         146m
test-service   ClusterIP   ...   8080/TCP       147m

→ candidate@cnpe4328:/course/6/service-red-velvet$ k -n baikal describe service cake
Name:                     cake
Namespace:                baikal
Labels:                   app=cake
Annotations:              <none>
Selector:                 app=red-velvet
Type:                     NodePort
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.97.255.73
IPs:                      10.97.255.73
Port:                     <unset>  80/TCP
TargetPort:               80/TCP
NodePort:                 <unset>  30060/TCP
Endpoints:                10.32.0.9:80
Session Affinity:         None
```

```
External Traffic Policy:  Cluster
Internal Traffic Policy:  Cluster
Events:                   <none>
```

We can see the new *Service* and also that it has one *Endpoint*, which means it maps properly to the existing *Pods* via labels.

▼ Question 7 | OpenCost, Prometheus

Solve this question on: `ssh cnpe1080`

OpenCost is installed with web interface at `http://cnpe1080:30070` and `kubectl cost` configured. OpenCost uses Prometheus for data and storage, reachable at `http://cnpe1080:30077`:

1. Update the OpenCost custom pricing model in *Namespace* `opencost`:
   - Set `internetNetworkEgress` to `0.25`
   - Set `spotCPU` to `0.015`

   Ensure OpenCost is working with the updated values.
2. Run Prometheus query `kube_pod_info{...}` filtered by *Namespace* `atlantic` and write the result into `/course/7/result.txt`
3. Find Prometheus targets with scraping errors and write their error message into `/course/7/error.txt`

ℹ️ To extract information from the Prometheus UI, simply highlight the text in Firefox, copy it, and paste it into the file

ℹ️ To use `kubectl cost` with OpenCost the `--opencost` argument might be needed

Answer:

OpenCost is a vendor-neutral open source project for measuring and allocating cloud infrastructure and container costs in real time. OpenCost requires Prometheus for scraping metrics and data storage.

Step 1: Update OpenCost custom pricing model

OpenCost uses a *ConfigMap* to store custom pricing configuration. First we need to find which *ConfigMap* is used. We can check the OpenCost *Deployment* for volume mounts:

```
➜ ssh cnpe1080
```

```
➜ candidate@cnpe1080:~$ k -n opencost get pod,cm
NAME                              READY   STATUS    RESTARTS      AGE
pod/opencost-795795d968-h67tp     2/2     Running   2 (22m ago)   22m

NAME                                  DATA   AGE
configmap/custom-pricing-model        1      25m
configmap/kube-root-ca.crt            1      25m
configmap/opencost-install-plugins    1      25m
configmap/opencost-ui-nginx-config    1      25m
```

We see `custom-pricing-model`, alternatively we could investigate the *Deployment* for custom config settings:

```
➜ candidate@cnpe1080:~$ k -n opencost get deploy opencost -oyaml
```

```yaml
# kubectl -n opencost get deploy opencost -oyaml
apiVersion: apps/v1
kind: Deployment
metadata:
...
  name: opencost
  namespace: opencost
spec:
...
  template:
...
    spec:
      automountServiceAccountToken: true
      containers:
      - env:
...
        - name: CLUSTER_ID
          value: default-cluster
        - name: CONFIG_PATH                             # CONFIG path
          value: /tmp/custom-config
...
        name: opencost
        volumeMounts:
        - mountPath: /opt/opencost/plugin
          name: plugins-dir
        - mountPath: /tmp/custom-config/default.json    # CONFIG volume mount
          name: custom-configs
          readOnly: true
          subPath: default.json
...
      volumes:
      - name: plugins-config
        secret:
          defaultMode: 420
          secretName: opencost-plugins-config
      - configMap:                                      # CONFIG volume
          defaultMode: 420
          name: custom-pricing-model
        name: custom-configs
...
```

We update the *ConfigMap* with the required values:

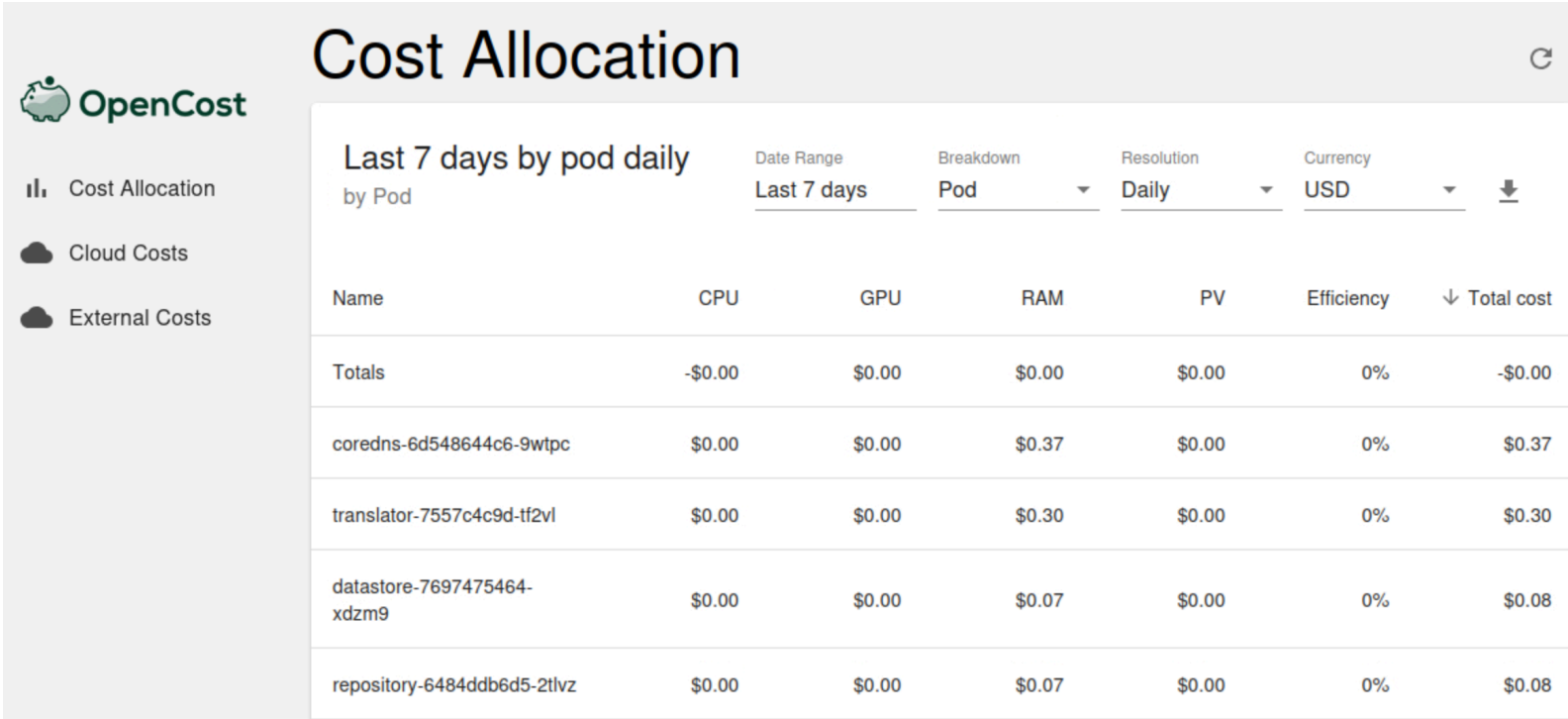
```
➜ candidate@cnpe1080:~$ k -n opencost edit cm custom-pricing-model
```

```
# kubectl -n opencost edit cm custom-pricing-model
apiVersion: v1
data:
  default.json: |-
    {
      "CPU": "1.25",
      "GPU": "0.95",
      "RAM": "0.5",
      "cpu": "0.03",
      "description": "Modified pricing configuration.",
      "gpu": "0.90",
      "internetNetworkEgress": "0.25",    # UPDATE
      "nodes": "map[default:map[hourlyCost:0.10 storagePerGiB:20.00]]",
      "ram": "10.00",
      "regionNetworkEgress": "0.01",
      "spot": "map[enabled:false]",
      "spotCPU": "0.015",                 # UPDATE
      "spotRAM": "0.000892",
      "storage": "20.00",
      "zoneNetworkEgress": "0.01",
      "provider" : "custom"
    }
kind: ConfigMap
metadata:
  name: custom-pricing-model
  namespace: opencost
...
```

After saving, the *ConfigMap* is updated but OpenCost won't automatically reload the configuration. We need to restart the *Deployment*:

```
→ candidate@cnpe1080:~$ k -n opencost rollout restart deploy opencost
deployment.apps/opencost restarted

→ candidate@cnpe1080:~$ k -n opencost rollout status deploy opencost
Waiting for deployment "opencost" rollout to finish: 0 of 1 updated replicas are available...
deployment "opencost" successfully rolled out
```

We would probably not see any changes on actual costs with these changes, but in general the OpenCost web interface can be used with the correct filters set:



Same can be achieved via the command line:

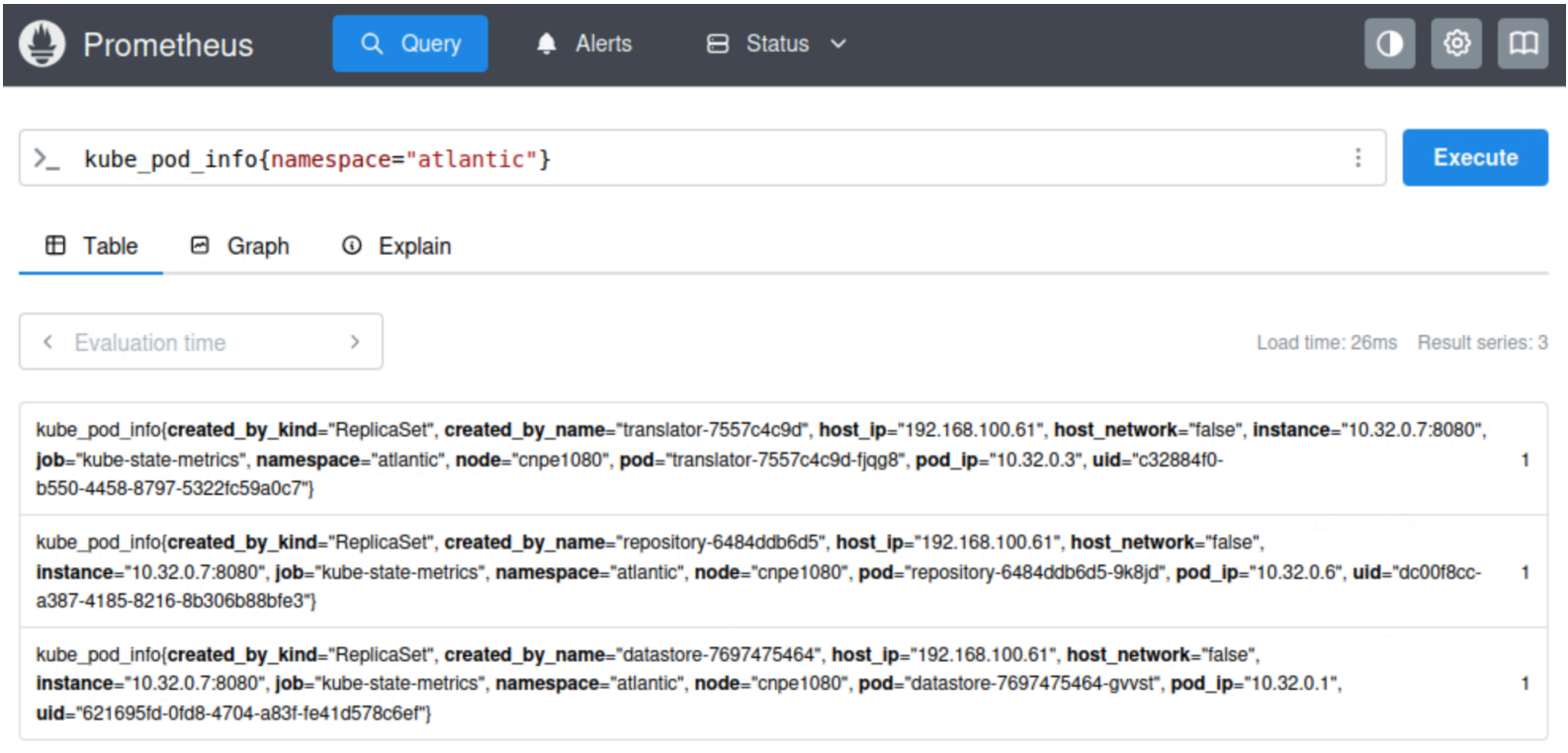
```
→ candidate@cnpe1080:~$ kubectl cost pod --opencost
...------------+------------------------------+--------------...
... NAMESPACE   | POD                          | MONTHLY RATE ...
...------------+------------------------------+--------------...
... opencost    | opencost-656b7b87bd-kdkdm    |   774.493309 ...
...             | opencost-795795d968-bbnf8    |   773.884800 ...
...             | opencost-795795d968-h67tp    |   773.867520 ...
... kube-system | etcd-cnpe1080                |   703.787950 ...
...             | coredns-b866c6688-wgstj      |   352.703061 ...
...             | coredns-b866c6688-j687p      |   352.703061 ...
... atlantic    | translator-7557c4c9d-fjqg8   |   281.746246 ...
...             | translator-7557c4c9d-5bgq7   |   281.664000 ...
...
...------------+------------------------------+--------------...
...             |                              |  4722.867222 ...
...------------+------------------------------+--------------...
```

Step 2: Prometheus run query

We open the Prometheus web interface in Firefox and run the query:

```
kube_pod_info{namespace="atlantic"}
```

If we don't know for which labels we need to filter, like `namespace` in this case, we could first run the query without any labels. Then look at the results for available ones. The web interface should also offer auto complete which can be very helpful.



We see three results for the three *Pods* in *Namespace* `atlantic`. Now we just need to copy the results into `/course/7/result.txt`.

```
→ candidate@cnpe1080:~$ vim /course/7/result.txt
```

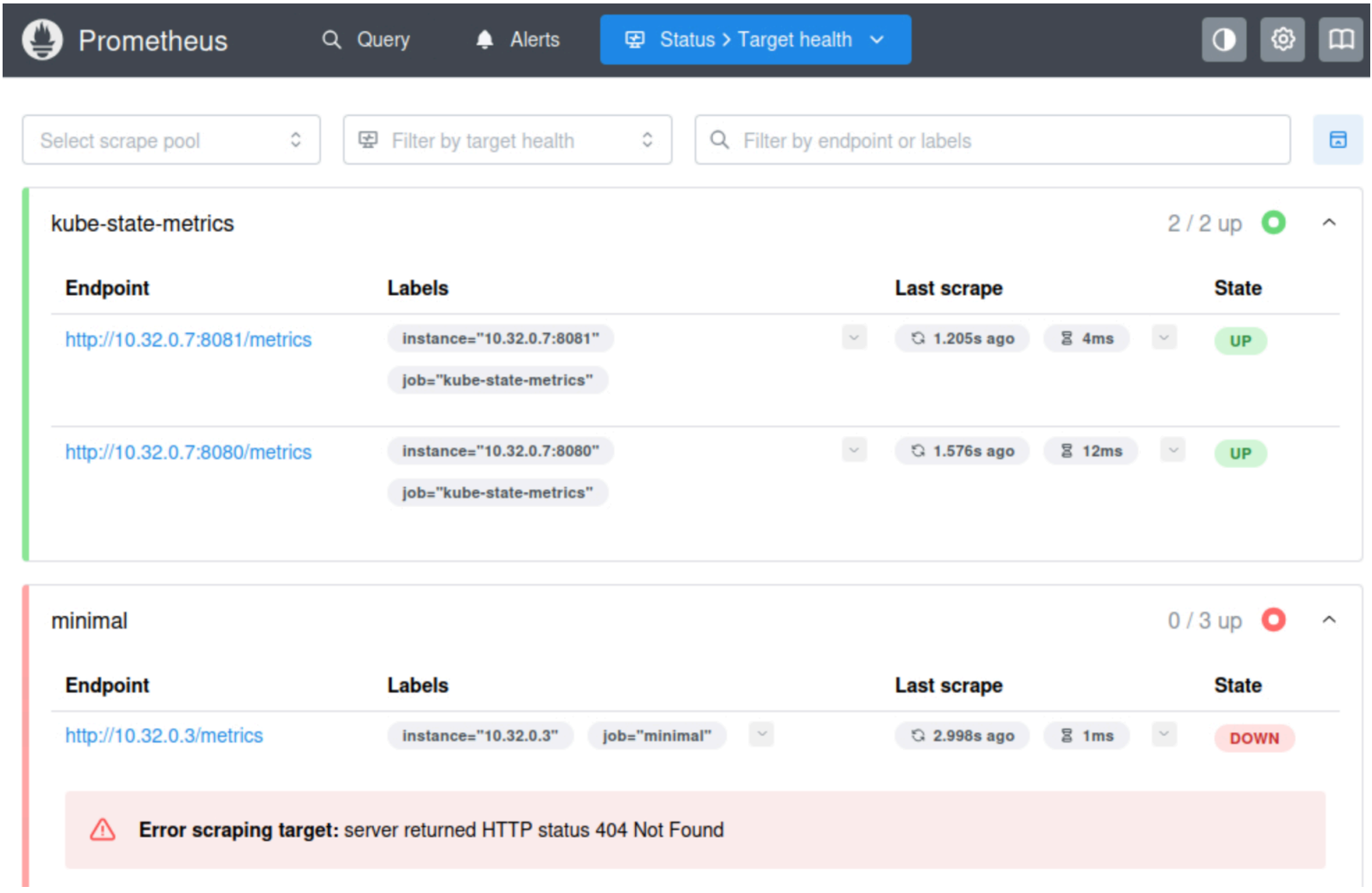
```
# cnpe1080:/course/7/result.txt
kube_pod_info{created_by_kind="ReplicaSet", created_by_name="translator-7557c4c9d", host_ip="192.168.100.61",
host_network="false", instance="10.32.0.7:8080", job="kube-state-metrics", namespace="atlantic",
node="cnpe1080", pod="translator-7557c4c9d-fjqg8", pod_ip="10.32.0.3", uid="c32884f0-b550-4458-8797-
5322fc59a0c7"}      1
kube_pod_info{created_by_kind="ReplicaSet", created_by_name="repository-6484ddb6d5", host_ip="192.168.100.61",
host_network="false", instance="10.32.0.7:8080", job="kube-state-metrics", namespace="atlantic",
node="cnpe1080", pod="repository-6484ddb6d5-9k8jd", pod_ip="10.32.0.6", uid="dc00f8cc-a387-4185-8216-
8b306b88bfe3"}    1
kube_pod_info{created_by_kind="ReplicaSet", created_by_name="datastore-7697475464", host_ip="192.168.100.61",
host_network="false", instance="10.32.0.7:8080", job="kube-state-metrics", namespace="atlantic",
node="cnpe1080", pod="datastore-7697475464-gvvst", pod_ip="10.32.0.1", uid="621695fd-0fd8-4704-a83f-
fe41d578c6ef"}
```

Step 3: Prometheus target info

A Prometheus target is simply a specific endpoint (usually an IP address and port, like `10.244.0.3:9090/metrics`). Prometheus periodically calls (scrapes) that endpoint to collect the metric data.

The kube-state-metrics (KSM) is a service that talks to the Kubernetes API server to generate metrics about the status of objects like *Pods*, *Deployments* or *Nodes*. In the end it's also just a *Pod* in *Namespace* `kube-system` which exposes metrics at `/metrics`. KSM is usually not installed by default.

We can see the targets in the web interface. We select `Status->Target health`:



We see kube-state-metrics with two endpoints on the same *Pod*, one port `8080` and one port `8081` used for different types of data.

For this step we actually need the targets with errors on the bottom. These are the *Pods* in *Namespace* `atlantic` which have been configured as scrape target in Prometheus. The issue is that they don't expose any metrics themselves, hence the error.

You might wonder: How can we query `kube_pod_info` for our application *Pods* if they aren't listed here as targets? This is because `kube_pod_info` is generated by kube-state-metrics and not by scraping the application *Pods* directly.

To solve this step we write the error to the required location:

```
➜ candidate@cnpe1080:~$ vim /course/7/error.txt
```

```
# cnpe1080:/course/7/error.txt
server returned HTTP status 404 Not Found
```

▼ Question 8 | Grafana, Loki, Logging, Monitoring

Solve this question on: `ssh cnpe0720`

Grafana can be accessed at `http://cnpe0720:30080` and Loki is configured as the only datasource.

1. Set "Maximum lines" for the Loki datasource to `100`
2. In the `logging` dashboard, update the existing panel's query to the following and save the dashboard:

   ```
   count(rate({pod=~"connection.*"}[5m]))
   ```

   Use the query **exactly** as shown, without additional spacing or reordering
3. Two *Pods* are producing error logs. Run the Loki query below to locate them, then scale their respective controllers down to `0`:

   ```
   {job=~"loki.*"} |= "ERROR"
   ```

Answer:

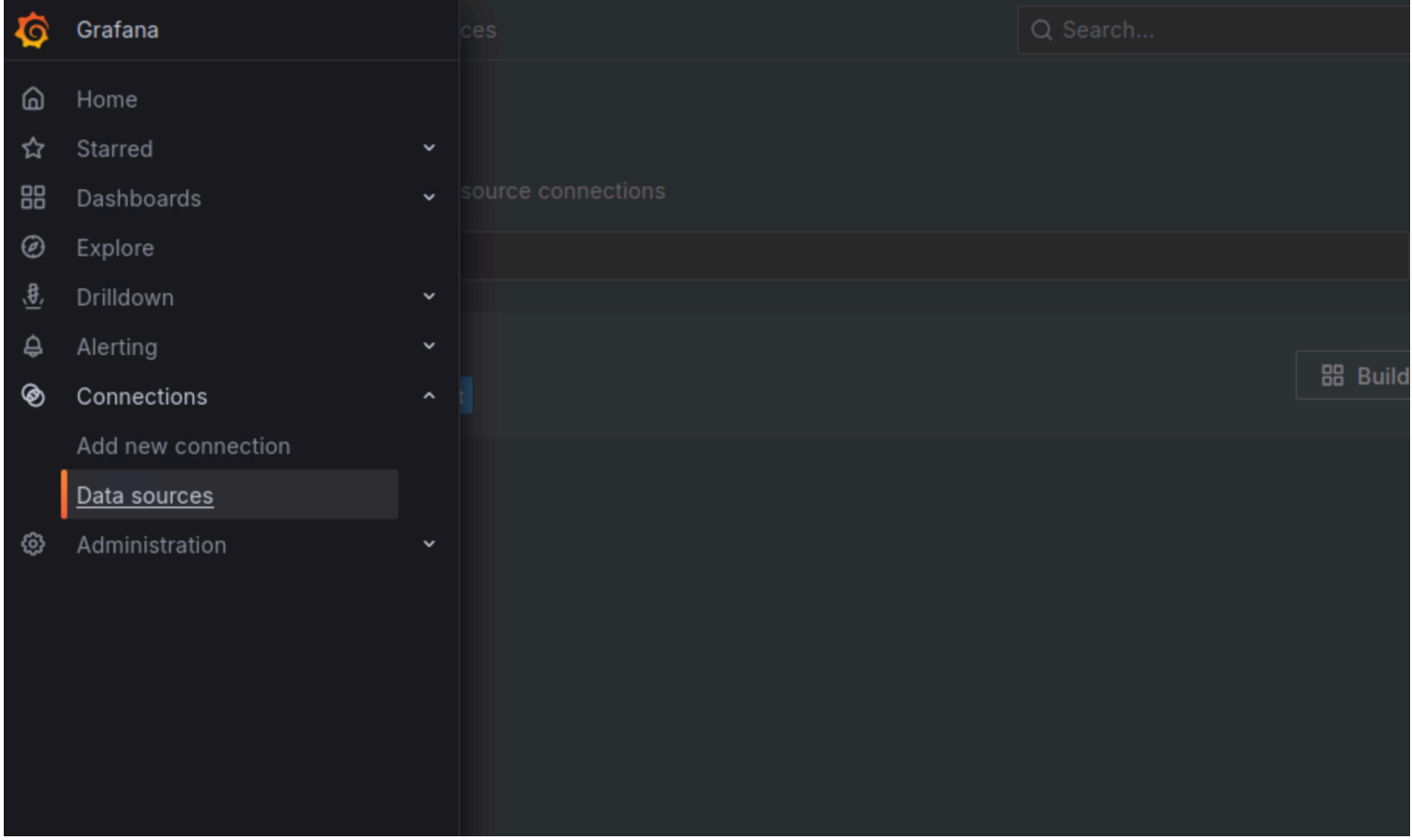
Step 1: Update datasource

We open `http://cnpe0720::30080` in Firefox.

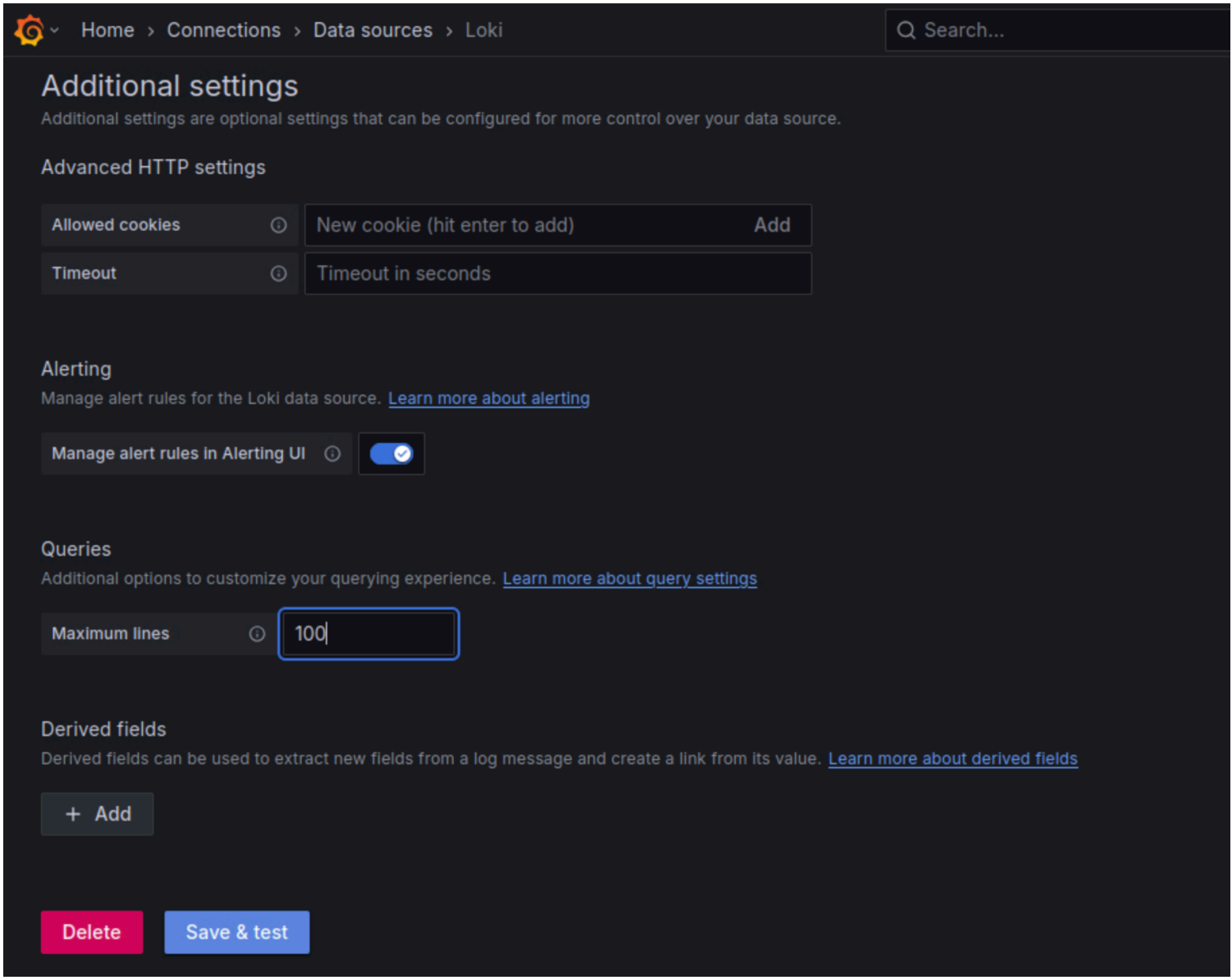
No login required, anonymous admin, very secure! For a simulator environment like this it's fine, but in real life all monitoring tools should be properly secured.

In the web interface:

1. Select the main menu in the top-left corner (Grafana icon)
2. Click Connections -> Data Sources
3. Select the Loki Datasource



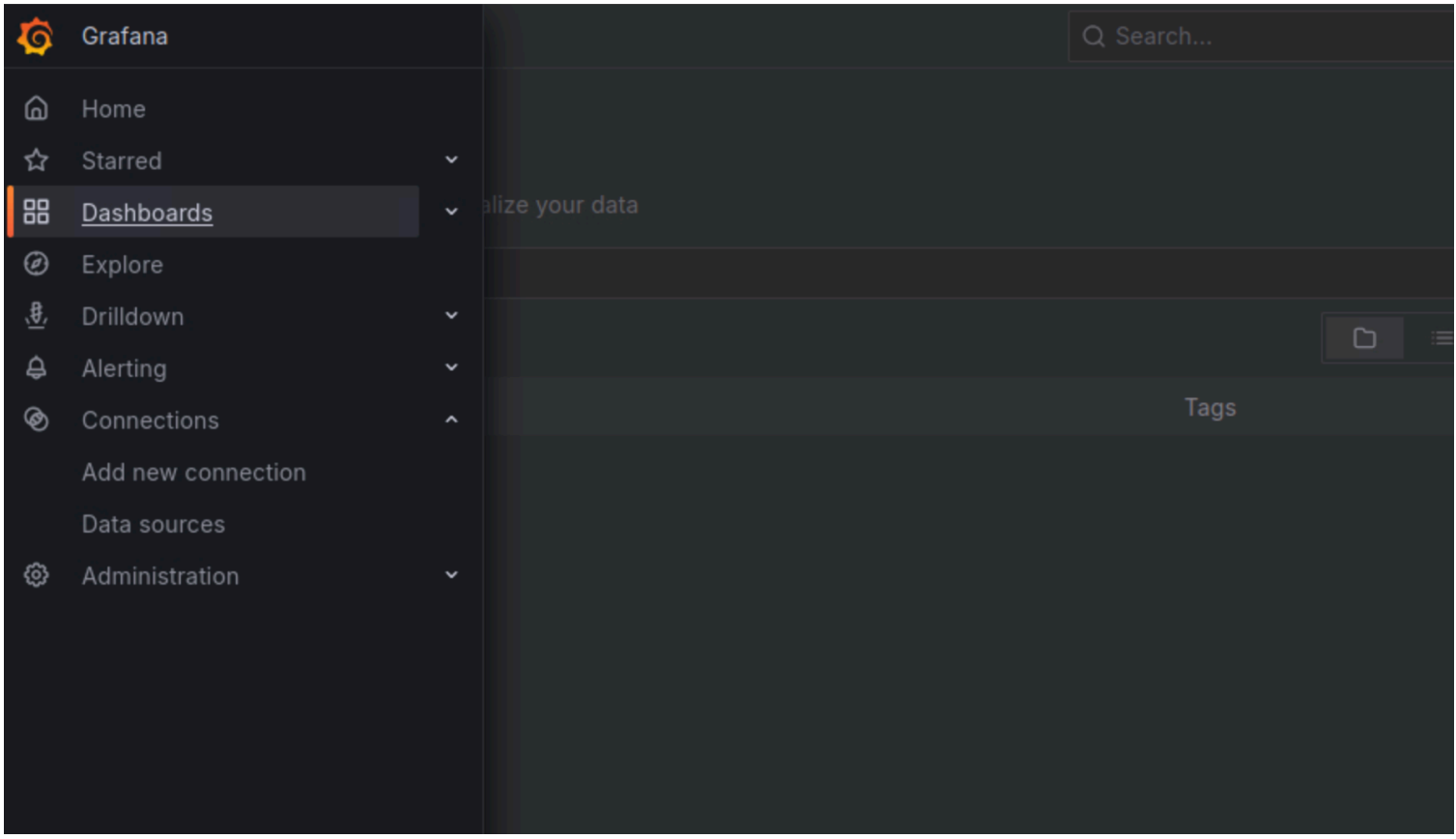
4. Locate "Maximum lines" and set to 100
5. Click on Test and Save



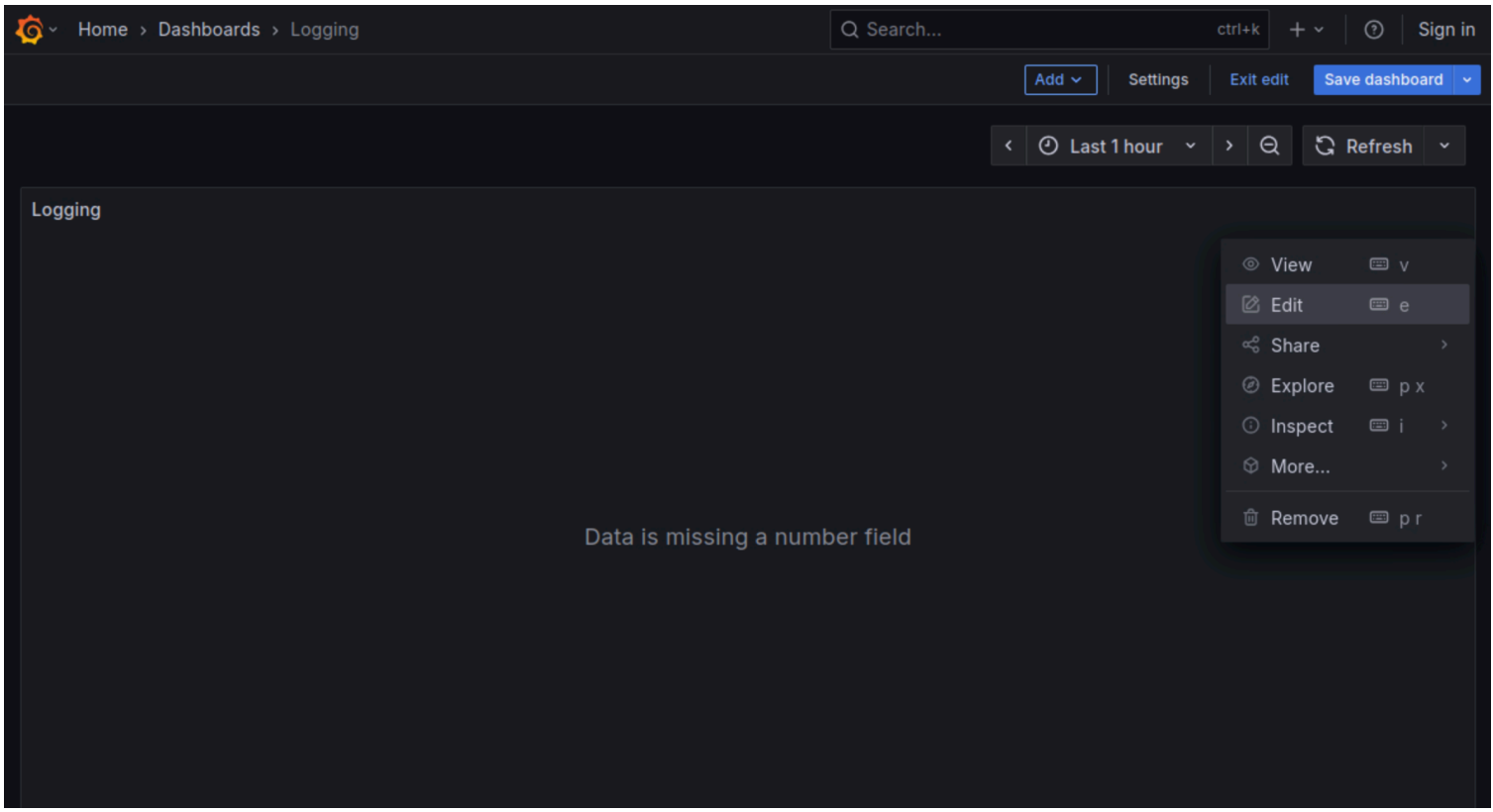
Step 2: Update dashboard

We have to update an existing dashboard:

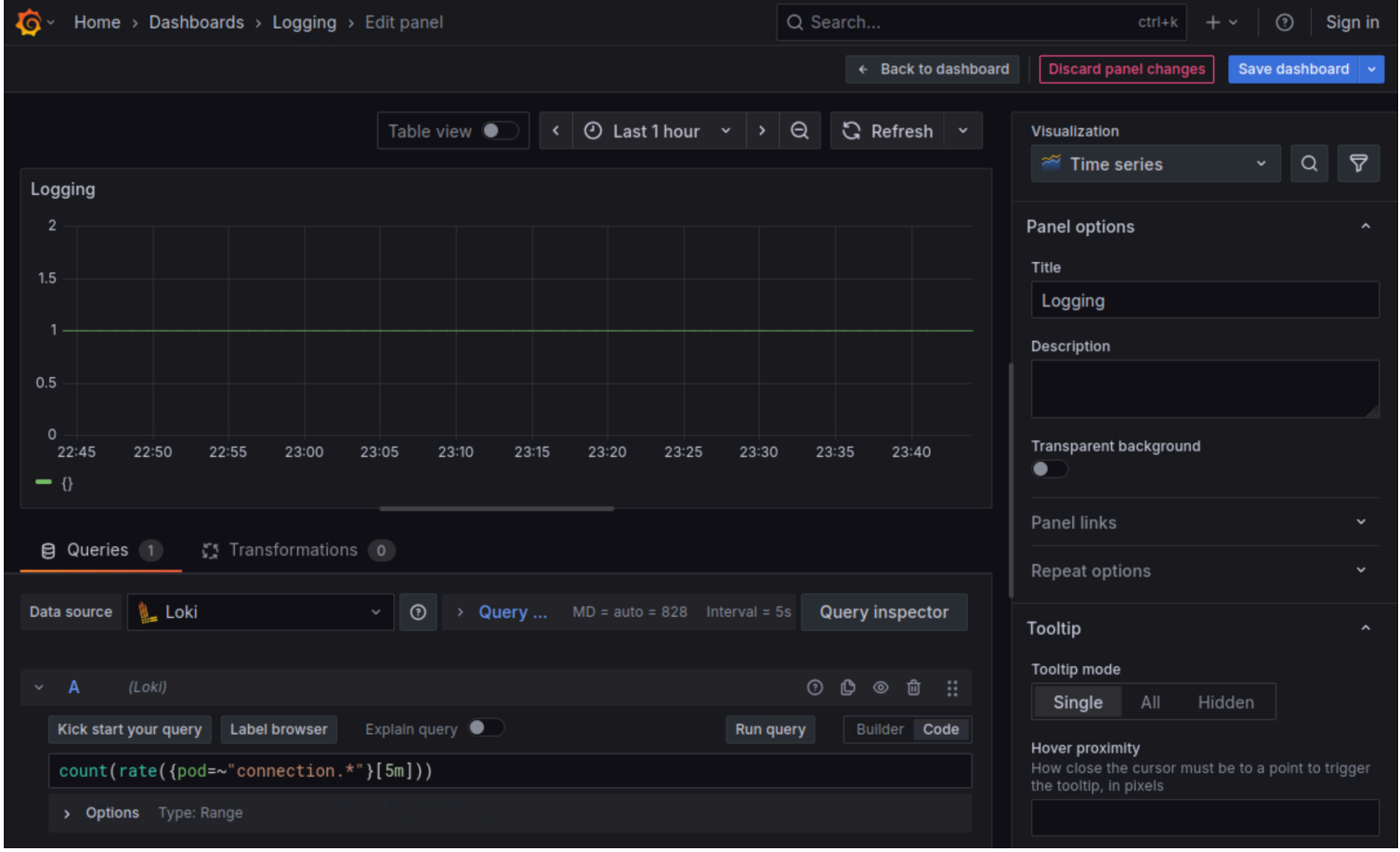
1. Select the main menu in the top-left corner (Grafana icon)
2. Click Dashboards



3. Select the `login` dashboard
4. Select Edit for the panel



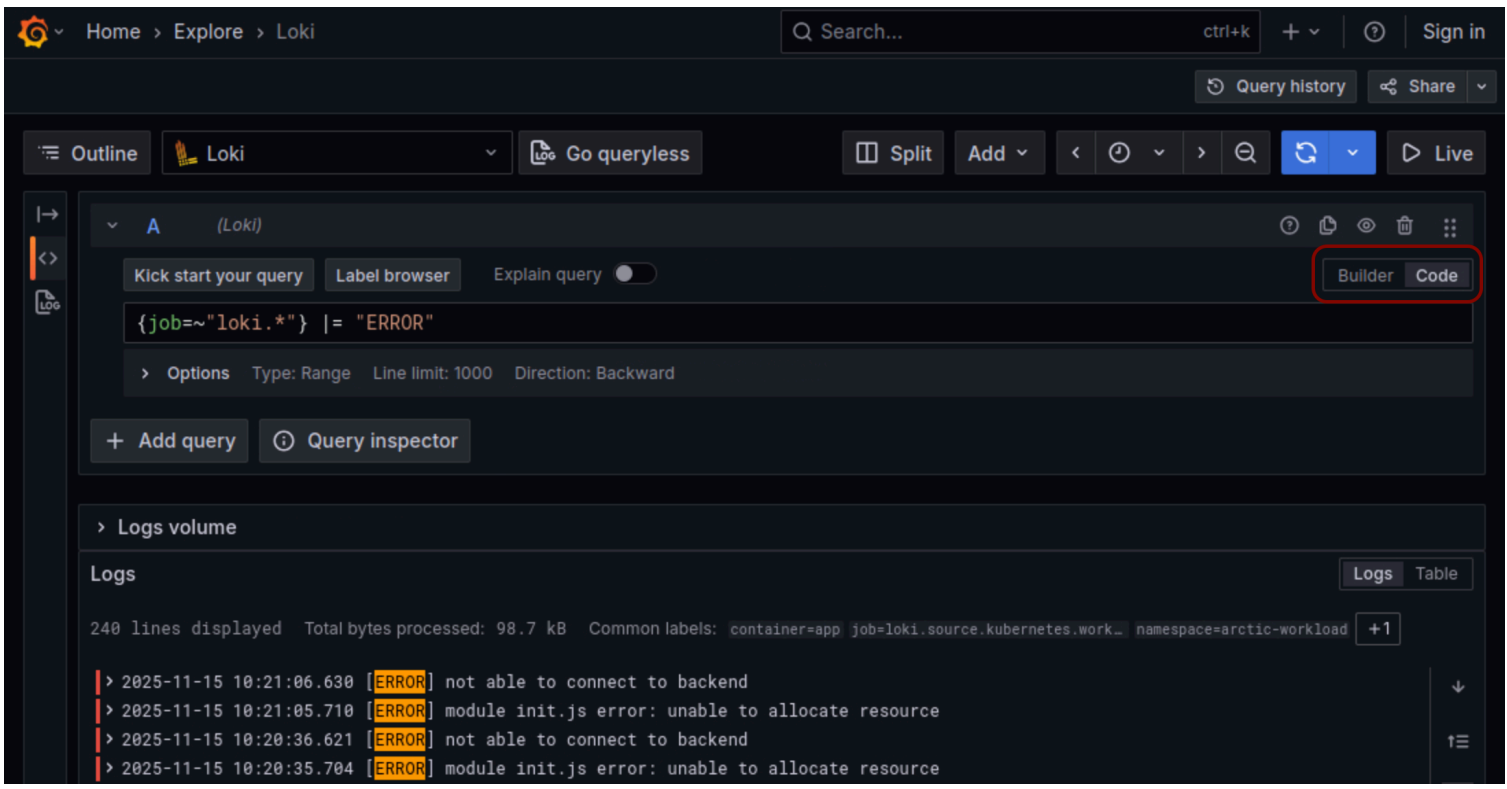
5. Replace the query with `count(rate({pod=~"connection.*"}[5m]))`



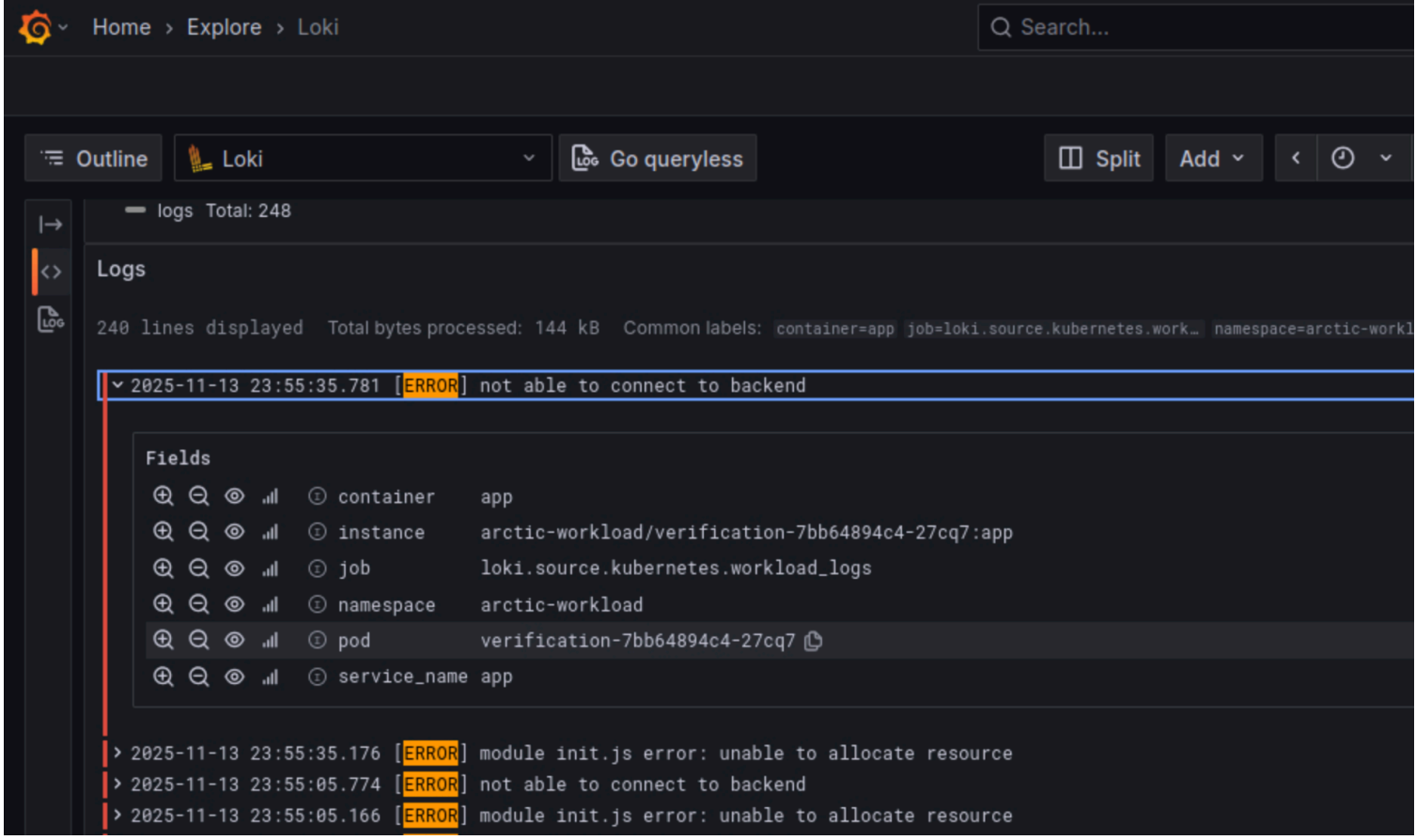
6. Click Save

Step 3: Scale down applications with errors

1. Select the main menu in the top-left corner (Grafana icon)
2. Click Explore
3. Select Code instead of Builder on top right to be able to enter queries
4. We see logs when running the query `{job=~"loki.*"} |= "ERROR"`



5. Once we check some of the log details, we see the *Pod* names are `verification-*` and `resource-limiter-*` in *Namespace* `arctic-workload`



We look for the controllers of the two *Pods* in their *Namespace*:

```
→ ssh cnpe0720
```

```
→ candidate@cnpe0720:~$ k -n arctic-workload get all
NAME                                        READY   STATUS    RESTARTS   AGE
pod/connection-uplift-56bb78b948-bbwnp      1/1     Running   0          7h53m
pod/resource-limiter-0                      1/1     Running   0          26s
pod/verification-7bb64894c4-nvvfc           1/1     Running   0          14s
pod/workflow-engine-0                       1/1     Running   0          7h53m

NAME                                   READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/connection-uplift      1/1     1            1           7h53m
deployment.apps/verification           1/1     1            1           7h53m

NAME                                              DESIRED   CURRENT   READY   AGE
replicaset.apps/connection-uplift-56bb78b948      1         1         1       7h53m
replicaset.apps/verification-7bb64894c4           1         1         1       7h53m

NAME                                  READY   AGE
statefulset.apps/resource-limiter     1/1     7h53m
statefulset.apps/workflow-engine      1/1     7h53m
```

We can see one is controlled by a *Deployment* and one by a *StatefulSet*, we scale them down:

```
→ candidate@cnpe0720:~$ kubectl -n arctic-workload scale deploy verification --replicas 0
deployment.apps/verification scaled

→ candidate@cnpe0720:~$ kubectl -n arctic-workload scale sts resource-limiter --replicas 0
statefulset.apps/resource-limiter scaled
```

That should be it.

Grafana works with Loki for logs similar to how it works with Prometheus for metrics. This allows us to create custom dashboards and even alerts based on logs just like we're used to from metrics.

▼ Question 9 | Kustomize, Prometheus CRDs

Solve this question on: `ssh cnpe2561`

A tightly customized Prometheus Operator configuration is managed via Kustomize with a `staging` and a `production` overlay at `/course/9/prom-config`. The configuration has been applied like this:

```
kubectl apply -k /course/9/prom-config/overlays/staging
kubectl apply -k /course/9/prom-config/overlays/production
```

Perform the following in the Kustomize configuration and apply all changes:

1. *ConfigMap* `operator-config` should have:
    - `reconcile_interval_seconds: "30"` in `staging`
    - `reconcile_interval_seconds: "10"` in `production`
2. *PodMonitor* `proxy-monitor` should have:
    - `attachMetadata: { node: true }` in `base` (inherited by both overlays)
    - `sampleLimit: 6000` in `staging`
    - `sampleLimit: 7000` in `production`
3. Add the `crd-prometheusrules.yaml` to `base` configuration so it will be installed in all environments

Answer:

Kustomize is a tool for managing Kubernetes YAML configurations and is also built into kubectl. A common pattern is to keep shared resources in a **base** and adjust them through overlays, such as the **staging** and **production** overlays used here.

Overview

```
➜ ssh cnpe2561

➜ candidate@cnpe2561:~$ cd /course/9/prom-config

➜ candidate@cnpe2561:/course/9/prom-config$ find
.
./base
./base/config.yaml
./base/kustomization.yaml
./base/prometheus-operator
./base/prometheus-operator/crd-prometheusrules.yaml
./base/prometheus-operator/crd-podmonitors.yaml
./base/monitors.yaml
./overlays
./overlays/staging
./overlays/staging/config.yaml
```

```
./overlays/staging/kustomization.yaml
./overlays/staging/monitors.yaml
./overlays/production
./overlays/production/kustomization.yaml
./overlays/production/monitors.yaml
```

This shows the layout of the Kustomize setup. The **base** directory holds the shared configuration and *CRDs*, while the **staging** and **production** overlays build on top of it and override only the parts that differ between environments.

Using Kustomize we can build overlays, which will simply generate the complete YAML:

```
kubectl kustomize ./overlays/staging
```

We could also do the same for the **base**, which can be helpful for debugging:

```
kubectl kustomize ./base
```

With this, we could diff changes using pipes:

```
kubectl kustomize ./overlays/staging | kubectl diff -f -
```

Or we could do the same with a shorter syntax:

```
kubectl diff -k ./overlays/staging
```

Step 1: Define *ConfigMap* values for different overlays

We are required to override a *ConfigMap* value and it should be different for **staging** and **production**.

```
➜ candidate@cnpe2561:/course/9/prom-config$ cat base/config.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: operator-config
data:
  operator_mode: "passive"
  reconcile_interval_seconds: "60"

  controller.settings: |
    enableMetrics=true
    maxConcurrentReconciles=4
    featureGates=ExperimentalChecks

  alerting.rules: |
    alert.lowDiskSpace=true
    alert.restartCountThreshold=10
    alert.latencyThresholdMs=250
```

This is the full *ConfigMap* in base, let's change **staging** first:

```
➜ candidate@cnpe2561:/course/9/prom-config$ vim overlays/staging/config.yaml
```

```
# cnpe2561:/course/9/prom-config/overlays/staging/config.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: operator-config
data:
  operator_mode: "idle"
  reconcile_interval_seconds: "30"   # ADD
```

Now we can diff and apply if we're satisfied:

```
➜ candidate@cnpe2561:/course/9/prom-config$ k diff -k overlays/staging
diff -u -N /tmp/LIVE-177595067/v1.ConfigMap.atlantic-staging.operator-config /tmp/MERGED-
276721272/v1.ConfigMap.atlantic-staging.operator-config
--- /tmp/LIVE-177595067/v1.ConfigMap.atlantic-staging.operator-config   2025-11-24 16:30:05.728917033 +0000
+++ /tmp/MERGED-276721272/v1.ConfigMap.atlantic-staging.operator-config 2025-11-24 16:30:05.731917298 +0000
@@ -9,7 +9,7 @@
     maxConcurrentReconciles=4
     featureGates=ExperimentalChecks
   operator_mode: idle
-  reconcile_interval_seconds: "60"
+  reconcile_interval_seconds: "30"
 kind: ConfigMap
 metadata:
   annotations:

➜ candidate@cnpe2561:/course/9/prom-config$ k apply -k overlays/staging/
customresourcedefinition.apiextensions.k8s.io/prometheusrules.monitoring.coreos.com unchanged
configmap/operator-config configured

➜ candidate@cnpe2561:/course/9/prom-config$ k -n atlantic-staging get cm operator-config -oyaml
apiVersion: v1
data:
  alerting.rules: |
    alert.lowDiskSpace=true
    alert.restartCountThreshold=10
    alert.latencyThresholdMs=250
  controller.settings: |
    enableMetrics=true
    maxConcurrentReconciles=4
    featureGates=ExperimentalChecks
  operator_mode: idle
  reconcile_interval_seconds: "30"
kind: ConfigMap
...
```

Looks like it worked just great.

Now for production we actually need to create the `config.yaml` because it does not yet exist. We can simply copy it from **staging** and then make the change:

```
➜ candidate@cnpe2561:/course/9/prom-config$ cp overlays/staging/config.yaml overlays/production/

➜ candidate@cnpe2561:/course/9/prom-config$ vim overlays/production/config.yaml
```

```
# cnpe2561:/course/9/prom-config/overlays/production/config.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: operator-config
data:
  reconcile_interval_seconds: "10"    # CHANGE
```

If you copy files from another stage ensure that you only keep the data that's wanted and not other settings as well.

For **production** we also need to add the `config.yaml` to the Kustomize configuration, we could look at the **staging** overlay in case we need to know how to do it:

```
→ candidate@cnpe2561:/course/9/prom-config$ vim overlays/production/kustomization.yaml
```

```
# cnpe2561:/course/9/prom-config/overlays/production/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - ../../base

patches:
  - path: monitors.yaml
  - path: config.yaml    # ADD

transformers:
  - |-
    apiVersion: builtin
    kind: NamespaceTransformer
    metadata:
      name: notImportantHere
      namespace: atlantic-production
```

Now we can diff and apply:

```
→ candidate@cnpe2561:/course/9/prom-config$ k diff -k overlays/production
diff -u -N /tmp/LIVE-30124080/v1.ConfigMap.atlantic-production.operator-config /tmp/MERGED-
3493795785/v1.ConfigMap.atlantic-production.operator-config
--- /tmp/LIVE-30124080/v1.ConfigMap.atlantic-production.operator-config 2025-11-24 16:38:04.241089292 +0000
+++ /tmp/MERGED-3493795785/v1.ConfigMap.atlantic-production.operator-config     2025-11-24
16:38:04.242089376 +0000
@@ -9,7 +9,7 @@
     maxConcurrentReconciles=4
     featureGates=ExperimentalChecks
   operator_mode: passive
-  reconcile_interval_seconds: "60"
+  reconcile_interval_seconds: "10"
 kind: ConfigMap
 metadata:
   annotations:

→ candidate@cnpe2561:/course/9/prom-config$ k apply -k overlays/production
customresourcedefinition.apiextensions.k8s.io/prometheusrules.monitoring.coreos.com unchanged
configmap/operator-config configured

→ candidate@cnpe2561:/course/9/prom-config$ k -n atlantic-production get cm operator-config -oyaml
apiVersion: v1
data:
  alerting.rules: |
```

```
    alert.lowDiskSpace=true
    alert.restartCountThreshold=10
    alert.latencyThresholdMs=250
  controller.settings: |
    enableMetrics=true
    maxConcurrentReconciles=4
    featureGates=ExperimentalChecks
  operator_mode: passive
  reconcile_interval_seconds: "10"
kind: ConfigMap
...
```

Step 2: Define *PodMonitor* settings for different overlays

Here we need to update the *PodMonitor* `proxy-monitor`. Helpful to look at the full YAML in **base** first where we should see all settings that need to be changed:

```
➜ candidate@cnpe2561:/course/9/prom-config$ vim base/monitors.yaml
```

```yaml
# cnpe2561:/course/9/prom-config/base/monitors.yaml
apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
metadata:
  name: proxy-monitor
  labels:
    app: proxy
    team: edge-proxy
spec:
  namespaceSelector:
    matchNames:
      - proxy
  selector:
    matchLabels:
      app: proxy
      component: http-proxy
  podMetricsEndpoints:
    - port: metrics
      path: /metrics
      scheme: http
      interval: 60s
      scrapeTimeout: 10s
      honorLabels: true
      relabelings:
        - sourceLabels: [__meta_kubernetes_pod_node_name]
          targetLabel: node
  sampleLimit: 5000   # WE NEED THIS
  labelLimit: 10
  targetLimit: 100
```

Well, we see the `sampleLimit` but not the `attachMetadata`. It is probably a direct child to `spec:` but we can make sure by searching in the *CRD*:

```
➜ candidate@cnpe2561:/course/9/prom-config$ grep -A10 -B10 attachMetadata base/prometheus-operator/crd-podmonitors.yaml
                Cannot be updated.
                In CamelCase.
                More info: https://git.k8s...
              type: string
            metadata:
```

```
            type: object
          spec:
            description: spec defines the specification of desired Pod selection for
              target discovery by Prometheus.
            properties:
              attachMetadata:
                description: |-
                  attachMetadata defines additional metadata which is added to the
                  discovered targets.

                  It requires Prometheus >= v2.35.0.
                properties:
                  node:
                    description: |-
                      node when set to true, Prometheus...
```

By doing that we can see that we indeed need to add it directly under `spec:`:

```
➜ candidate@cnpe2561:/course/9/prom-config$ vim base/monitors.yaml
```

```yaml
# cnpe2561:/course/9/prom-config/base/monitors.yaml
apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
metadata:
  name: proxy-monitor
  labels:
    app: proxy
    team: edge-proxy
spec:
  namespaceSelector:
    matchNames:
      - proxy
  selector:
    matchLabels:
      app: proxy
      component: http-proxy
  podMetricsEndpoints:
    - port: metrics
      path: /metrics
      scheme: http
      interval: 60s
      scrapeTimeout: 10s
      honorLabels: true
      relabelings:
        - sourceLabels: [__meta_kubernetes_pod_node_name]
          targetLabel: node
  sampleLimit: 5000
  labelLimit: 10
  targetLimit: 100
  attachMetadata: { node: true }   # ADD
```

Next for **staging**:

```
➜ candidate@cnpe2561:/course/9/prom-config$ vim overlays/staging/monitors.yaml
```

```yaml
# cnpe2561:/course/9/prom-config/overlays/staging/monitors.yaml
apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
metadata:
  name: proxy-monitor
spec:
  namespaceSelector:
    matchNames:
      - atlantic-staging
  labelLimit: 25
  sampleLimit: 6000    # ADD
```

Important to only add the values that should be overwritten! Next we diff and apply:

```
→ candidate@cnpe2561:/course/9/prom-config$ k diff -k overlays/staging
diff -u -N /tmp/LIVE-3912499013/monitoring.coreos.com.v1.PodMonitor.atlantic-staging.proxy-monitor
/tmp/MERGED-3784972855/monitoring.coreos.com.v1.PodMonitor.atlantic-staging.proxy-monitor
--- /tmp/LIVE-3912499013/monitoring.coreos.com.v1.PodMonitor.atlantic-staging.proxy-monitor     2025-11-24
17:48:22.313201810 +0000
+++ /tmp/MERGED-3784972855/monitoring.coreos.com.v1.PodMonitor.atlantic-staging.proxy-monitor   2025-11-24
17:48:22.318202228 +0000
@@ -5,7 +5,7 @@
     kubectl.kubernetes.io/last-applied-configuration: |
       {"apiVersion":"monitoring.coreos.com/v1","kind":"PodMonitor","metadata":{"annotations":{},"labels":
{"app":"proxy","team":"edge-proxy"},"name":"proxy-monitor","namespace":"atlantic-staging"},"spec":
{"labelLimit":25,"namespaceSelector":{"matchNames":["atlantic-staging"]},"podMetricsEndpoints":
[{"honorLabels":true,"interval":"60s","path":"/metrics","port":"metrics","relabelings":[{"sourceLabels":
["__meta_kubernetes_pod_node_name"],"targetLabel":"node"}],"scheme":"http","scrapeTimeout":"10s"}],"sampleLi
mit":5000,"selector":{"matchLabels":{"app":"proxy","component":"http-proxy"}},"targetLimit":100}}
   creationTimestamp: "2025-11-24T17:22:26Z"
-  generation: 1
+  generation: 2
   labels:
     app: proxy
     team: edge-proxy
@@ -14,6 +14,8 @@
   resourceVersion: "28007"
   uid: 94fc5cc8-c97b-4879-8219-bd058cfbf26c
 spec:
+  attachMetadata:
+    node: true
   labelLimit: 25
   namespaceSelector:
     matchNames:
@@ -30,7 +32,7 @@
       targetLabel: node
     scheme: http
     scrapeTimeout: 10s
-  sampleLimit: 5000
+  sampleLimit: 6000
   selector:
     matchLabels:
       app: proxy

→ candidate@cnpe2561:/course/9/prom-config$ k apply -k overlays/staging
customresourcedefinition.apiextensions.k8s.io/podmonitors.monitoring.coreos.com unchanged
configmap/operator-config unchanged
podmonitor.monitoring.coreos.com/proxy-monitor configured
```

And for **production**:

```
➜ candidate@cnpe2561:/course/9/prom-config$ vim overlays/production/monitors.yaml
```

```yaml
# cnpe2561:/course/9/prom-config/overlays/production/monitors.yaml
apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
metadata:
  name: proxy-monitor
spec:
  namespaceSelector:
    matchNames:
      - atlantic-production
  labelLimit: 50
  sampleLimit: 7000   # ADD
```

Now to diff and apply:

```
➜ candidate@cnpe2561:/course/9/prom-config$ k diff -k overlays/production
diff -u -N /tmp/LIVE-287304218/monitoring.coreos.com.v1.PodMonitor.atlantic-production.proxy-monitor
/tmp/MERGED-942215349/monitoring.coreos.com.v1.PodMonitor.atlantic-production.proxy-monitor
--- /tmp/LIVE-287304218/monitoring.coreos.com.v1.PodMonitor.atlantic-production.proxy-monitor   2025-11-24
17:50:13.970524499 +0000
+++ /tmp/MERGED-942215349/monitoring.coreos.com.v1.PodMonitor.atlantic-production.proxy-monitor 2025-11-24
17:50:13.973524749 +0000
@@ -5,7 +5,7 @@
     kubectl.kubernetes.io/last-applied-configuration: |
       {"apiVersion":"monitoring.coreos.com/v1","kind":"PodMonitor","metadata":{"annotations":{},"labels":
{"app":"proxy","team":"edge-proxy"},"name":"proxy-monitor","namespace":"atlantic-production"},"spec":
{"labelLimit":50,"namespaceSelector":{"matchNames":["atlantic-production"]},"podMetricsEndpoints":
[{"honorLabels":true,"interval":"60s","path":"/metrics","port":"metrics","relabelings":[{"sourceLabels":
["__meta_kubernetes_pod_node_name"],"targetLabel":"node"}],"scheme":"http","scrapeTimeout":"10s"}],"sampleLi
mit":5000,"selector":{"matchLabels":{"app":"proxy","component":"http-proxy"}},"targetLimit":100}}
   creationTimestamp: "2025-11-24T17:22:30Z"
-  generation: 1
+  generation: 2
   labels:
     app: proxy
     team: edge-proxy
@@ -14,6 +14,8 @@
   resourceVersion: "28015"
   uid: c87009ac-dc22-4fba-8ff7-f734834c96e9
 spec:
+  attachMetadata:
+    node: true
   labelLimit: 50
   namespaceSelector:
     matchNames:
@@ -30,7 +32,7 @@
       targetLabel: node
     scheme: http
     scrapeTimeout: 10s
-  sampleLimit: 5000
+  sampleLimit: 7000
   selector:
     matchLabels:
       app: proxy

➜ candidate@cnpe2561:/course/9/prom-config$ k apply -k overlays/production
```

```
customresourcedefinition.apiextensions.k8s.io/podmonitors.monitoring.coreos.com unchanged
configmap/operator-config unchanged
podmonitor.monitoring.coreos.com/proxy-monitor configured
```

Step3: Install new *CRD*

Finally we need to add another *CRD* which should be installed. We can see the file:

```
➜ candidate@cnpe2561:/course/9/prom-config$ find base/
base/
base/config.yaml
base/kustomization.yaml
base/prometheus-operator
base/prometheus-operator/crd-prometheusrules.yaml
base/prometheus-operator/crd-podmonitors.yaml
base/monitors.yaml
```

We only need to add it to the `base/kustomization.yaml`:

```
➜ candidate@cnpe2561:/course/9/prom-config$ vim base/kustomization.yaml
```

```yaml
# cnpe2561:/course/9/prom-config/base/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - prometheus-operator/crd-podmonitors.yaml
  - prometheus-operator/crd-prometheusrules.yaml   # ADD
  - monitors.yaml
  - config.yaml

transformers:
  - |-
    apiVersion: builtin
    kind: NamespaceTransformer
    metadata:
      name: notImportantHere
      namespace: NAMESPACE_REPLACE
```

Now we install it in **staging**:

```
➜ candidate@cnpe2561:/course/9/prom-config$ k diff -k overlays/staging
diff -u -N /tmp/LIVE-
1059032839/apiextensions.k8s.io.v1.CustomResourceDefinition..prometheusrules.monitoring.coreos.com
/tmp/MERGED-
2427058298/apiextensions.k8s.io.v1.CustomResourceDefinition..prometheusrules.monitoring.coreos.com
--- /tmp/LIVE-
1059032839/apiextensions.k8s.io.v1.CustomResourceDefinition..prometheusrules.monitoring.coreos.com
2025-11-24 17:54:29.804849780 +0000
+++ /tmp/MERGED-
2427058298/apiextensions.k8s.io.v1.CustomResourceDefinition..prometheusrules.monitoring.coreos.com
2025-11-24 17:54:29.812850447 +0000
@@ -0,0 +1,277 @@
+apiVersion: apiextensions.k8s.io/v1
+kind: CustomResourceDefinition
+metadata:
+  annotations:
```

```
+    controller-gen.kubebuilder.io/version: v0.19.0
+    operator.prometheus.io/version: 0.86.1
+  creationTimestamp: "2025-11-24T17:54:29Z"
+  generation: 1
+  name: prometheusrules.monitoring.coreos.com
+  uid: 45f95144-7b82-48ed-9471-6f49d587c06c
+spec:
+  conversion:
+    strategy: None
+  group: monitoring.coreos.com
+  names:
+    categories:
+    - prometheus-operator
+    kind: PrometheusRule
+    listKind: PrometheusRuleList
+    plural: prometheusrules
+    shortNames:
+    - promrule
+    singular: prometheusrule
+  scope: Namespaced
+  versions:
+  - name: v1
+    schema:
+      openAPIV3Schema:
...

➜ candidate@cnpe2561:/course/9/prom-config$ k apply -k overlays/staging
customresourcedefinition.apiextensions.k8s.io/podmonitors.monitoring.coreos.com unchanged
customresourcedefinition.apiextensions.k8s.io/prometheusrules.monitoring.coreos.com created
configmap/operator-config unchanged
podmonitor.monitoring.coreos.com/proxy-monitor configured

➜ candidate@cnpe2561:/course/9/prom-config$ k get crd
NAME                                  CREATED AT
podmonitors.monitoring.coreos.com     2025-11-24T17:20:43Z
prometheusrules.monitoring.coreos.com 2025-11-24T17:54:49Z
```

Looking great! Now we should see no change for **production**:

```
➜ candidate@cnpe2561:/course/9/prom-config$ k diff -k overlays/production

➜ candidate@cnpe2561:/course/9/prom-config$
```

Since *CRDs* are cluster-wide and both overlays target the same Kubernetes cluster, the *CRD* only needs to be installed once. If both overlays were installed in different clusters, **production** would install the *CRD* as well.

Kustomize / Helm and State

In this scenario we did not have to delete any resources, but let's shortly talk about it. If you are asked to delete resources from Kustomize configuration and apply the change, you will **need to perform the deletion manually**.

Kustomize won't delete remote resources if they only exist remote. This is because it does not keep any state and hence doesn't know which remote resources were created by Kustomize or by anything else.

Helm on the other hand will remove remote resources if they only exist remote and if they were created by Helm. It can do this because it keeps a state (or release information) of all performed changes.

Both approaches have pros and cons:

- Kustomize is less complex by not having to manage state, but might need more manual work cleaning up
- Helm can keep better track of remote resources, but things can get complex and messy if there is a state error or mismatch. State changes (Helm actions) at the same time need to be prevented or accounted for

▼ Question 10 | ResourceQuota, Git

Solve this question on: `ssh cnpe1080`

Using *ResourceQuotas*, limit storage usage in *Namespaces* `caspian-pipeline1`, `caspian-pipeline2` and `caspian-pipeline3`.

One of these *Namespaces* recently requested `100Gi` storage, but the change was reverted again. Check commits in the Git repository `/course/10/pipelines-repo` to identify it. Apply the following rules:

1. For the identified *Namespace* (which previously requested the `100Gi` storage):
    - Prevent creation of any *PVCs*
    - Also delete any existing *PVCs*, scale down *Pods* if necessary
2. The other two *Namespaces* should each be limited to:
    - Creating a maximum of `2` *PVCs*
    - Requesting a total of `100Mi` storage across all *PVCs*

ℹ️ You can perform the changes directly in `/course/10/pipelines-repo`, but it is not required for the solution

Answer:

Find the *Namespace*

We need to check the Git history for changes:

```
➜ ssh cnpe1080

➜ candidate@cnpe1080:~$ cd /course/10/pipelines-repo

➜ candidate@cnpe1080:/course/10/pipelines-repo$ git log
commit b711410e07173519624c6f3747e147f9049eedb9 (HEAD -> main)
Author: CNPE User <cnpe-user@simulator>
Date:   Sun Nov 16 01:36:02 2025 +1000

    force update

commit d63f7ddb3b76ddd783122dbe744b6116b696c25e
Author: CNPE User <cnpe-user@simulator>
Date:   Sun Nov 16 12:55:10 2025 +1000

    force update

commit 7b159853624a83b47fcbb81f0a1cc7f4313e2ec6
```

```
Author: CNPE User <cnpe-user@simulator>
Date:   Sun Nov 16 10:10:00 2025 +1000

    fixed pipelines

commit e29536708b0a00e7e56165e58662f87abab058af
Author: CNPE User <cnpe-user@simulator>
Date:   Sat Nov 15 15:00:30 2025 +1000

    updated pipelines

commit 981e2698a7f51283046ecd8b9d0d4e88bf192f81
Author: CNPE User <cnpe-user@simulator>
Date:   Sun Nov 9 17:01:19 2025 +1000

    updated pipeline 1

commit 7341f2892db090653a534959d45c3c14de2fddc0
Author: CNPE User <cnpe-user@simulator>
Date:   Fri Nov 7 08:35:55 2025 +1000

    added two more pipelines

commit a61a3ac62d42c86be016f5b1eb5718c1139ff53e
Author: CNPE User <cnpe-user@simulator>
Date:   Wed Nov 5 20:19:58 2025 +1000

    added first pipeline
```

Quite a few commits! We could go through each by copying the hash and use `git show`:

```
→ candidate@cnpe1080:/course/10/pipelines-repo$ git show a61a3ac62d42c86be016f5b1eb5718c1139ff53e
commit a61a3ac62d42c86be016f5b1eb5718c1139ff53e
Author: CNPE User <cnpe-user@simulator>
Date:   Wed Nov 5 20:19:58 2025 +1000

    added first pipeline

diff --git a/pipeline1.yaml b/pipeline1.yaml
new file mode 100644
index 0000000..09a44ce
--- /dev/null
+++ b/pipeline1.yaml
@@ -0,0 +1,48 @@
+apiVersion: v1
+kind: Namespace
+metadata:
+  name: caspian-pipeline1
+---
+apiVersion: v1
+kind: PersistentVolumeClaim
+metadata:
+  name: gitlab-runner-ae76c
+  namespace: caspian-pipeline1
+  labels:
+    app: gitlab-runner-ae76c
+spec:
+  accessModes:
+    - ReadWriteOnce
```

```
+  resources:
+    requests:
+      storage: 5Mi
+  storageClassName: local-path
+---
+apiVersion: apps/v1
+kind: Deployment
+metadata:
+  name: gitlab-runner-ae76c
+  namespace: caspian-pipeline1
+  labels:
+    app: gitlab-runner-ae76c
+spec:
+  replicas: 1
+  selector:
+    matchLabels:
+      app: gitlab-runner-ae76c
+  template:
+    metadata:
+      labels:
+        app: gitlab-runner-ae76c
+    spec:
+      terminationGracePeriodSeconds: 3
+      containers:
+        - name: runner
+          image: nginx:1-alpine
+          volumeMounts:
+            - name: runner-data-volume
+              mountPath: /mnt/runner-cache
+      volumes:
+        - name: runner-data-volume
+          persistentVolumeClaim:
+            claimName: gitlab-runner-ae76c
```

Above we show the very first commit, where file `pipeline1.yaml` was added.

But we could also show all commits by using `git log -p`:

```
→ candidate@cnpe1080:/course/10/pipelines-repo$ git log -p
commit b711410e07173519624c6f3747e147f9049eedb9 (HEAD -> main)
Author: CNPE User <cnpe-user@simulator>
Date:   Sun Nov 16 01:36:02 2025 +1000

    force update

commit d63f7ddb3b76ddd783122dbe744b6116b696c25e
Author: CNPE User <cnpe-user@simulator>
Date:   Sun Nov 16 12:55:10 2025 +1000

    force update

commit 7b159853624a83b47fcbb81f0a1cc7f4313e2ec6
Author: CNPE User <cnpe-user@simulator>
Date:   Sun Nov 16 10:10:00 2025 +1000

    fixed pipelines

diff --git a/pipeline2.yaml b/pipeline2.yaml
index 6f1e331..749cbbf 100644
```

```
--- a/pipeline2.yaml
+++ b/pipeline2.yaml
@@ -15,7 +15,7 @@ spec:
     - ReadWriteOnce
   resources:
     requests:
-      storage: 100Gi
+      storage: 10Mi
   storageClassName: local-path
 ---
...
```

This should show us that the *PVC* `gitlab-runner-2d60t` in *Namespace* `caspian-pipeline2` tried to allocate `100Gi`. This is the one we need to work with in the next step.

Step1: *ResourceQuota* for *Namespace* `caspian-pipeline2`

The question does not require us to implement the changes in the Git repo, but we decide to do it.

```
➜ candidate@cnpe1080:/course/10/pipelines-repo$ vim pipeline2.yaml
```

```
# cnpe1080:/course/10/pipelines-repo/pipeline2.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: caspian-pipeline2
---
apiVersion: v1                      # ADD from here
kind: ResourceQuota
metadata:
  name: storage-limit-quota
  namespace: caspian-pipeline2
spec:
  hard:
    persistentvolumeclaims: "0"     # TILL here
---
#apiVersion: v1                     # we delete the PVC
#kind: PersistentVolumeClaim
#metadata:
#  name: gitlab-runner-2d60t
#  namespace: caspian-pipeline2
#  labels:
#    app: gitlab-runner-2d60t
#spec:
#  accessModes:
#    - ReadWriteOnce
#  resources:
#    requests:
#      storage: 10Mi
#  storageClassName: local-path
#---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: gitlab-runner-2d60t
  namespace: caspian-pipeline2
  labels:
    app: gitlab-runner-2d60t
spec:
  replicas: 0                       # SCALE down the deployment
  selector:
```

```
    matchLabels:
...
```

> We are not required to do the changes in `/course/10/pipelines-repo`, so we could also just create the *ResourceQuota* from a temporary YAML file instead

We add the new *ResourceQuota* somewhere into the existing YAML. For this *Namespace* we forbid the creation of any *PVCs* by setting the amount to `0`.

```
➜ candidate@cnpe1080:/course/10/pipelines-repo$ k -f pipeline2.yaml diff
...
@@ -15,7 +15,7 @@
   uid: ef9af18f-fa40-444c-b5c1-5f2a3ac36c97
 spec:
   progressDeadlineSeconds: 600
-  replicas: 1
+  replicas: 0
   revisionHistoryLimit: 10
   selector:
     matchLabels:
diff -u -N /tmp/LIVE-2899470031/v1.ResourceQuota.caspian-pipeline2.storage-limit-quota /tmp/MERGED-
385717793/v1.ResourceQuota.caspian-pipeline2.storage-limit-quota
--- /tmp/LIVE-2899470031/v1.ResourceQuota.caspian-pipeline2.storage-limit-quota 2025-12-04
15:42:08.780526888 +0000
+++ /tmp/MERGED-385717793/v1.ResourceQuota.caspian-pipeline2.storage-limit-quota        2025-12-04
15:42:08.783527135 +0000
@@ -0,0 +1,11 @@
+apiVersion: v1
+kind: ResourceQuota
+metadata:
+  creationTimestamp: "2025-12-04T15:42:08Z"
+  name: storage-limit-quota
+  namespace: caspian-pipeline2
+  uid: 99dfa676-9a32-4efd-a0d4-e4e38f691315
+spec:
+  hard:
+    persistentvolumeclaims: "0"
+status: {}

➜ candidate@cnpe1080:/course/10/pipelines-repo$ k -f pipeline2.yaml apply
namespace/caspian-pipeline2 unchanged
resourcequota/storage-limit-quota created
deployment.apps/gitlab-runner-2d60t configured
```

A new/updated *ResourceQuota* will not affect existing resources, hence the question also requires us to delete any existing *PVCs*.

```
➜ candidate@cnpe1080:/course/10/pipelines-repo$ k -n caspian-pipeline2 get pvc
NAME                 STATUS   VOLUME                 CAPACITY   ...
gitlab-runner-2d60t   Bound    pvc-bbfa11b9-314f-...   10Mi       ...

➜ candidate@cnpe1080:/course/10/pipelines-repo$ k -n caspian-pipeline2 delete pvc gitlab-runner-2d60t
persistentvolumeclaim "gitlab-runner-2d60t" deleted from caspian-pipeline2 namespace
^C

➜ candidate@cnpe1080:/course/10/pipelines-repo$ k -n caspian-pipeline2 get pvc
No resources found in caspian-pipeline2 namespace.
```

If the *Deployment* would not have been scaled down, the *PVC* would remain in status `Terminating` because there is still a *Pod* using it. It's necessary to scale down the *Deployment* before we can delete the *PVC*.

(Optional) Generate *ResourceQuota* YAML

Instead of copying from the K8s docs or writing manually we could also generate the YAML like this:

```
➜ candidate@cnpe1080:/course/10/pipelines-repo$ kubectl -n caspian-pipeline2 create quota storage-limit-quota --hard=persistentvolumeclaims=0 -oyaml --dry-run=client
apiVersion: v1
kind: ResourceQuota
metadata:
  name: storage-limit-quota
  namespace: caspian-pipeline2
spec:
  hard:
    persistentvolumeclaims: "0"
status: {}
```

And because this scenario does not require us to store the YAML anywhere we could even just create the *ResourceQuota* resources directly.

Step2: *ResourceQuotas* for other *Namespaces*

Now we need to create the same *ResourceQuota* in *Namespaces* `caspian-pipeline1` and `caspian-pipeline3`:

```
➜ candidate@cnpe1080:/course/10/pipelines-repo$ vim pipeline1.yaml
```

```
# cnpe1080:/course/10/pipelines-repo/pipeline1.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: caspian-pipeline1
---
apiVersion: v1                    # ADD from here
kind: ResourceQuota
metadata:
  name: storage-limit-quota
  namespace: caspian-pipeline1
spec:
  hard:
    requests.storage: 100Mi
    persistentvolumeclaims: "2"   # TILL here
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: gitlab-runner-ae76c
...
```

```
➜ candidate@cnpe1080:/course/10/pipelines-repo$ k -f pipeline1.yaml diff
...
@@ -0,0 +1,12 @@
+apiVersion: v1
+kind: ResourceQuota
+metadata:
+  creationTimestamp: "2025-12-04T14:05:39Z"
```

```
+  name: storage-limit-quota
+  namespace: caspian-pipeline1
+  uid: a5988d68-5d74-42cd-91aa-8f42336ec2fa
+spec:
+  hard:
+    persistentvolumeclaims: "2"
+    requests.storage: 100Mi
+status: {}

➜ candidate@cnpe1080:/course/10/pipelines-repo$ k -f pipeline1.yaml apply
namespace/caspian-pipeline1 unchanged
resourcequota/storage-limit-quota created
persistentvolumeclaim/gitlab-runner-ae76c unchanged
deployment.apps/gitlab-runner-ae76c unchanged
```

And the second one:

```
➜ candidate@cnpe1080:/course/10/pipelines-repo$ vim pipeline3.yaml
```

```
# cnpe1080:/course/10/pipelines-repo/pipeline3.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: caspian-pipeline3
---
apiVersion: v1                    # ADD from here
kind: ResourceQuota
metadata:
  name: storage-limit-quota
  namespace: caspian-pipeline3
spec:
  hard:
    requests.storage: 100Mi
    persistentvolumeclaims: "2"   # TILL here
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: gitlab-runner-98p8e
...
```

```
➜ candidate@cnpe1080:/course/10/pipelines-repo$ k -f pipeline3.yaml diff
...
@@ -0,0 +1,12 @@
+apiVersion: v1
+kind: ResourceQuota
+metadata:
+  creationTimestamp: "2025-12-04T14:09:29Z"
+  name: storage-limit-quota
+  namespace: caspian-pipeline3
+  uid: e6cfdfb5-2caa-46b0-a855-b51ce0205a4c
+spec:
+  hard:
+    persistentvolumeclaims: "2"
+    requests.storage: 100Mi
+status: {}

➜ candidate@cnpe1080:/course/10/pipelines-repo$ k -f pipeline3.yaml apply
namespace/caspian-pipeline3 unchanged
resourcequota/storage-limit-quota created
```

```
persistentvolumeclaim/gitlab-runner-98p8e unchange
```

Completed.

▼ Question 11 | Argo Workflows

Solve this question on: `ssh cnpe3849`

Your platform team wants to provide some example Argo *WorkflowTemplates* to ease adoption for new users. Argo Workflows has been installed with `argo` CLI and UI at `http://cnpe3849:30110`.

1. The existing *Workflow* of *WorkflowTemplate* `greeter` failed:
    - Fix the error in the *WorkflowTemplate*
    - Submit a new *Workflow* which succeeds
2. There is *WorkflowTemplate* `/course/11/configurator.yaml` which creates *ConfigMaps* in a passed *Namespace*:
    - Create step `create-config2` by copying `create-config1`, it should create *ConfigMap* `cm2`
    - Run the new step in parallel to the existing one
    - Apply the updated *WorkflowTemplate*
    - Submit a new *Workflow* for *Namespace* `kaw` which succeeds

Delete failed *Workflow(s)* and **keep only one successful** one per *WorkflowTemplate*.

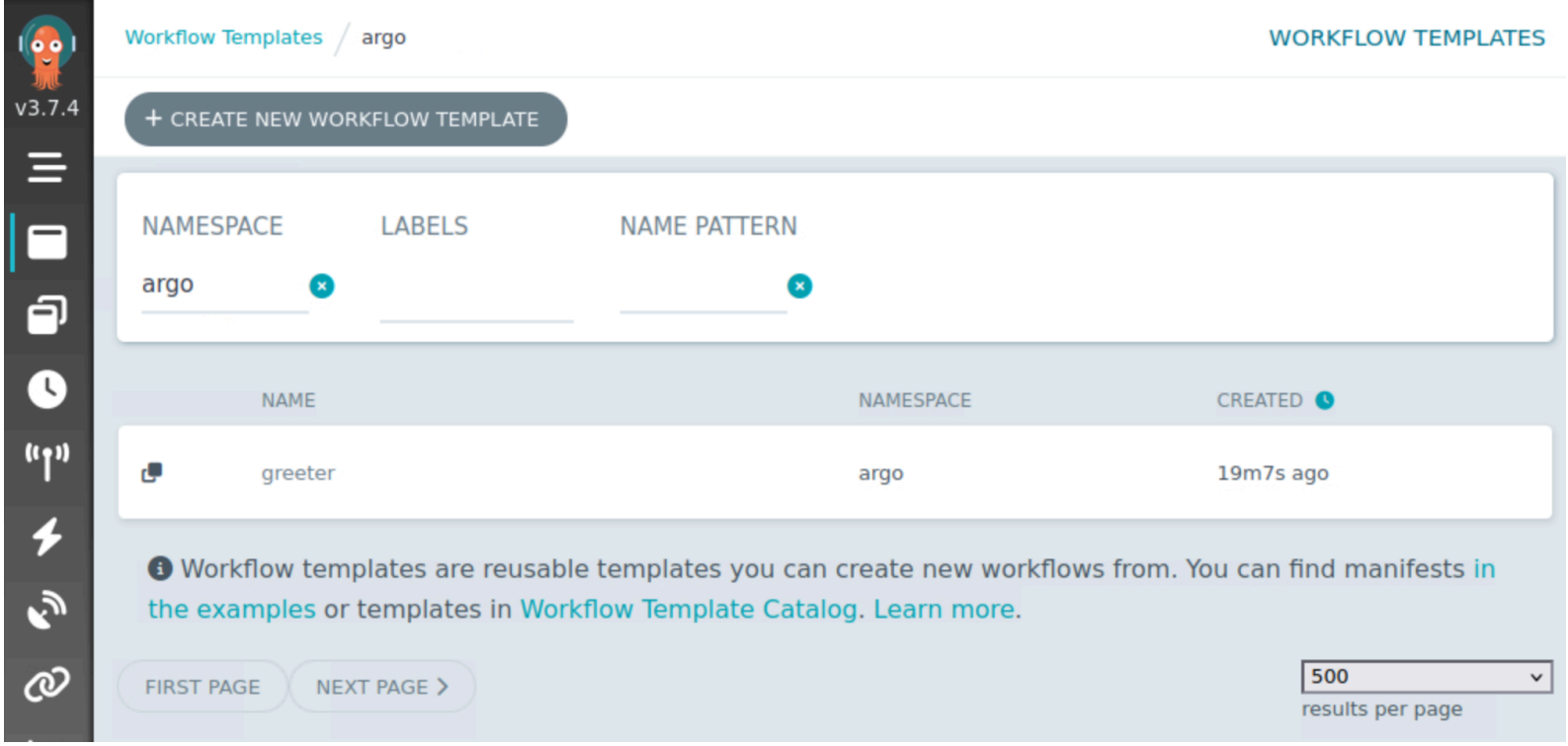
Answer:

Argo Workflows is an open source container-native workflow engine for orchestrating parallel jobs on Kubernetes. Argo Workflows is implemented as a Kubernetes *CRD*.

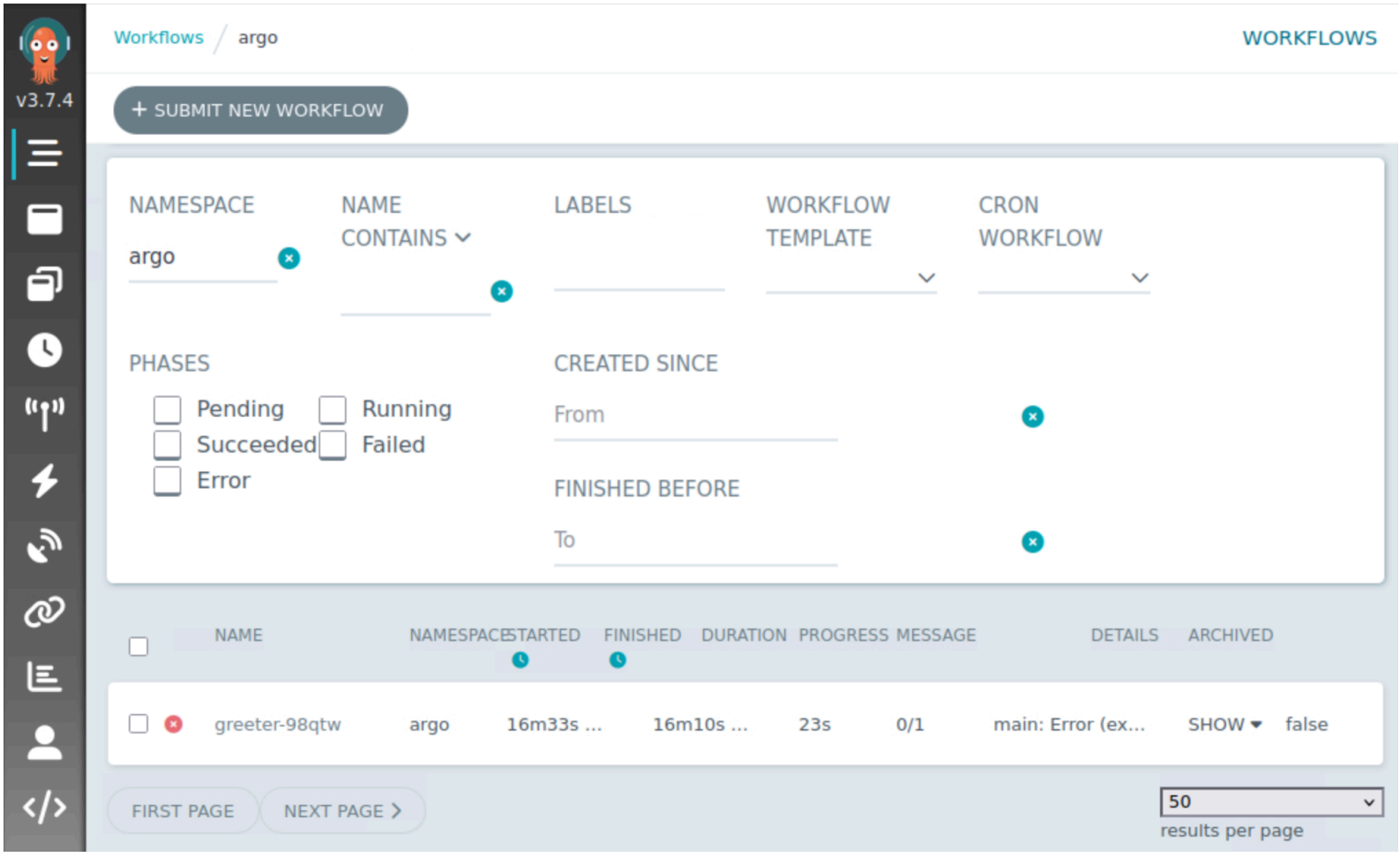
Step 1: Fix *WorkflowTemplate* via UI

A *WorkflowTemplate* defines a reusable workflow specification, while a *Workflow* is an instantiated run that uses such a workflow specification to execute tasks.

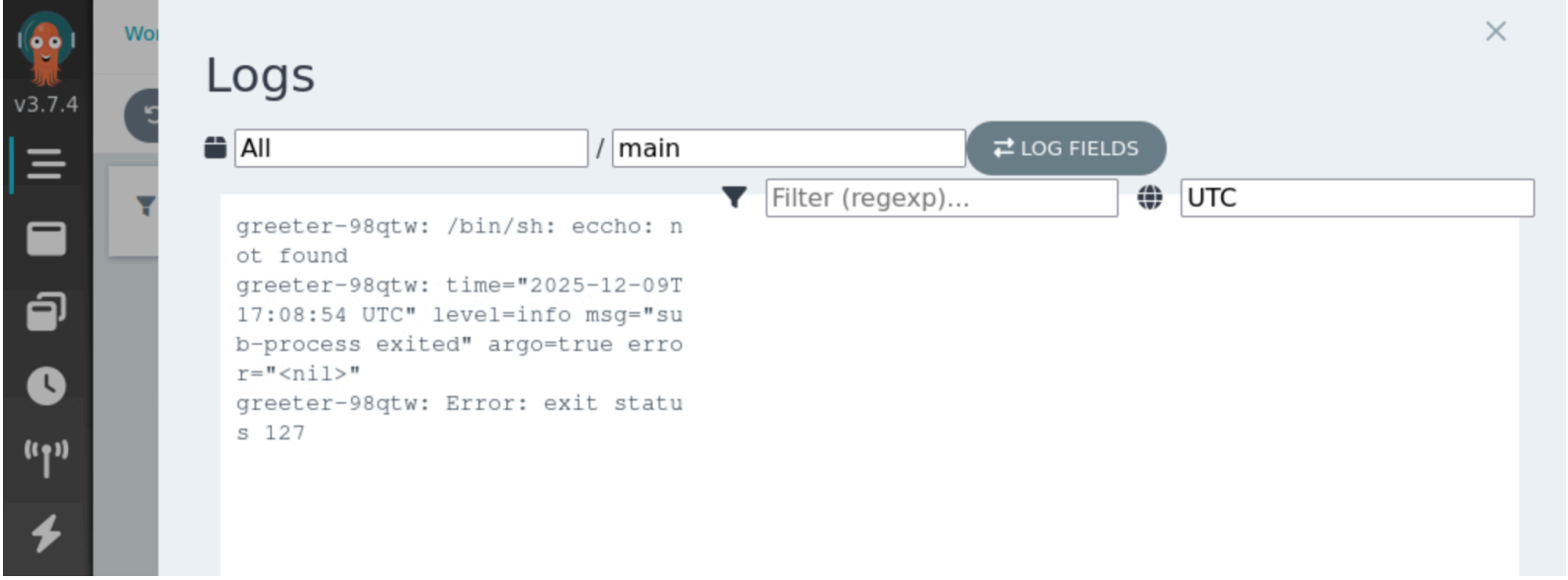
In the web interface under Workflow Templates we can see the `greeter`:



And under Workflows we can see the failed run of that template:

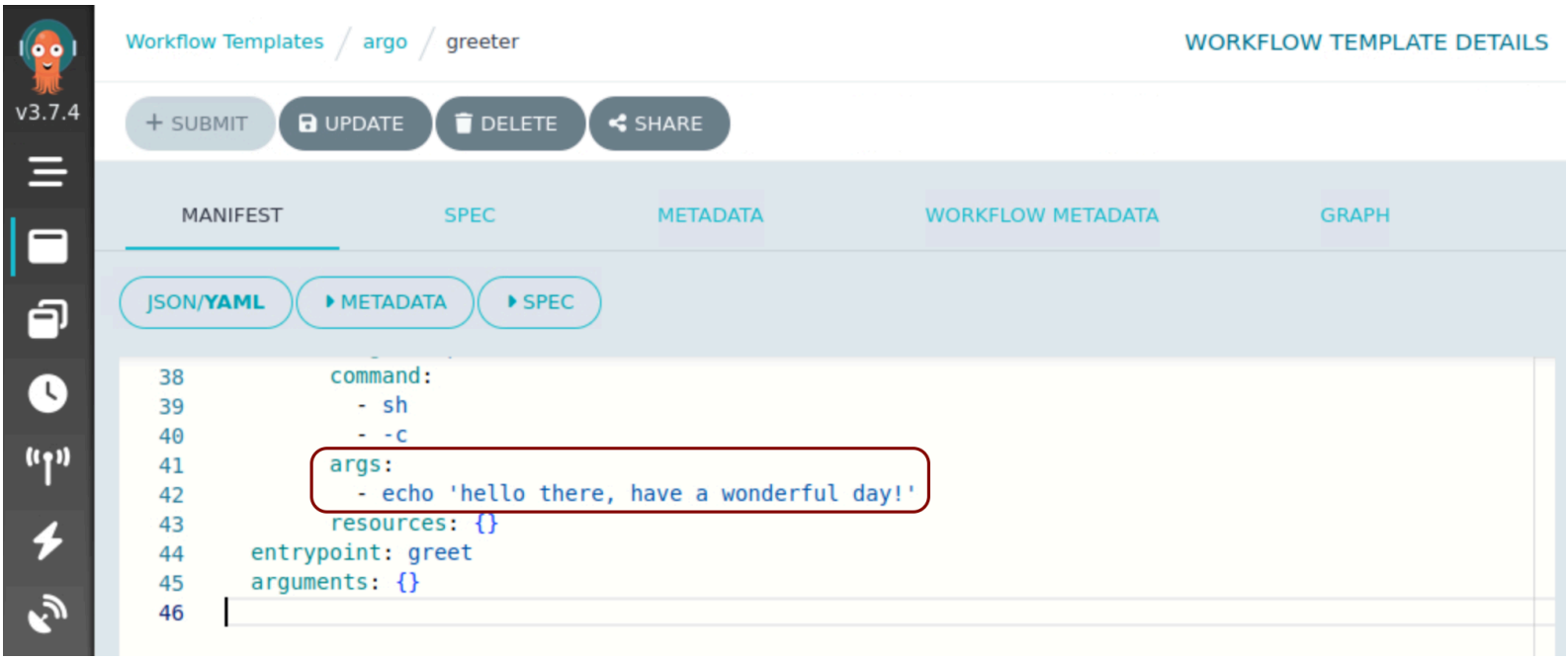


If we select it and click on Logs we can see the error:

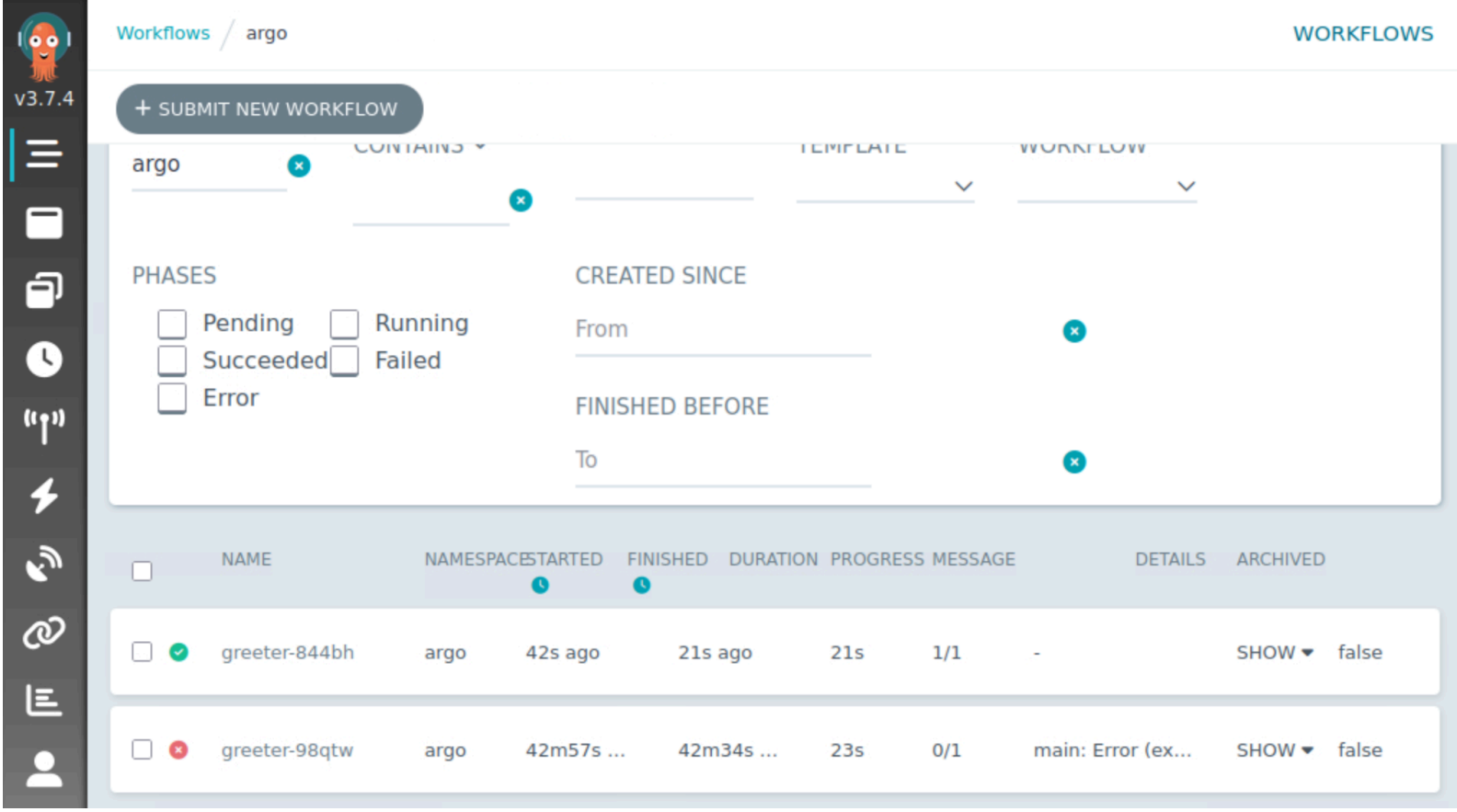


It looks like a typo because the command `eccho` was not found.

To fix this, head to the Workflow Template section and edit `greeter`:



We click Update, then head to Workflows and submit a new one for *WorkflowTemplate* `greeter`. We should see the new one succeed:



Now all we still have to do is to **delete the failed one**, and we can do this via the UI.

Step 1: Fix *WorkflowTemplate* via CLI

We now want to solve this step without the UI. But in reality we could use a combination of both. Let's view existing *Workflows*:

```
→ ssh cnpe3849

→ candidate@cnpe3849:~$ argo -n argo list
NAME            STATUS   AGE   DURATION   PRIORITY   MESSAGE
greeter-98qtw   Failed   51m   23s        0          main: Error (exit code 127)

→ candidate@cnpe3849:~$ argo -n argo get greeter-98qtw
Name:                greeter-98qtw
Namespace:           argo
ServiceAccount:      unset
Status:              Failed
Message:             main: Error (exit code 127)
Conditions:
 PodRunning          False
 Completed           True
Created:             Tue Dec 09 17:08:36 +0000 (51 minutes ago)
Started:             Tue Dec 09 17:08:37 +0000 (51 minutes ago)
Finished:            Tue Dec 09 17:09:00 +0000 (51 minutes ago)
Duration:            23 seconds
Progress:            0/1
ResourcesDuration:   0s*(1 cpu),15s*(100Mi memory)

STEP              TEMPLATE   PODNAME         DURATION   MESSAGE
 ✖ greeter-98qtw  greet      greeter-98qtw   18s        main: Error (exit code 127)

→ candidate@cnpe3849:~$ argo -n argo logs greeter-98qtw
greeter-98qtw: /bin/sh: eccho: not found
greeter-98qtw: time="2025-12-09T17:08:54.954Z" level=info msg="sub-process exited" argo=true error="<nil>"
```

```
greeter-98qtw: Error: exit status 127
```

The error is because of a typo in the command `eccho`. We can do the same also by just using kubectl:

```
→ candidate@cnpe3849:~$ k -n argo get workflow
NAME            STATUS   AGE   MESSAGE
greeter-98qtw   Failed   46m   main: Error (exit code 127)

→ candidate@cnpe3849:~$ k -n argo get pod greeter-98qtw
NAME            READY   STATUS   RESTARTS   AGE
greeter-98qtw   0/2     Error    0          46m

→ candidate@cnpe3849:~$ k -n argo logs greeter-98qtw
/bin/sh: eccho: not found
time="2025-12-09T17:08:54.954Z" level=info msg="sub-process exited" argo=true error="<nil>"
Error: exit status 127
```

Using just kubectl also works because Argo Workflows manages the *CRD Workflow* and creates *Pods* for it. We can think of a *Workflow* as similar to a *Deployment*: both create and manage *Pods* for their needs.

Now we fix the *WorkflowTemplate*:

```
→ candidate@cnpe3849:~$ k -n argo edit workflowtemplate greeter
```

```
# kubectl -n argo edit workflowtemplate greeter
apiVersion: argoproj.io/v1alpha1
kind: WorkflowTemplate
metadata:
...
  name: greeter
  namespace: argo
spec:
  arguments: {}
  entrypoint: greet
  serviceAccountName: argo
  templates:
  - container:
      args:
      - echo 'hello there, have a wonderful day!'    # FIX
      command:
      - sh
      - -c
      image: alpine:3
      name: ""
      resources: {}
...
```

Once done we create a new *Workflow*:

```
→ candidate@cnpe3849:~$ argo submit -n argo --from workflowtemplate/greeter
Name:                greeter-qqz65
Namespace:           argo
ServiceAccount:      unset
Status:              Pending
Created:             Tue Dec 09 18:04:40 +0000 (now)
Progress:

→ candidate@cnpe3849:~$ argo -n argo list
NAME            STATUS    AGE   DURATION   PRIORITY   MESSAGE
greeter-qqz65   Running   2s    1s         0
```

```
greeter-98qtw   Failed    56m   23s         0          main: Error (exit code 127)

→ candidate@cnpe3849:~$ argo -n argo list
NAME            STATUS      AGE   DURATION   PRIORITY   MESSAGE
greeter-qqz65   Succeeded   30s   24s        0
greeter-98qtw   Failed      56m   23s        0          main: Error (exit code 127)
```

Looking great! And finally we delete the failed one:

```
→ candidate@cnpe3849:~$ argo -n argo delete greeter-98qtw
Workflow 'greeter-98qtw' deleted

→ candidate@cnpe3849:~$ argo -n argo list
NAME            STATUS      AGE   DURATION   PRIORITY   MESSAGE
greeter-qqz65   Succeeded   1m    24s        0
```

Alternatively we could delete like this: `kubectl -n argo delete workflow greeter-98qtw`.

Step 2

This *WorkflowTemplate* is a bit more complex. As provided it creates a *ConfigMap* named `cm1` in *Namespace* `default` (or a custom one passed as an argument).

We are tasked with adding a second step, which runs parallel to the first one and creates a similar *ConfigMap* named `cm2`.

```
→ candidate@cnpe3849:~$ vim /course/11/configurator.yaml
```

```yaml
# cnpe3849:/course/11/configurator.yaml
apiVersion: argoproj.io/v1alpha1
kind: WorkflowTemplate
metadata:
  name: configurator
  namespace: argo
  annotations:
    description: "Generates certain ConfigMaps in passed Namespace"
spec:
  entrypoint: chain
  serviceAccountName: argo

  arguments:
    parameters:
      - name: target_namespace
        value: "default"

  templates:
    - name: chain
      steps:
        - - name: config1
            template: create-config1
            arguments:
              parameters:
                - name: ns
                  value: "{{workflow.parameters.target_namespace}}"

          # run step in parallel
          - name: config2                                          # ADD
            template: create-config2                               # ADD
            arguments:                                             # ADD
              parameters:                                          # ADD
                - name: ns                                         # ADD
```

```
                value: "{{workflow.parameters.target_namespace}}"        # ADD

    - name: create-config1
      inputs:
        parameters:
          - name: ns
      container:
        image: alpine/kubectl:latest
        command: ["/bin/sh","-c", "kubectl -n {{inputs.parameters.ns}} create configmap cm1 --from-
literal=debug=true -o yaml --dry-run=client | kubectl apply -f -"]

# ADD FROM HERE DOWN
    - name: create-config2
      inputs:
        parameters:
          - name: ns
      container:
        image: alpine/kubectl:latest
        command: ["/bin/sh","-c", "kubectl -n {{inputs.parameters.ns}} create configmap cm2 --from-
literal=debug=true -o yaml --dry-run=client | kubectl apply -f -"]
```

We copied the step and just made some changes:

1. Set `name: create-config2`
2. Create *ConfigMap* `cm2` instead of `cm1`

For the `steps:` section:

- `- -` (two dashes): Starts a new sequential step (runs *after* the previous block)
- `-` (one dash): Adds to the current block (runs *in parallel* with others in the same block)

If we wanted to run in sequence we would do:

```
# example showing how to run steps in sequence
      steps:
        - - name: config1
            template: create-config1
            arguments:
              parameters:
                - name: ns
                  value: "{{workflow.parameters.target_namespace}}"

          # add to run in sequence
        - - name: config2                                                 # ADD
            template: create-config2                                      # ADD
            arguments:                                                    # ADD
              parameters:                                                 # ADD
                - name: ns                                                # ADD
                  value: "{{workflow.parameters.target_namespace}}"       # ADD
```

Now we apply the changes and try to create a *Workflow*:

```
➜ candidate@cnpe3849:~$ k apply -f /course/11/configurator.yaml
workflowtemplate.argoproj.io/configurator configured

➜ candidate@cnpe3849:~$ argo submit -n argo --from workflowtemplate/configurator -p target_namespace=kaw
Name:                configurator-ntfwx
Namespace:           argo
ServiceAccount:      unset
Status:              Pending
Created:             Tue Dec 09 19:53:16 +0000 (now)
Progress:
```

```
Parameters:
  target_namespace:  kaw

→ candidate@cnpe3849:~$ argo wait -n argo @latest
@latest Succeeded at 2025-12-09 19:54:08 +0000 UTC

→ candidate@cnpe3849:~$ argo -n argo list
NAME                STATUS      AGE   DURATION   PRIORITY   MESSAGE
configurator-ntfwx  Succeeded   1m    51s        0
greeter-ctqzh       Succeeded   1h    42s        0
```

And the result should be that we see two *ConfigMaps* created with the same AGE:

```
→ candidate@cnpe3849:~$ k -n kaw get cm
NAME              DATA   AGE
cm1               1      2m22s
cm2               1      2m22s
kube-root-ca.crt  1      31m
```

If we look at the *Pods* created we should see two, because the steps run parallel in that way:

```
→ candidate@cnpe3849:~$ k -n argo get pod
NAME                                          READY   STATUS      ...
...
configurator-ntfwx-create-config1-3209532527  0/2     Completed   ...
configurator-ntfwx-create-config2-3226310146  0/2     Completed   ...
...
```

▼ Question 12| Tekton

Solve this question on: `ssh cnpe2561`

Your platform team uses Tekton to automate various team tasks. Tekton Pipelines has been installed, with `tkn` CLI, `kubectl tkn` plugin and the Tekton Dashboard at `http://cnpe2561:30120`.

All Tekton *Pipelines* etc should run in *Namespace* `builder`. The code for two pipelines is available at `/course/12`.

1. Add new *Task* `p1-create-labels` to the *Pipeline* `p1-team-onboarding`:
   - It should add label `auto-created: true` to the *Namespace* that was created in `p1-create-namespace`
   - It should run in parallel with *Task* `p1-create-roles`
   - Run the updated *Pipeline* for team name `butter`
   - Run the updated *Pipeline* for team name `croissant`
2. Apply all resources from `/course/12/p2-team-scanner`:
   - Run the *Pipeline* `p2-team-scanner` with:
     - team-name: `bread`
     - forbidden1: `miner`
     - forbidden2: `torrent`
   - Write the *PipelineRun* logs into `/course/12/p2.log`

If a *Pipeline* you run fails, delete the failed *PipelineRun* for cleanup.

Answer:

Tekton is a powerful and flexible open-source framework for creating CI/CD systems, allowing developers to build, test, and deploy across cloud providers and on-premise systems. Tekton provides some *CRDs* like:

- *Task*: A template for a single job (like "build" or "test") consisting of sequential steps that always run together inside one *Pod*
- *Pipeline:* A workflow definition that stitches multiple *Tasks* together into a graph to define their execution order and data sharing
- *PipelineRun:* The actual execution instance of a *Pipeline* that binds specific parameters and data to the template to trigger the workload on the cluster

Step 1: Overview of *Pipeline*

The *Pipeline* `p1-team-onboarding` references two *Task* resources and it accepts one string team-name as parameter. It will do the following if we run it with `butter` as team-name:

1. Receive `butter` as "team-name"
2. Calls *Task* `p1-create-namespace` which creates *Namespace* `team-butter`. So it adds a `team-` prefix
3. Calls *Task* `p1-create-roles` which creates *RoleBinding* `view-access` in *Namespace* `team-butter`

Let's have a look at how it's implemented:

```
➜ ssh cnpe2561

➜ candidate@cnpe2561:~$ tkn -n builder pipeline list
NAME                AGE              LAST RUN   STARTED   DURATION   STATUS
p1-team-onboarding   32 seconds ago   ---        ---       ---        ---

➜ candidate@cnpe2561:~$ tkn -n builder pipeline describe p1-team-onboarding
Name:        p1-team-onboarding
Namespace:   builder

⚓ Params

 NAME          TYPE     DESCRIPTION   DEFAULT VALUE
 • team-name   string                 ---

📋 Tasks

 NAME                TASKREF              RUNAFTER           ...
 • create-namespace   p1-create-namespace                    ...
 • create-roles       p1-create-roles      create-namespace   ...
```

Using the CLI we can list and describe *Pipelines* and it shows a great overview of which *Tasks* it contains and how they're executed. We see two *Tasks* right now that run in sequence, because `create-roles` has the RUNAFTER set to `create-namespace`.

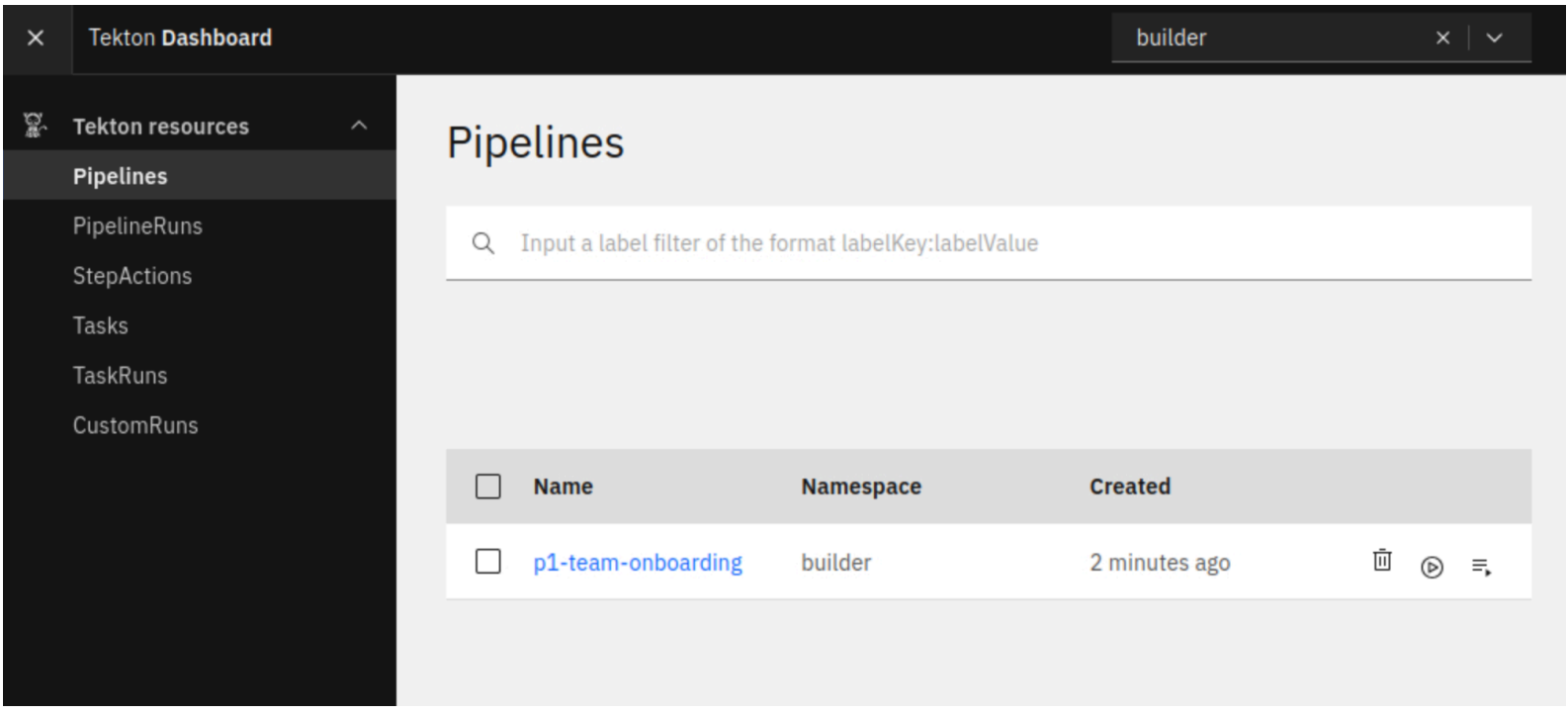
Since Tekton uses *CRDs*, we can also manage them using standard `kubectl` commands:

```
➜ candidate@cnpe2561:~$ k -n builder get pipeline
NAME                AGE
p1-team-onboarding   68s

➜ candidate@cnpe2561:~$ k -n builder describe pipeline p1-team-onboarding
Name:         p1-team-onboarding
```

```
Namespace:    builder
Labels:       <none>
Annotations:  <none>
API Version:  tekton.dev/v1
Kind:         Pipeline
Metadata:
  Creation Timestamp:  2025-12-10T17:39:07Z
  Generation:          1
  Resource Version:    172143
  UID:                 c75fa8cc-8fac-4981-a1b5-6d7273592e86
Spec:
  Params:
    Name:  team-name
    Type:  string
  Tasks:
    Name:  create-namespace
    Params:
      Name:   ns-name
      Value:  team-$(params.team-name)
    Task Ref:
      Kind:  Task
      Name:  p1-create-namespace
    Name:    create-roles
    Params:
      Name:   ns-name
      Value:  team-$(params.team-name)
    Run After:
      create-namespace
    Task Ref:
      Kind:  Task
      Name:  p1-create-roles
Events:      <none>
```

It can definitely be useful, but the `tkn` output is much more human readable.

If we log into the UI we can also see the pipeline:



Step 1: Add new *Task* to *Pipeline*

We need to add a new *Task* to the pipeline `team-onboarding`:

```
➜ candidate@cnpe2561:~$ cd /course/12/p1-team-onboarding

➜ candidate@cnpe2561:/course/12/p1-team-onboarding$ vim pipeline.yaml
```

```yaml
# cnpe2561:/course/12/p1-team-onboarding/pipeline.yaml
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: p1-create-namespace
  namespace: builder
spec:
  params:
    - name: ns-name
      type: string
  steps:
    - name: create
      image: alpine/kubectl:latest
      script: |
        echo "Creating namespace $(params.ns-name)..."
        kubectl create ns $(params.ns-name)
        until kubectl get sa default -n $(params.ns-name); do sleep 1; done
---
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: p1-create-roles
  namespace: builder
spec:
  params:
    - name: ns-name
      type: string
  steps:
    - name: create-role
      image: alpine/kubectl:latest
      script: |
        echo "Creating roles in namespace $(params.ns-name)..."
        kubectl create rolebinding view-access \
        --clusterrole=view \
        --serviceaccount=$(params.ns-name):default \
        -n $(params.ns-name)
---
### ADD FROM HERE
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: p1-create-labels
  namespace: builder
spec:
  params:
    - name: ns-name
      type: string
  steps:
    - name: create-label
      image: alpine/kubectl:latest
      script: |
        kubectl label namespace $(params.ns-name) auto-created=true
---
### ADD UNTIL HERE
apiVersion: tekton.dev/v1beta1
kind: Pipeline
metadata:
```

```
  name: p1-team-onboarding
  namespace: builder
spec:
  params:
    - name: team-name
      type: string
  tasks:
    - name: create-namespace
      taskRef:
        name: p1-create-namespace
      params:
        - name: ns-name
          value: "team-$(params.team-name)"
    - name: create-roles
      taskRef:
        name: p1-create-roles
      runAfter:
        - create-namespace
      params:
        - name: ns-name
          value: "team-$(params.team-name)"
### ADD FROM HERE
    - name: create-labels
      taskRef:
        name: p1-create-labels
      runAfter:
        - create-namespace
      params:
        - name: ns-name
          value: "team-$(params.team-name)"
### ADD UNTIL HERE
```

Above we simply copied one *Task* and made the changes. Then we referenced it under `tasks:` in the *Pipeline*. Important that we add the runAfter `p1-create-namespace`, that way it will run in parallel with `p1-create-roles`.

We diff:

```
➜ candidate@cnpe2561:/course/12/p1-team-onboarding$ k diff -f pipeline.yaml
diff -u -N /tmp/LIVE-3285987709/tekton.dev.v1beta1.Pipeline.builder.p1-team-onboarding /tmp/MERGED-
315835011/tekton.dev.v1beta1.Pipeline.builder.p1-team-onboarding
...
     taskRef:
       kind: Task
       name: p1-create-roles
+  - name: create-labels
+    params:
+    - name: ns-name
+      value: team-$(params.team-name)
+    runAfter:
+    - create-namespace
+    taskRef:
+      kind: Task
+      name: p1-create-labels
diff -u -N /tmp/LIVE-3285987709/tekton.dev.v1beta1.Task.builder.p1-create-labels
...
+apiVersion: tekton.dev/v1beta1
+kind: Task
+metadata:
+  creationTimestamp: "2025-12-10T17:49:59Z"
+  generation: 1
+  name: p1-create-labels
+  namespace: builder
```

```
+  uid: a73f25c9-233c-4ee6-a851-d08e606c7e60
+spec:
+  params:
+  - name: ns-name
+    type: string
+  steps:
+  - image: alpine/kubectl:latest
+    name: create-label
+    resources: {}
+    script: |
+      kubectl label namespace $(params.ns-name) auto-created=true
```

And if we're satisfied we apply:

```
➜ candidate@cnpe2561:/course/12/p1-team-onboarding$ k apply -f pipeline.yaml
task.tekton.dev/p1-create-namespace configured
task.tekton.dev/p1-create-roles configured
task.tekton.dev/p1-create-labels created
pipeline.tekton.dev/p1-team-onboarding configured

➜ candidate@cnpe2561:/course/12/p1-team-onboarding$ tkn -n builder pipeline describe p1-team-onboarding
Name:        p1-team-onboarding
Namespace:   builder

⚓ Params

 NAME          TYPE     DESCRIPTION   DEFAULT VALUE
 • team-name   string                 ---

📦 Tasks

 NAME                TASKREF               RUNAFTER           TIMEOUT   PARAMS
 • create-namespace  p1-create-namespace                       ---       ns-name:
 • create-roles      p1-create-roles       create-namespace    ---       ns-name:
 • create-labels     p1-create-labels      create-namespace    ---       ns-name:
```

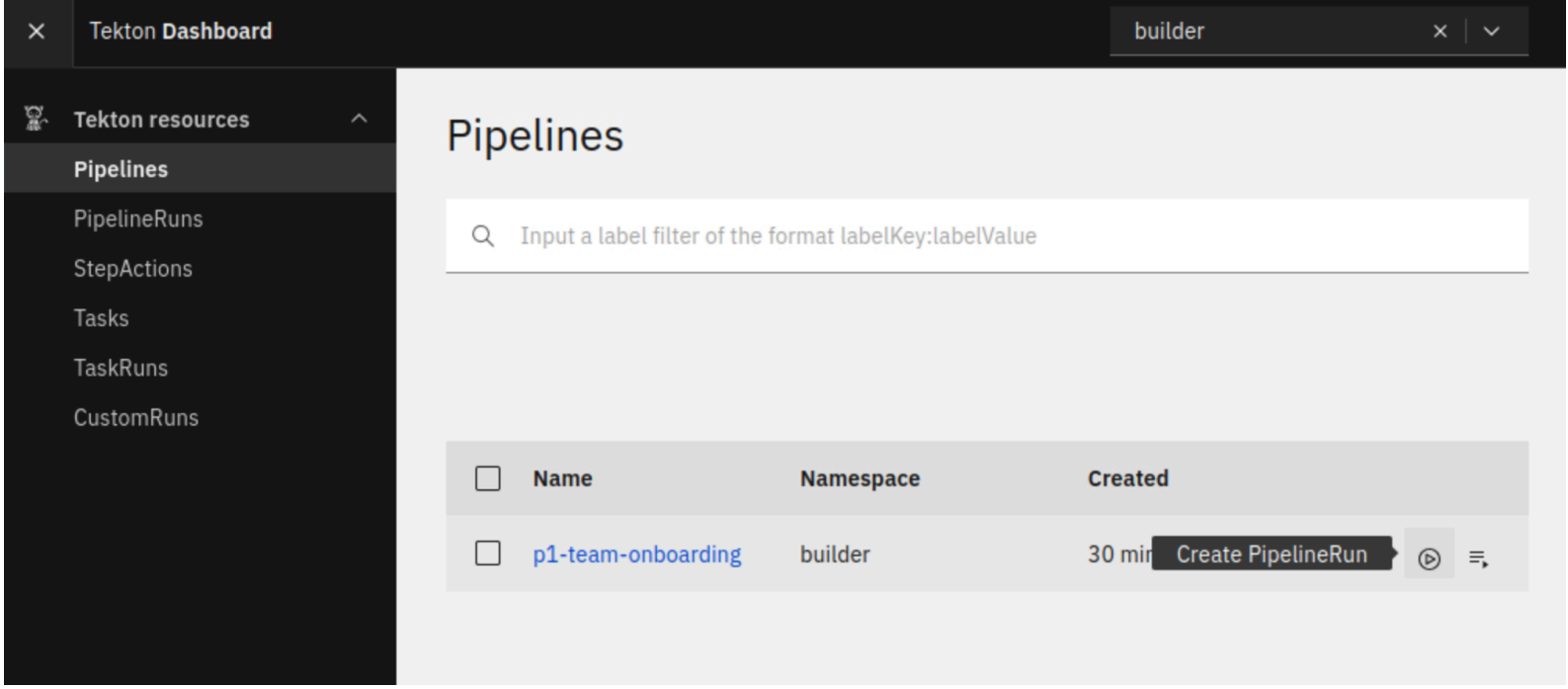
The result looks promising, now there are two *Tasks* `p1-create-roles` and `p1-create-labels` with RUNAFTER `create-namespace`, which means they'll run in parallel.

In Tekton Pipelines, if you do not explicitly define a dependency (using runAfter or from), all *Tasks* are scheduled to execute simultaneously the moment the *Pipeline* starts. The from keyword in Tekton is used to pass *PipelineResources* (like a Git repo or a Docker image) from one *Task* to another.

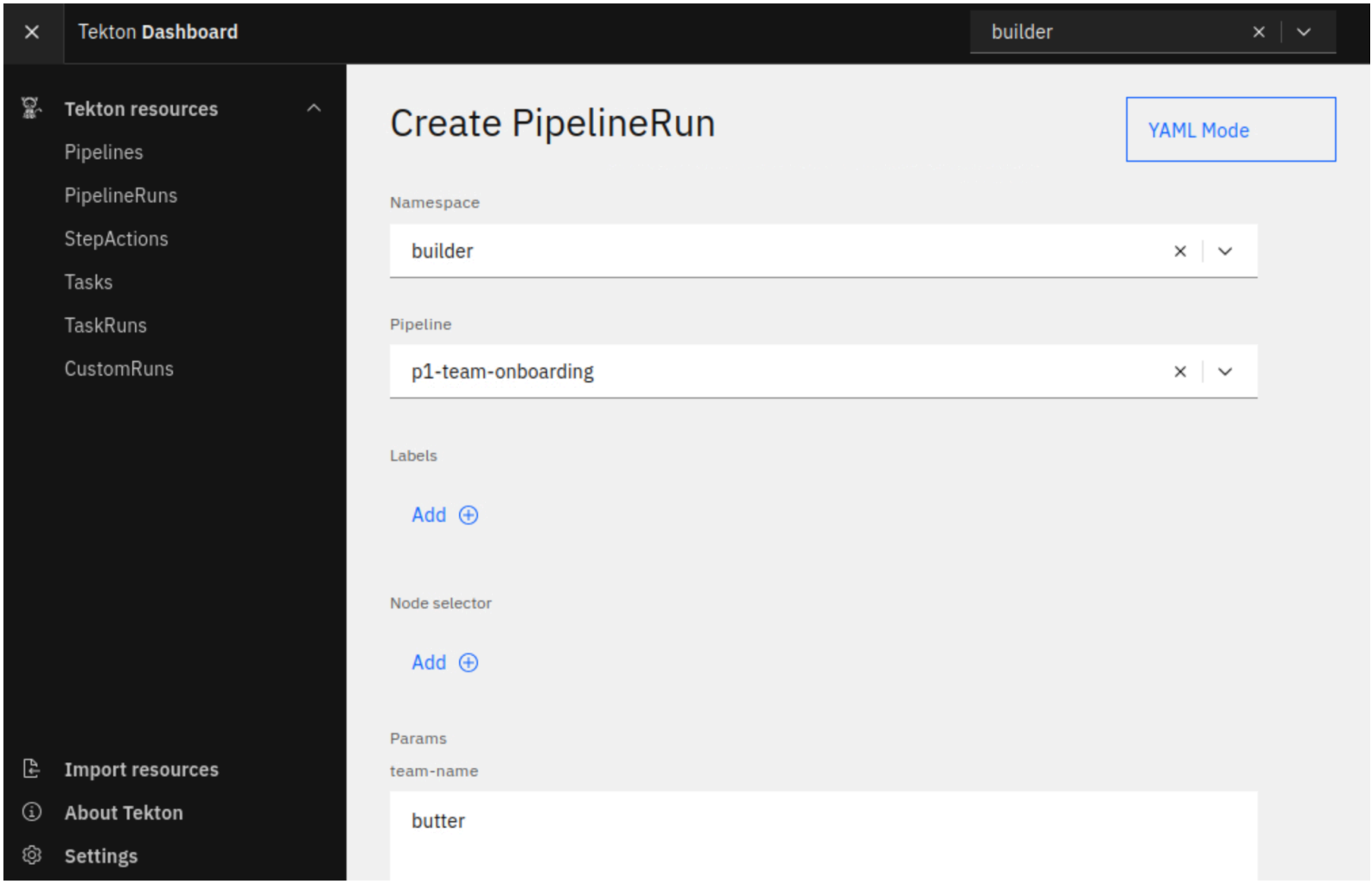
Step 1: Run *Pipeline* via dashboard

The scenario requires us to run *Pipeline* `p1-team-onboarding` for two teams named `butter` and `croissant`. Let's do `butter` first via the dashboard.

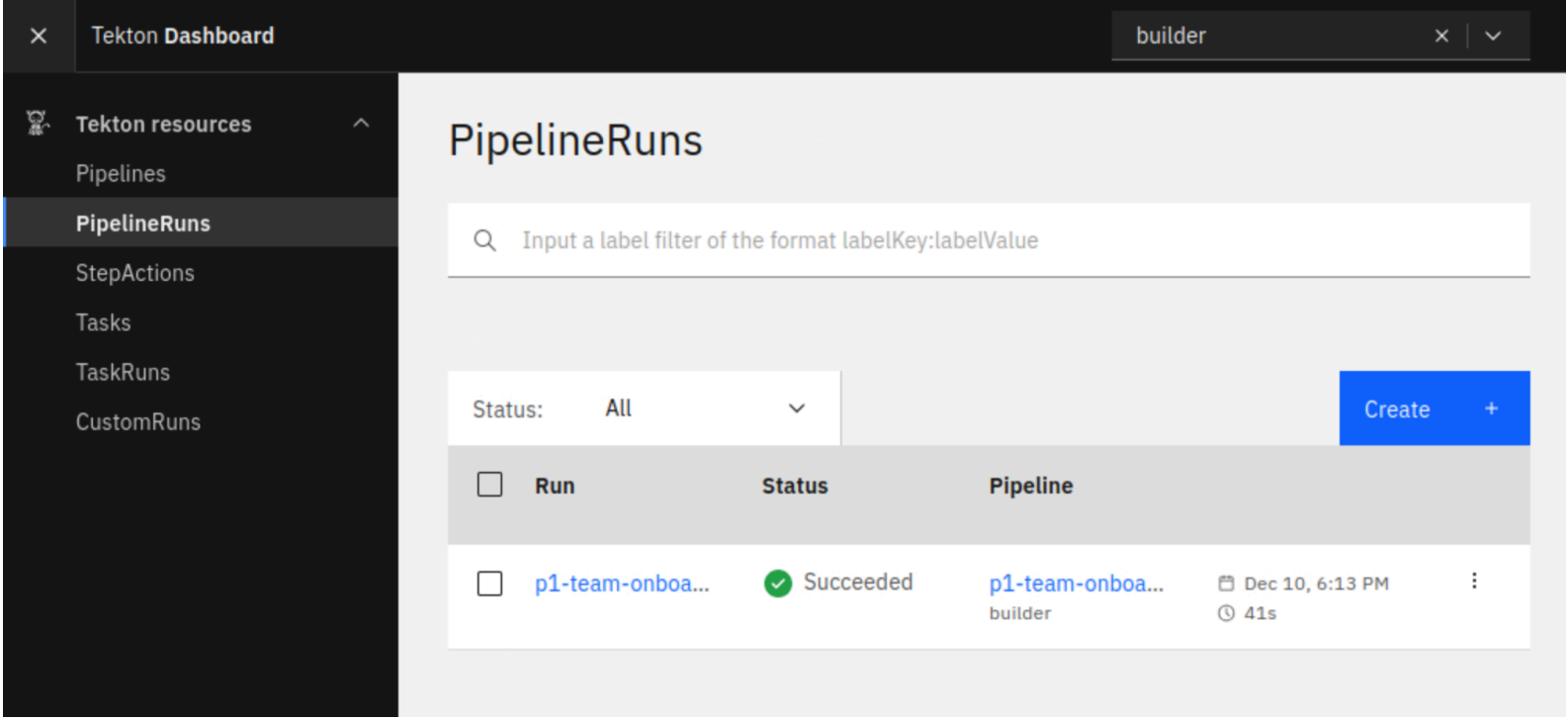
In the Pipeline overview click on the run icon. Or under PipelineRuns click on create:



Now enter `butter` for team-name and create:



If there have been errors we could view the logs, but hopefully we should see:



If we look at the *Pods* being created we would see three in total, and two running in parallel because we configured the `create-roles` and `create-labels` like that:

```
→ candidate@cnpe2561:~$ k -n builder get pod
NAME                                                    READY   STATUS
p1-team-onboarding-run-1765390400953-create-labels-pod      1/1     Running
p1-team-onboarding-run-1765390400953-create-namespace-pod   0/1     Completed
p1-team-onboarding-run-1765390400953-create-roles-pod       1/1     Running
```

Now we check the result, which should be a new *Namespace* `team-butter` with correct label and a *Rolebinding* in it:

```
→ candidate@cnpe2561:~$ k get ns team-butter --show-labels
NAME          STATUS   AGE     LABELS
team-butter   Active   6m35s   auto-created=true,...

→ candidate@cnpe2561:~$ k -n team-butter get rolebinding
NAME          ROLE               AGE
view-access   ClusterRole/view   6m17s
```

The *PipelineRun* status should also show Succeeded:

```
→ candidate@cnpe2561:~$ tkn -n builder pipelinerun list
NAME                                        STARTED          DURATION   STATUS
p1-team-onboarding-run-1765390400953   1 minutes ago   41s        Succeeded
```

Step 1: Run *Pipeline* via CLI

The scenario requires us to run *Pipeline* `p1-team-onboarding` for two teams named `butter` and `croissant`. Let's do `croissant` now via CLI:

```
➜ candidate@cnpe2561:~$ tkn -n builder pipeline start p1-team-onboarding
? Value for param `team-name` of type `string`? croissant
PipelineRun started: p1-team-onboarding-run-b28nl

In order to track the PipelineRun progress run:
tkn pipelinerun logs p1-team-onboarding-run-b28nl -f -n builder
```

It interactively asks us for the team-name. And it even provides us with the command to run to follow along:

```
➜ candidate@cnpe2561:~$ tkn pipelinerun logs p1-team-onboarding-run-b28nl -f -n builder
[create-namespace : create] Creating namespace team-croissant...
[create-namespace : create] namespace/team-croissant created

[create-roles : create-role] Creating roles in namespace team-croissant...
[create-roles : create-role] rolebinding.rbac.authorization.k8s.io/view-access created
[create-labels : create-label] namespace/team-croissant labeled
```

The result:

```
➜ candidate@cnpe2561:~$ tkn -n builder pipelinerun list
NAME                                  STARTED          DURATION   STATUS
p1-team-onboarding-run-b28nl          1 minute ago     55s        Succeeded
p1-team-onboarding-run-1765390400953  11 minutes ago   41s        Succeeded

➜ k get ns -l auto-created
NAME            STATUS   AGE
team-butter     Active   12m
team-croissant  Active   2m46s
```

Step 2: Apply resources, run *Pipeline* and collect logs

First let's have a look at the resources:

```
➜ candidate@cnpe2561:~$ cd /course/12/p2-team-scanner

➜ candidate@cnpe2561:/course/12/p2-team-scanner$ vim pipeline.yaml
```

```yaml
# cnpe2561:/course/12/p2-team-scanner/pipeline.yaml
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: p2-scan
  namespace: builder
spec:
  params:
    - name: ns-name
      type: string
    - name: forbidden
      type: string
  steps:
    - name: scan
      image: alpine/kubectl:latest
      script: |
        echo "Scanning in namespace $(params.ns-name) for $(params.forbidden)..."
        kubectl get pods -n $(params.ns-name) -oyaml | grep $(params.forbidden) && \
        echo "Forbidden resource $(params.forbidden) found in namespace $(params.ns-name)! Alert sent" || \
        echo "No forbidden resources found in namespace $(params.ns-name)."
```

```yaml
---
apiVersion: tekton.dev/v1beta1
kind: Pipeline
metadata:
  name: p2-team-scanner
  namespace: builder
spec:
  params:
    - name: team-name
      type: string
    - name: forbidden1
      type: string
    - name: forbidden2
      type: string
  tasks:
    - name: scan1
      taskRef:
        name: p2-scan
      params:
        - name: ns-name
          value: "team-$(params.team-name)"
        - name: forbidden
          value: "$(params.forbidden1)"
    - name: scan2
      taskRef:
        name: p2-scan
      params:
        - name: ns-name
          value: "team-$(params.team-name)"
        - name: forbidden
          value: "$(params.forbidden2)"
```

This *Pipeline* only has one *Task* which it calls two times in parallel. It will search the YAML of all *Pods* in a team-*Namespace* for the forbidden words.

Now we go ahead and create the resources:

```
→ candidate@cnpe2561:/course/12/p2-team-scanner$ k apply -f pipeline.yaml
task.tekton.dev/p2-scan created
pipeline.tekton.dev/p2-team-scanner created

→ candidate@cnpe2561:/course/12/p2-team-scanner$ k -n builder get pipeline
NAME                AGE
p1-team-onboarding   86m
p2-team-scanner      10s

→ candidate@cnpe2561:/course/12/p2-team-scanner$ tkn -n builder pipeline list
NAME                AGE              LAST RUN                      ...
p1-team-onboarding  1 hour ago       p1-team-onboarding-run-b28nl  ...
p2-team-scanner     24 seconds ago   ---
```

Next we can call the *Pipeline* as required in the question:

```
→ candidate@cnpe2561:~$ tkn -n builder pipeline start p2-team-scanner
? Value for param `team-name` of type `string`? bread
? Value for param `forbidden1` of type `string`? miner
? Value for param `forbidden2` of type `string`? torrent
PipelineRun started: p2-team-scanner-run-8smw6

In order to track the PipelineRun progress run:
tkn pipelinerun logs p2-team-scanner-run-8smw6 -f -n builder
```

```
➜ candidate@cnpe2561:~$ tkn pipelinerun logs p2-team-scanner-run-8smw6 -f -n builder
[scan1 : scan] Scanning in namespace team-bread for miner...
[scan2 : scan] Scanning in namespace team-bread for torrent...
[scan1 : scan]         value: miner
[scan1 : scan] Forbidden resource miner found in namespace team-bread! Alert sent
[scan2 : scan] No forbidden resources found in namespace team-bread.
```

If it finds a forbidden word it will not fail but just output the text `Forbidden resource...Alert sent`. The *Task* could for example contact an actual alert system in the future. Another possibility would be to let a *Task* fail if a forbidden value was found, then get notified for failed ones.

To solve this step we need to write the logs to the required location:

```
➜ candidate@cnpe2561:~$ tkn pipelinerun logs p2-team-scanner-run-8smw6 -f -n builder > /course/12/p2.log
```

▼ Question 13| Pod Security Standards

Solve this question on: `ssh cnpe0720`

The *Namespace* `ammersee-legacy` currently has no Pod Security Standards applied. The manifests for existing workloads are available at `/course/13`. You're tasked with:

1. Configure the *Namespace* to enforce the `restricted` Pod Security Standard
2. Identify and fix any non-compliant workloads so they can be restarted

Answer:

Introduction

Pod Security Standards define three levels:

1. `privileged` (unrestricted)
2. `baseline` (minimally restrictive, prevents known privilege escalations)
3. `restricted` (heavily restricted, follows hardening best practices)

These can be applied in three modes:

1. `enforce` (rejects *Pods* that violate)
2. `audit` (logs violations in audit logs)
3. `warn` (shows warning to users but allows creation)

The `restricted` level requires:

- `runAsNonRoot: true`
- `allowPrivilegeEscalation: false`
- `seccompProfile` must be set (typically `RuntimeDefault`)
- Capabilities must drop `ALL`
- No privileged containers, hostNetwork, hostPID, etc.

Step 1: Apply restricted standard in enforce mode

First, let's check what's running:

```
➜ ssh cnpe0720

➜ candidate@cnpe0720:~$ k -n ammersee-legacy get pods
NAME                  READY   STATUS    RESTARTS   AGE
logging-agent-zhbq2   1/1     Running   0          68s
web-cache-0           1/1     Running   0          63s
web-cache-1           1/1     Running   0          59s

➜ candidate@cnpe0720:~$ k -n ammersee-legacy get ds,sts
NAME                DESIRED   CURRENT   READY   ...
.../logging-agent   1         1         1       ...

NAME                          READY   AGE
statefulset.apps/web-cache    2/2     3m25s
```

We can see one *DaemonSet* and one *StatefulSet*. Now we label the *Namespace* to enforce the `restricted` standard:

```
➜ candidate@cnpe0720:~$ k label namespace ammersee-legacy pod-security.kubernetes.io/enforce=restricted
Warning: existing pods in namespace "ammersee-legacy" violate the new PodSecurity enforce level
"restricted:latest"
Warning: logging-agent-zhbq2: allowPrivilegeEscalation != false, unrestricted capabilities, runAsNonRoot !=
true
namespace/ammersee-legacy labeled
```

We see a warning for the *DaemonSet Pod*, but the *StatefulSet Pods* are already compliant. Existing *Pods* continue to run, PSS changes are only applied to created/restarted *Pods*.

Now if we try to restart, the new *Pods* will be rejected:

```
➜ candidate@cnpe0720:~$ k -n ammersee-legacy rollout restart ds logging-agent
Warning: would violate PodSecurity "restricted:latest": allowPrivilegeEscalation != false (container "agent"
must set securityContext.allowPrivilegeEscalation=false), unrestricted capabilities (container "agent" must
set securityContext.capabilities.drop=["ALL"]), runAsNonRoot != true (pod or container "agent" must set
securityContext.runAsNonRoot=true)
daemonset.apps/logging-agent restarted
```

We see no new *Pods* for the *DaemonSet* are being created.

```
➜ candidate@cnpe0720:~$ k -n ammersee-legacy get pod
NAME          READY   STATUS    RESTARTS   AGE
web-cache-0   1/1     Running   0          5m22s
web-cache-1   1/1     Running   0          5m20s
```

If we restart the *StatefulSet* we see no issues:

```
➜ candidate@cnpe0720:~$ k -n ammersee-legacy rollout restart sts web-cache
statefulset.apps/web-cache restarted

➜ candidate@cnpe0720:~$ k -n ammersee-legacy get pod
NAME          READY   STATUS    RESTARTS   AGE
web-cache-0   1/1     Running   0          2s
web-cache-1   1/1     Running   0          35s
```

Step 2: Fix the *DaemonSet*

Now we check for errors by describing the *DaemonSet*:

```
➜ candidate@cnpe0720:~$ k -n ammersee-legacy describe ds logging-agent
Name:           logging-agent
Namespace:      ammersee-legacy
Selector:       app=logging-agent
Node-Selector:  <none>
...
Events:
  Type     Reason            Age                From                  Message
  ----     ------            ----               ----                  -------
...
  Warning  FailedCreate      23s (x5 over 61s)  daemonset-controller  (combined from similar events): Error
creating: pods "logging-agent-b8jvk" is forbidden: violates PodSecurity "restricted:latest":
allowPrivilegeEscalation != false (container "agent" must set
securityContext.allowPrivilegeEscalation=false), unrestricted capabilities (container "agent" must set
securityContext.capabilities.drop=["ALL"]), runAsNonRoot != true (pod or container "agent" must set
securityContext.runAsNonRoot=true)
```

The errors tell us exactly what needs to be fixed:

- `allowPrivilegeEscalation` must be `false`
- Must drop `ALL` capabilities
- `runAsNonRoot` must be `true`

Edit the *DaemonSet* manifest:

```
➜ candidate@cnpe0720:~$ vim /course/13/daemonset.yaml
```

Update the `securityContext` section:

```yaml
# cnpe0720:/course/13/daemonset.yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: logging-agent
  namespace: ammersee-legacy
...
spec:
...
  template:
...
    spec:
      serviceAccountName: default
      containers:
      - name: agent
        image: busybox:1.36
```

```
        command: ["sleep", "infinity"]
...
        securityContext:
          runAsUser: 1000
          runAsNonRoot: true                  # ADD
          allowPrivilegeEscalation: false     # CHANGED from true
          seccompProfile:
            type: RuntimeDefault
          capabilities:                       # ADD
            drop:                             # ADD
              - ALL                           # ADD
...
```

Apply the changes:

```
➜ candidate@cnpe0720:~$ k apply -f /course/13/daemonset.yaml
daemonset.apps/logging-agent configured
```

The *DaemonSet* will now create compliant *Pods*:

```
➜ candidate@cnpe0720:~$ k -n ammersee-legacy get pods -l app=logging-agent
NAME                  READY   STATUS    RESTARTS   AGE
logging-agent-6dxtm   1/1     Running   0          16s
```

Verify all *Pods* are running:

```
➜ candidate@cnpe0720:~$ k -n ammersee-legacy get pods
NAME                  READY   STATUS    RESTARTS   AGE
logging-agent-6dxtm   1/1     Running   0          24s
web-cache-0           1/1     Running   0          6m
web-cache-1           1/1     Running   0          6m33s
```

The *StatefulSet* `web-cache` was already compliant with the `restricted` standard and didn't need any changes.

▼ Question 14 | Jaeger

Solve this question on: `ssh cnpe7683`

Namespace `eyre` contains a Jaeger instance with UI at `http://cnpe7683:30014` and multiple services which generate distributed traces. Using Jaeger:

1. Find the service with tag `ai.model=fast_v1.2` and update its *Deployment* to use `thinking_v1.6` instead
2. Find the service with tag `access.public=true` and scale its *Deployment* to `2` replicas
3. Export exactly `10` traces from service `speechai` in JSON format to `/course/14/traces.json` on `cnpe7683`

Answer:

Jaeger: Distributed tracing backend that collects, stores, and visualizes traces from instrumented applications

Trace: A complete record of a request's journey through a distributed system, identified by a unique trace ID

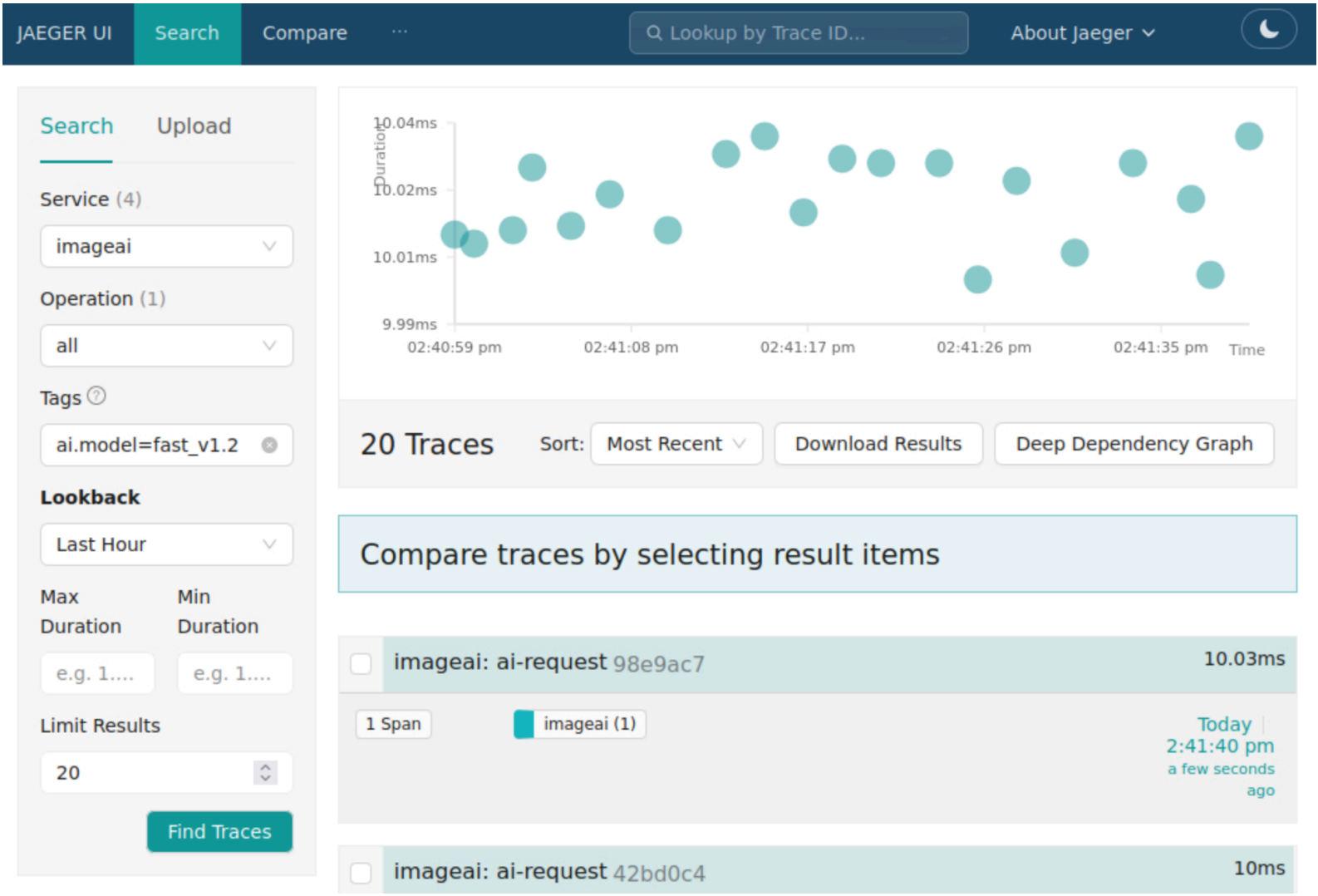
Span: A single operation within a trace, containing timing data, tags, and parent-child relationships

OpenTelemetry: Vendor-neutral instrumentation standard that applications use to generate and export traces

Step 1: Find service with specific tag and update *Deployment*

First we need to open Jaeger UI and filter for the tag:

1. Select a service from the dropdown
2. Click "Find Traces"
3. Repeat for a different service in the dropdown



If we go through all services we'll see that `imageai` is the one we need. The question mentions that we need to update the *Deployment*:

```
→ ssh cnpe7683
```

```
→ candidate@cnpe7683:~$ k -n eyre edit deploy imageai
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
...
  name: imageai
  namespace: eyre
spec:
...
  template:
...
    spec:
      containers:
      - args:
```

```
      - |
        apk add --no-cache curl > /dev/null 2>&1
        sh /scripts/send-traces.sh
      command:
      - /bin/sh
      - -c
      env:
      - name: SERVICE_NAME
        value: imageai
      - name: AI_MODEL
        value: thinking_v1.6    # CHANGED from fast_v1.2
      - name: ACCESS_PUBLIC
        value: "false"
...
```

Here we just have to search for the value in the *Deployment* and replace it. Check if the value is hardcoded in the *Deployment* or stored in a *ConfigMap*.

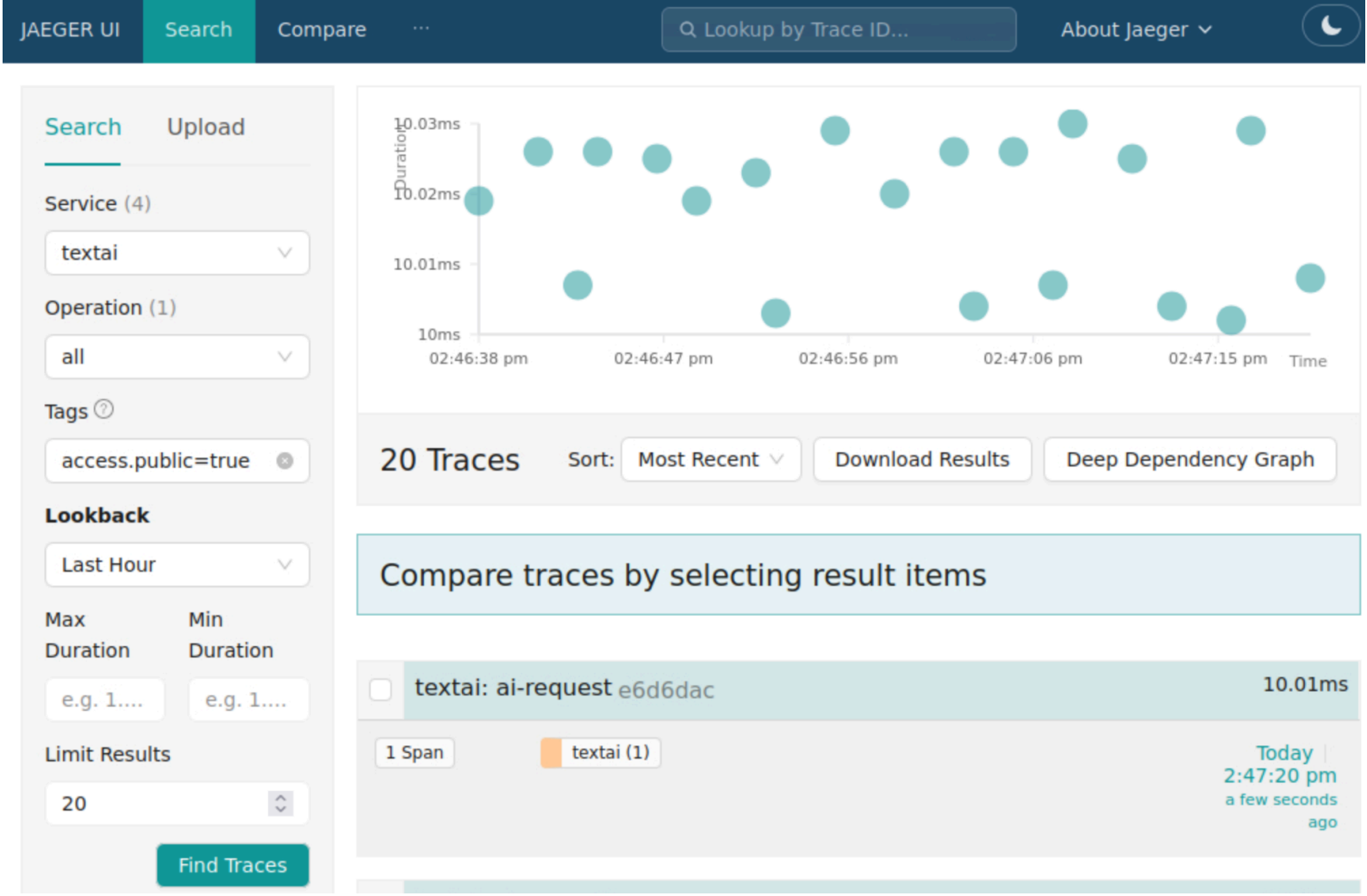
After the new *Pod* has been created we will see that the new traces in Jaeger have the updated `ai.model`.

We could also find the service via the Jaeger API:

```
➜ candidate@cnpe7683:~$ curl -s "http://localhost:30014/api/services" | yq '.data[]'
imageai
speechai
textai

➜ candidate@cnpe7683:~$ curl -s "http://localhost:30014/api/traces?service=imageai&limit=1" | grep fast_v1.2
{"data":[{"traceID":"0d7c5529c4d358049090903f234e29ec","spans":
[{"traceID":"0d7c5529c4d358049090903f234e29ec","spanID":"b756d5798e04b556","operationName":"ai-
request","references":[],"startTime":1767884486000000,"duration":10025,"tags":
[{"key":"access.public","type":"string","value":"false"},
{"key":"ai.completion.tokens","type":"int64","value":445},
{"key":"ai.model","type":"string","value":"fast_v1.2"},
{"key":"ai.prompt.tokens","type":"int64","value":352},
{"key":"ai.request.type","type":"string","value":"embedding"},
{"key":"http.status_code","type":"int64","value":500},{"key":"otel.scope.name","type":"string","value":"ai-
tracer"},{"key":"span.kind","type":"string","value":"server"}],"logs":
[],"processID":"p1","warnings":null}],"processes":{"p1":{"serviceName":"imageai","tags":
[]}},"warnings":null}],"total":0,"limit":0,"offset":0,"errors":null}
```

Step 2: Find service with specific tag and scale up the *Deployment*

Now we search for the service which has tag `access.public=true`.



The service is `textai`, so we scale it to 2 replicas:

```
→ candidate@cnpe7683:~$ k -n eyre scale deploy textai --replicas=2
deployment.apps/textai scaled

→ candidate@cnpe7683:~$ k -n eyre get deploy textai
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
textai    2/2     2            2           53m
```

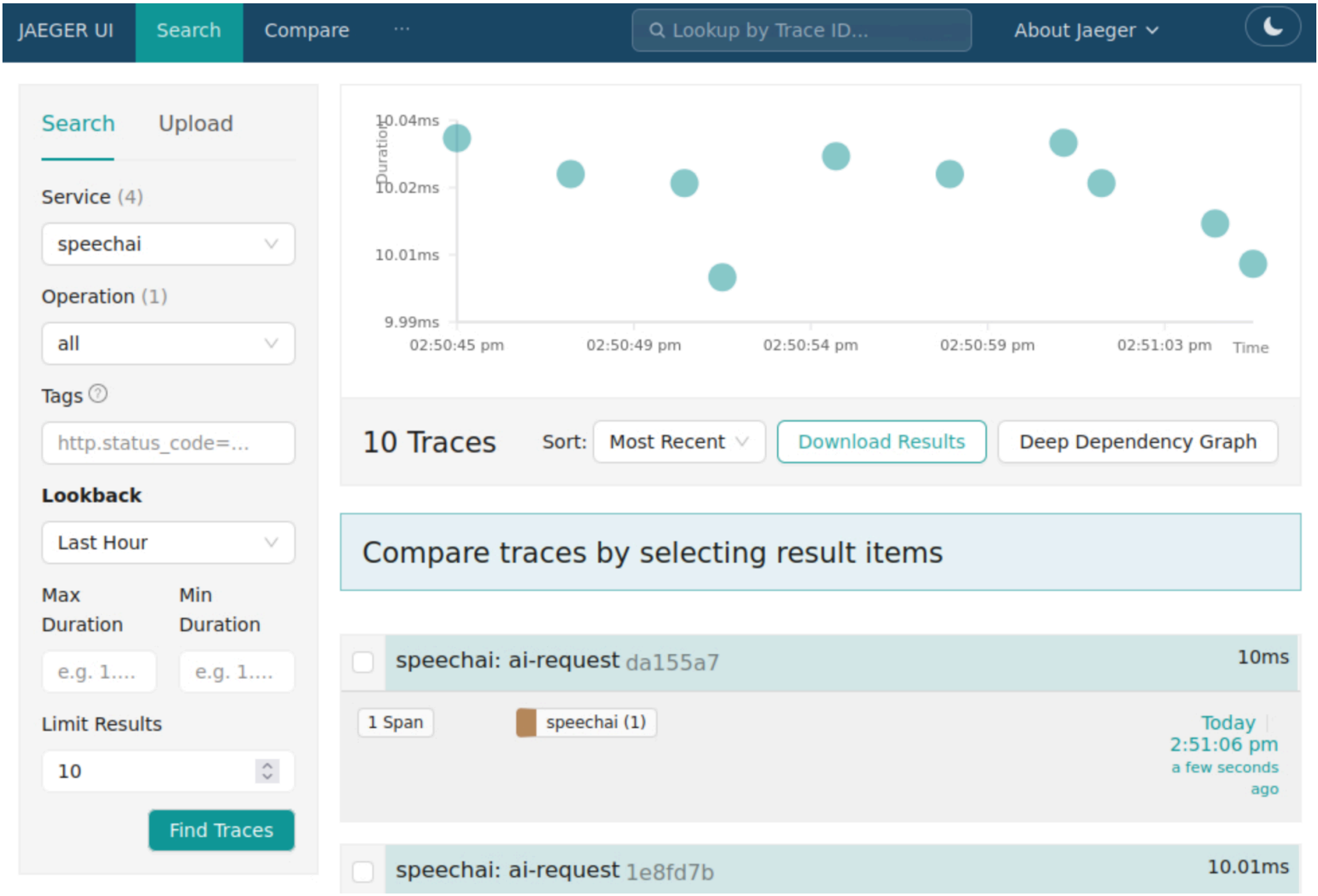
We could also find the service via the Jaeger API (see Step 1 for listing services):

```
→ candidate@cnpe7683:~$ curl -s "http://localhost:30014/api/traces?service=textai&limit=1" | grep
access.public
{"data":[{"traceID":"0e0fd95b1590b24cb5584cf2c2d601a3","spans":
[{"traceID":"0e0fd95b1590b24cb5584cf2c2d601a3","spanID":"60b0f9bf5d305c11","operationName":"ai-
request","references":[],"startTime":1767884431000000,"duration":10019,"tags":
[{"key":"access.public","type":"string","value":"true"},
{"key":"ai.completion.tokens","type":"int64","value":751},
{"key":"ai.model","type":"string","value":"cheap_v1.4"},
{"key":"ai.prompt.tokens","type":"int64","value":491},
{"key":"ai.request.type","type":"string","value":"embedding"},
{"key":"http.status_code","type":"int64","value":200},{"key":"otel.scope.name","type":"string","value":"ai-
tracer"},{"key":"span.kind","type":"string","value":"server"}],"logs":
[],"processID":"p1","warnings":null}],"processes":{"p1":{"serviceName":"textai","tags":
[]}},"warnings":null}],"total":0,"limit":0,"offset":0,"errors":null}
```

Step 3: Export traces

In the Jaeger UI:

1. Select service `speechai`
2. Set "Limit Results" to `10`
3. Click "Find Traces"
4. Click "Download Results" button



The file will be saved where the browser runs, which is the main `candidate@terminal`. This means we'll need to copy it to the required location on `cnpe7683` via `scp`:

```
→ candidate@terminal:~$ cd ~/Downloads

→ candidate@terminal:~/Downloads$ ls
traces-1767883959763.json

→ candidate@terminal:~/Downloads$ scp traces-1767883959763.json cnpe7683:/course/14/traces.json
traces-1767883959763.json  100% 9444     5.5MB/s   00:00
```

We could also achieve this via the Jaeger API:

```
→ candidate@cnpe7683:~$ curl -s "http://localhost:30014/api/traces?service=speechai&limit=10" >
/course/14/traces.json

→ candidate@cnpe7683:~$ cat /course/14/traces.json | yq '.data | length'
10
```

▼ Question 15 | Vertical Pod Autoscaler (VPA)

Solve this question on: `ssh cnpe1080`

A single etcd instance is running in *Namespace* `sargasso` and you should create a *VerticalPodAutoscaler* (*VPA*) resource for it.

Add *VPA* named `etcd-vpa` to file `/course/15/etcd.yaml` and create it. Don't make any changes to the *StatefulSet* in that file.

1. The *VPA* should only apply recommendations at *Pod* creation:
    - Minimum `cpu: 20m`, `memory: 20Mi`
    - Maximum `cpu: 50m`, `memory: 50Mi`
2. Restart the *Pod* so that the *VPA* recommendations are applied

Answer:

Introduction

The *Vertical Pod Autoscaler* (*VPA*) automatically adjusts CPU and memory requests for *Pods*. Unlike *Horizontal Pod Autoscaler* (*HPA*) which adds/removes replicas, *VPA* scales vertically by adjusting resource requests.

VPA is ideal for:

- Single-instance stateful applications (like this etcd instance)
- Databases where horizontal scaling is complex
- Workloads with variable resource needs but fixed replica count

VPA update modes (https://kubernetes.io/docs/concepts/workloads/autoscaling/):

- `Off`: Only provides recommendations, no automatic changes
- `Initial`: Only applies recommendations at *Pod* creation
- `Recreate`: Evicts *Pods* when resource requests differ significantly from recommendations
- `InPlaceOrRecreate`: Attempts to update *Pod* resources without restarting, falls back to eviction if not possible

Step 1: Create the VPA

First we check what's running:

```
➜ ssh cnpe1080

➜ candidate@cnpe1080:~$ k -n sargasso get sts,pod
NAME                    READY   AGE
statefulset.apps/etcd   1/1     21m

NAME         READY   STATUS    RESTARTS   AGE
pod/etcd-0   1/1     Running   0          21m
```

Now we edit the file and add the *VPA* resource:

```
➜ candidate@cnpe1080:~$ vim /course/15/etcd.yaml
```

```
# cnpe1080:/course/15/etcd.yaml
apiVersion: autoscaling.k8s.io/v1
```

```yaml
kind: VerticalPodAutoscaler
metadata:
  name: etcd-vpa
  namespace: sargasso
spec:
  targetRef:
    apiVersion: apps/v1
    kind: StatefulSet
    name: etcd
  updatePolicy:
    updateMode: "Initial"
  resourcePolicy:
    containerPolicies:
      - containerName: "*"
        minAllowed:
          cpu: 20m
          memory: 20Mi
        maxAllowed:
          cpu: 50m
          memory: 50Mi
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: etcd
  namespace: sargasso
...
```

We add the *VPA* at the top with `---` separator before the existing *StatefulSet* as done in multi-resource files.

The `updateMode: "Initial"` means the *VPA* will only apply recommendations when *Pods* are created.

```
➜ candidate@cnpe1080:~$ k apply -f /course/15/etcd.yaml
verticalpodautoscaler.autoscaling.k8s.io/etcd-vpa created
statefulset.apps/etcd unchanged

➜ candidate@cnpe1080:~$ k -n sargasso get vpa
NAME       MODE      CPU   MEM   PROVIDED   AGE
etcd-vpa   Initial                          32s
```

The *VPA* shows `Initial` mode but does not yet provide CPU/memory recommendations based on observed usage, this can take a minute.

> ℹ️ Wait until the *VPA* shows recommendations (CPU/MEM columns populated and PROVIDED=True) before restarting the *Pod*

```
➜ candidate@cnpe1080:~$ k -n sargasso get vpa
NAME       MODE      CPU   MEM    PROVIDED   AGE
etcd-vpa   Initial   25m   50Mi   True       60s
```

There we go!

Step 2: Restart the *Pod*

Since we're using `Initial` mode, the *VPA* only applies recommendations at *Pod* creation. Right now we see the defaults from the *StatefulSet* in `/course/15/etcd.yaml`:

```
➜ candidate@cnpe1080:~$ k -n sargasso get pod etcd-0 -oyaml | grep -A4 resources:
    resources:
      requests:
        cpu: 10m
        memory: 10Mi
    terminationMessagePath: /dev/termination-log
--
    resources:
      requests:
        cpu: 10m
        memory: 10Mi
    restartCount: 0
```

We need to restart the *Pod* so the *VPA* admission controller can set the new resource requests:

```
➜ candidate@cnpe1080:~$ k -n sargasso rollout restart sts etcd
statefulset.apps/etcd restarted

➜ candidate@cnpe1080:~$ k -n sargasso get pod
NAME     READY   STATUS    RESTARTS   AGE
etcd-0   1/1     Running   0          10s
```

The *StatefulSet* recreates the *Pod* and the *VPA* admission controller applies the recommended resources. We can verify:

```
➜ candidate@cnpe1080:~$ k -n sargasso get pod etcd-0 -oyaml | grep -A4 resources:
    resources:
      requests:
        cpu: 25m
        memory: 50Mi
    terminationMessagePath: /dev/termination-log
--
    resources:
      requests:
        cpu: 25m
        memory: 50Mi
    restartCount: 0
```

The original *StatefulSet* had `cpu: 10m` and `memory: 10Mi`, but after restart the *VPA* applied its recommendations (25m CPU, 50Mi memory) which are within the configured min/max limits.

▼ Question 16 | Argo Rollouts, Canary

Solve this question on: `ssh cnpe2561`

Argo Rollouts is installed with dashboard at `http://cnpe2561:30160`.

In *Namespace* `baltic`, a *Rollout* `webapp` is currently paused during a canary deployment at 50% traffic.

1. Promote the *Rollout* to complete all remaining steps
2. Replace the pause step with an analysis step
    - Use template at `/course/16/analysis_template.yaml`

- Complete the template URL to check the `webapp-canary` *Service*

3. Trigger a new rollout by setting environment variable `VERSION` to `1.18.4`

> ℹ️ The webapp can be reached at `http://cnpe2561:30161` and its *Pods* respond with their version

Answer:

Argo Rollouts: Kubernetes controller providing advanced deployment capabilities like blue-green, canary, and progressive delivery

Rollout: Custom resource that replaces Deployment for progressive delivery strategies

AnalysisTemplate: Defines metrics and success criteria for automated canary validation. Without analysis, Argo Rollouts only checks basic *Pod* readiness

Canary Strategy: Gradually shifts traffic to new version using weight steps and pauses

Investigate

We're working with an ongoing canary deployment which is currently paused and waits for a manual promote.

```
➜ ssh cnpe2561

➜ candidate@cnpe2561:~$ k argo rollouts -n baltic get rollout webapp
Name:            webapp
Namespace:       baltic
Status:          ॥ Paused
Message:         CanaryPauseStep
Strategy:        Canary
  Step:          2/5
  SetWeight:     50
  ActualWeight:  50
Images:          nginx:1-alpine (canary, stable)
Replicas:
  Desired:       4
  Current:       4
  Updated:       2
  Ready:         4
  Available:     4

NAME                               KIND        STATUS     AGE  INFO
⟳ webapp                           Rollout     ॥ Paused   58s
├──# revision:2
│  └──⧉ webapp-6bfc5b4cfc          ReplicaSet  ✓ Healthy  47s  canary
│     ├──□ webapp-6bfc5b4cfc-nsnx4 Pod         ✓ Running  46s  ready:1/1
│     └──□ webapp-6bfc5b4cfc-6sz7x Pod         ✓ Running  43s  ready:1/1
└──# revision:1
   └──⧉ webapp-865fb4d978          ReplicaSet  ✓ Healthy  58s  stable
      ├──□ webapp-865fb4d978-dtj7w Pod         ✓ Running  58s  ready:1/1
      └──□ webapp-865fb4d978-ptn4j Pod         ✓ Running  58s  ready:1/1
```

We can see a weight of 50%, let's investigate how this is implemented under the hood:

```
→ candidate@cnpe2561:~$ k -n baltic get rollout,pod --show-labels
NAME                          DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE   LABELS
rollout.argoproj.io/webapp    4         4         2            4           92s   <none>

NAME                          ...  AGE   LABELS
pod/webapp-6bfc5b4cfc-6sz7x   ...  77s   ...,rollouts-pod-template-hash=6bfc5b4cfc
pod/webapp-6bfc5b4cfc-nsnx4   ...  80s   ...,rollouts-pod-template-hash=6bfc5b4cfc
pod/webapp-865fb4d978-dtj7w   ...  92s   ...,rollouts-pod-template-hash=865fb4d978
pod/webapp-865fb4d978-ptn4j   ...  92s   ...,rollouts-pod-template-hash=865fb4d978
```

Above we see 4 *Pods* and if we check the labels we see two different pod-template-hashes, this means we have two canary *Pods* and two stable *Pods*.

Without a mesh like Linkerd or Istio, Argo Rollouts approximates traffic weights by adjusting replica counts and traffic is distributed via Kubernetes *Service* round-robin across all *Pods*. For example:

- With 4 replicas at 25% weight, it runs 1 canary *Pod* and 3 stable *Pods*
- With 4 replicas at 50% weight, it runs 2 canary *Pods* and 2 stable *Pods*

We can also test this with curl and should see ~50% different versions returned:

```
→ candidate@cnpe2561:~$ k -n baltic get svc
NAME             TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
webapp           NodePort    10.109.22.182   <none>        80:30161/TCP   3m3s
webapp-canary    ClusterIP   10.105.246.79   <none>        80/TCP         3m3s

→ candidate@cnpe2561:~$ curl http://cnpe2561:30161
webapp version 1.18.2

→ candidate@cnpe2561:~$ curl http://cnpe2561:30161
webapp version 1.18.2

→ candidate@cnpe2561:~$ curl http://cnpe2561:30161
webapp version 1.18.3

→ candidate@cnpe2561:~$ curl http://cnpe2561:30161
webapp version 1.18.3

→ candidate@cnpe2561:~$ curl http://cnpe2561:30161
webapp version 1.18.2
```

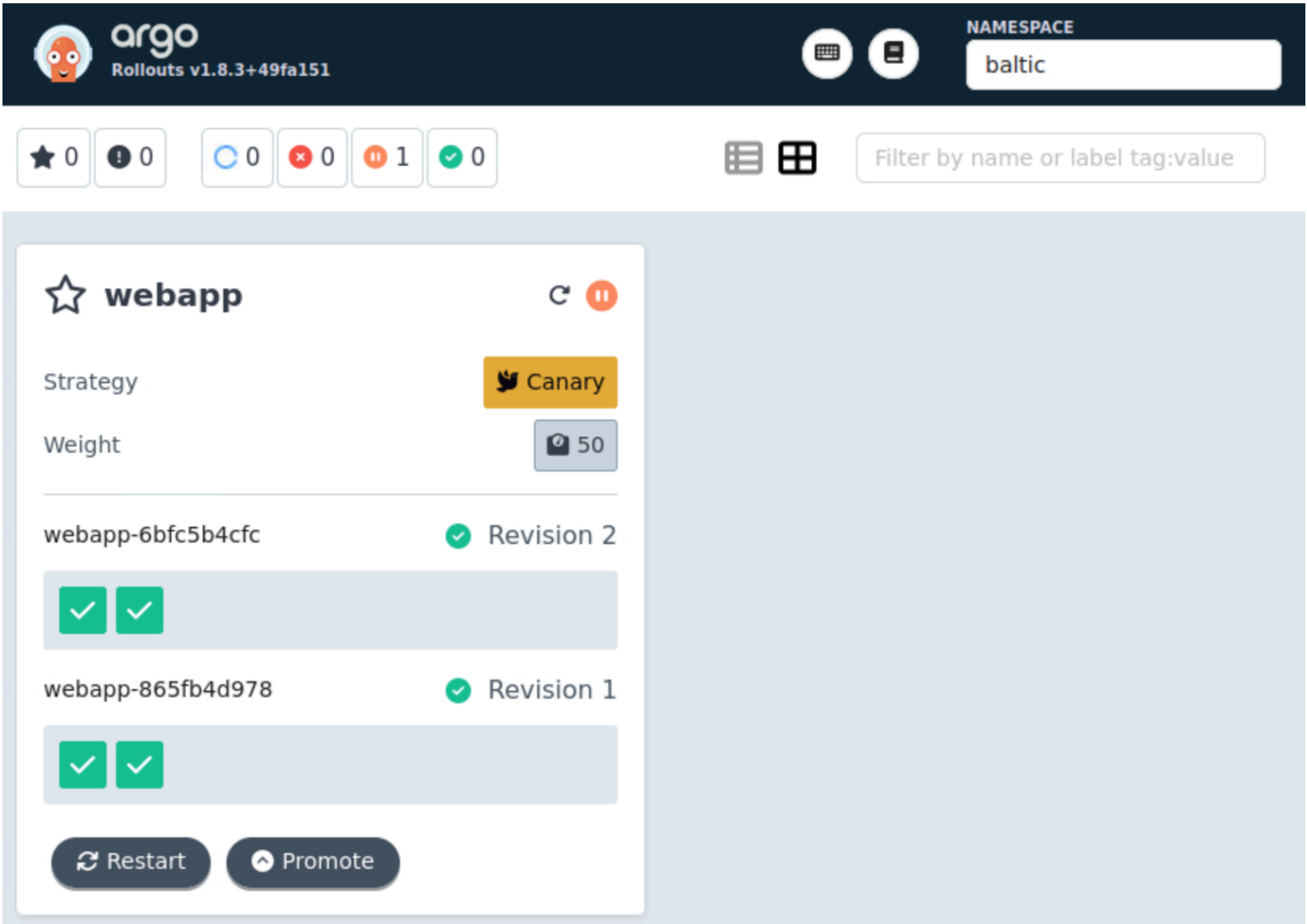
Step 1: Promote the *Rollout*

First, let's check the current rollout status:

```
→ candidate@cnpe2561:~$ k argo rollouts -n baltic get rollout webapp
Name:            webapp
Namespace:       baltic
Status:          ‖ Paused
Message:         CanaryPauseStep
Strategy:        Canary
  Step:          2/5
  SetWeight:     50
  ActualWeight:  50
Images:          nginx:1-alpine (canary, stable)
Replicas:
  Desired:       4
```

```
  Current:        4
  Updated:        2
  Ready:          4
  Available:      4
...
```

The rollout is paused after step 2 (50% weight). We can also see this in the dashboard, make sure to **select the correct** *Namespace* on top right.



We either promote via the dashboard or the terminal:

```
→ candidate@cnpe2561:~$ k argo rollouts -n baltic promote webapp
rollout 'webapp' promoted

→ candidate@cnpe2561:~$ k argo rollouts -n baltic get rollout webapp
Name:            webapp
Namespace:       baltic
Status:          ✓ Healthy
Strategy:        Canary
  Step:          5/5
  SetWeight:     100
  ActualWeight:  100
Images:          nginx:1-alpine (stable)
Replicas:
  Desired:       4
  Current:       4
  Updated:       4
  Ready:         4
  Available:     4
```

```
NAME                                KIND        STATUS        AGE    INFO
⟳ webapp                            Rollout     ✓ Healthy     6m1s
├──# revision:2
│  └──⧉ webapp-6bfc5b4cfc           ReplicaSet  ✓ Healthy     5m50s  stable
│     ├──□ webapp-6bfc5b4cfc-nsnx4  Pod         ✓ Running     5m49s  ready:1/1
│     ├──□ webapp-6bfc5b4cfc-6sz7x  Pod         ✓ Running     5m46s  ready:1/1
│     ├──□ webapp-6bfc5b4cfc-vz7qc  Pod         ✓ Running     32s    ready:1/1
│     └──□ webapp-6bfc5b4cfc-d7nm4  Pod         ✓ Running     29s    ready:1/1
└──# revision:1
   └──⧉ webapp-865fb4d978           ReplicaSet  • ScaledDown  6m1s
```

The rollout is now fully promoted and healthy. And we now only receive version `1.18.3`:

```
→ candidate@cnpe2561:~$ curl http://cnpe2561:30161
webapp version 1.18.3

→ candidate@cnpe2561:~$ curl http://cnpe2561:30161
webapp version 1.18.3

→ candidate@cnpe2561:~$ curl http://cnpe2561:30161
webapp version 1.18.3
```

Step 2: Replace Pause with Analysis

First, check and update the provided template:

```
→ candidate@cnpe2561:~$ vim /course/16/analysis_template.yaml
```

```yaml
# cnpe2561:/course/16/analysis_template.yaml
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: http-check
  namespace: baltic
spec:
  metrics:
  - name: webcheck
    provider:
      web:
        url: http://webapp-canary.baltic  # UPDATED
    successCondition: asInt(result.status) >= 200 && asInt(result.status) < 300
    interval: 10s
    count: 3
```

The template uses Argo Rollouts' built-in web metric provider to perform HTTP health checks. We need to complete the URL to point to the `webapp-canary` *Service*, which routes only to canary *Pods*. We need to use URL `http://webapp-canary.baltic` or even `http://webapp-canary.baltic.svc.cluster.local` because the health checks will be performed from the `argo-rollouts` *Namespace*.

Apply the template:

```
→ candidate@cnpe2561:~$ k -f /course/16/analysis_template.yaml apply
analysistemplate.argoproj.io/http-check created
```

Now update the *Rollout* to use analysis instead of pause:

```
➜ candidate@cnpe2561:~$ k -n baltic edit rollout webapp
```

```
# kubectl -n baltic edit rollout webapp
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
...
  name: webapp
  namespace: baltic
spec:
  replicas: 4
  selector:
    matchLabels:
      app: webapp
  strategy:
    canary:
      canaryService: webapp-canary
      steps:
      - setWeight: 25
      - setWeight: 50
      - analysis:                          # REPLACED "pause"
          templates:
          - templateName: http-check
      - setWeight: 75
      - setWeight: 100
```

The analysis step runs 3 HTTP checks (every 10s) against the canary *Service*. If all return 2xx status codes, the rollout proceeds automatically.

Step 3: Trigger a New Rollout

To trigger a new rollout, update the VERSION environment variable. Edit the rollout:

```
➜ candidate@cnpe2561:~$ k -n baltic edit rollout webapp
```

```
# kubectl -n baltic edit rollout webapp
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
...
  name: webapp
  namespace: baltic
spec:
  template:
    spec:
      containers:
      - name: webapp
        env:
        - name: VERSION
          value: "1.18.4"    # UPDATE
```

After saving, verify the rollout status. The rollout will progress through steps and run the analysis at 50%:

```
➜ candidate@cnpe2561:~$ k argo rollouts -n baltic get rollout webapp
Name:            webapp
Namespace:       baltic
Status:          ◌ Progressing
Message:         more replicas need to be updated
Strategy:        Canary
```

```
  Step:          2/5
  SetWeight:     50
  ActualWeight:  50
Images:          nginx:1-alpine (canary, stable)
Replicas:
  Desired:       4
  Current:       4
  Updated:       2
  Ready:         4
  Available:     4

NAME                                KIND         STATUS         AGE  INFO
⟳ webapp                            Rollout      ◌ Progressing  18m
├──# revision:3
|  ├──⧉ webapp-6df55bccd8           ReplicaSet   ✓ Healthy      20s  canary
|  |  ├──□ webapp-6df55bccd8-26j2x  Pod          ✓ Running      20s  ready:1/1
|  |  └──□ webapp-6df55bccd8-jqclj  Pod          ✓ Running      17s  ready:1/1
|  └──α webapp-6df55bccd8-3-2       AnalysisRun  ◌ Running      14s  ✓ 2
├──# revision:2
|  └──⧉ webapp-6bfc5b4cfc           ReplicaSet   ✓ Healthy      18m  stable
|     ├──□ webapp-6bfc5b4cfc-nsnx4  Pod          ✓ Running      18m  ready:1/1
|     └──□ webapp-6bfc5b4cfc-d7nm4  Pod          ✓ Running      12m  ready:1/1
└──# revision:1
   └──⧉ webapp-865fb4d978           ReplicaSet   • ScaledDown   18m
```

Above we can see the *AnalysisRun* is in progress. After all 3 HTTP checks pass (30s total), the rollout continues:

```
➜ candidate@cnpe2561:~$ k argo rollouts -n baltic get rollout webapp
Name:            webapp
Namespace:       baltic
Status:          ✓ Healthy
Strategy:        Canary
  Step:          5/5
  SetWeight:     100
  ActualWeight:  100
Images:          nginx:1-alpine (stable)
Replicas:
  Desired:       4
  Current:       4
  Updated:       4
  Ready:         4
  Available:     4

NAME                                KIND         STATUS        AGE  INFO
⟳ webapp                            Rollout      ✓ Healthy     18m
├──# revision:3
|  ├──⧉ webapp-6df55bccd8           ReplicaSet   ✓ Healthy     44s  stable
|  |  ├──□ webapp-6df55bccd8-26j2x  Pod          ✓ Running     44s  ready:1/1
|  |  ├──□ webapp-6df55bccd8-jqclj  Pod          ✓ Running     41s  ready:1/1
|  |  ├──□ webapp-6df55bccd8-662xx  Pod          ✓ Running     18s  ready:1/1
|  |  └──□ webapp-6df55bccd8-pfjgf  Pod          ✓ Running     16s  ready:1/1
|  └──α webapp-6df55bccd8-3-2       AnalysisRun  ✓ Successful  38s  ✓ 3
├──# revision:2
|  └──⧉ webapp-6bfc5b4cfc           ReplicaSet   • ScaledDown  18m
└──# revision:1
   └──⧉ webapp-865fb4d978           ReplicaSet   • ScaledDown  18m
```

The rollout completed successfully. The *AnalysisRun* ran 3 HTTP checks against the canary *Service*, all returned 2xx status codes, so the rollout proceeded automatically through to 100%.

▼ Question 17 | FluxCD

Solve this question on: `ssh cnpe7683`

FluxCD is installed and the `flux` CLI is available.

1. Resume the *Kustomization* `havel-west` to correct the drift of repository `/course/17/havel-west`
2. Deploy `/course/17/havel-east`:
    - Create *GitRepository* `havel-east` pointing to `http://192.168.100.21:3000/projects/havel-east.git` branch `main`
    - Create *Kustomization* `havel-east` deploying from *GitRepository* `havel-east` to *Namespace* `havel-east`

Answer:

FluxCD: GitOps operator that continuously syncs Kubernetes cluster state to match what is defined in a Git repository

FluxCD *GitRepository*: FluxCD source that defines a Git repository to fetch manifests from

FluxCD *Kustomization*: FluxCD resource that defines how to apply manifests from a source to the cluster, with optional Kustomize transformations

Drift: When cluster state diverges from the desired state in Git, typically caused by manual changes

Investigate

Let's explore the Git repositories:

```
➜ ssh cnpe7683

➜ candidate@cnpe7683:~$ cd /course/17/havel-west

➜ candidate@cnpe7683:/course/17/havel-west$ ls
configmap.yaml  deployment.yaml  kustomization.yaml

➜ candidate@cnpe7683:/course/17/havel-west$ git remote -v
origin  http://192.168.100.21:3000/projects/havel-west.git (fetch)
origin  http://192.168.100.21:3000/projects/havel-west.git (push)

➜ candidate@cnpe7683:/course/17/havel-west$ cd ../havel-east

➜ candidate@cnpe7683:/course/17/havel-east$ ls
kustomization.yaml  secret-api.yaml  secret-db.yaml  statefulset.yaml

➜ candidate@cnpe7683:/course/17/havel-east$ git remote -v
origin  http://192.168.100.21:3000/projects/havel-east.git (fetch)
origin  http://192.168.100.21:3000/projects/havel-east.git (push)
```

Two Git repositories are provided. Now let's check existing FluxCD resources:

```
➜ candidate@cnpe7683:~$ flux get kustomizations
NAME            REVISION                SUSPENDED       ...
havel-west      main@sha1:a9333527      True            ...

➜ candidate@cnpe7683:~$ flux get sources git
NAME            REVISION                SUSPENDED       ...
havel-west      main@sha1:a9333527      False           ...
```

The `flux get kustomizations` command shows deployment configurations (similar to ArgoCD Applications), while `flux get sources git` shows the Git repositories being synced.

FluxCD works with CRDs and we can see that the *Kustomization* references the *GitRepository* resource:

```
candidate@cnpe7683:~$ k -n flux-system edit kustomization havel-west
```

```
# kubectl -n flux-system edit kustomization havel-west
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
...
  name: havel-west
  namespace: flux-system
spec:
  force: false
  interval: 30s
  path: ./
  prune: true
  sourceRef:                    # reference to GitRepository
    kind: GitRepository
    name: havel-west
  targetNamespace: havel-west
...
```

Now we check for existing resources in the *Namespaces*:

```
➜ candidate@cnpe7683:~$ k -n havel-west get pod
NAME                      READY   STATUS    RESTARTS   AGE
logger-7b78974876-9f6g7   1/1     Running   0          51m

➜ candidate@cnpe7683:~$ k -n havel-east get pod
No resources found in havel-east namespace.
```

We only see resources so far in one *Namespace*. This makes sense because according to the question, `havel-west` has already been configured in FluxCD and deployed, `havel-east` not yet.

Step 1: Correct the Drift

We can have a look at the current drift by using `kubectl diff`:

```
➜ candidate@cnpe7683:/course/17/havel-west$ kubectl diff -k .
diff -u -N /tmp/LIVE-3069104311/apps.v1.Deployment.havel-west.logger
...
 spec:
   progressDeadlineSeconds: 600
-  replicas: 2
+  replicas: 1
   revisionHistoryLimit: 10
```

```
   selector:
     matchLabels:
diff -u -N /tmp/LIVE-3069104311/v1.ConfigMap.havel-west.logger-config
...
 apiVersion: v1
 data:
   log-format: json
-  log-level: debug
+  log-level: info
   retention-days: "30"
 kind: ConfigMap
 metadata:
```

It looks like the replicas and the *ConfigMap* values were updated outside the GitOps workflow.

Check the *Kustomization* status:

```
➜ candidate@cnpe7683:~$ flux get kustomizations
NAME         REVISION              SUSPENDED   READY ...
havel-west   main@sha1:a9333527    True        True  ...
```

The *Kustomization* is suspended, which means FluxCD won't auto-correct drift. Resume it:

```
➜ candidate@cnpe7683:/course/17/havel-west$ flux resume kustomization havel-west
► resuming kustomization havel-west in flux-system namespace
✓ kustomization resumed
◎ waiting for Kustomization reconciliation
✓ Kustomization havel-west reconciliation completed
✓ applied revision main@sha1:a9333527c892c437d30a1843e8ab3fe9e866f3b8

➜ candidate@cnpe7683:/course/17/havel-west$ kubectl diff -k .

➜ candidate@cnpe7683:/course/17/havel-west$
```

Once resumed, FluxCD immediately reconciles and re-applies the manifests from Git. The cluster state now matches Git again.

Step 2: Deploy havel-east

Create a *GitRepository* pointing to the havel-east Git repository and a *Kustomization* referencing it:

> ℹ️ Use existing *GitRepository* and *Kustomization* as examples

```
➜ candidate@cnpe7683:~$ vim 17.yaml
```

```yaml
# cnpe7683: /home/candidate/17.yaml
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: havel-east
  namespace: flux-system
spec:
  interval: 30s
  url: http://192.168.100.21:3000/projects/havel-east.git
  ref:
    branch: main
---
apiVersion: kustomize.toolkit.fluxcd.io/v1
```

```yaml
kind: Kustomization
metadata:
  name: havel-east
  namespace: flux-system
spec:
  interval: 30s
  sourceRef:
    kind: GitRepository
    name: havel-east
  path: ./
  prune: true
  targetNamespace: havel-east
```

```
➜ candidate@cnpe7683:~$ k apply -f 17.yaml
gitrepository.source.toolkit.fluxcd.io/havel-east created
kustomization.kustomize.toolkit.fluxcd.io/havel-east created

➜ candidate@cnpe7683:~$ flux get sources git
NAME          REVISION              SUSPENDED     READY   ...
havel-east    main@sha1:ebc54518    False         True    ...
havel-west    main@sha1:a9333527    False         True    ...

➜ candidate@cnpe7683:~$ flux get kustomizations
NAME          ...   READY   MESSAGE
havel-east    ...   False   Source artifact not found, retrying in 30s
havel-west    ...   True    Applied revision: main@sha1:a9333527
```

The `Source artifact not found` message is normal for a few seconds while FluxCD performs the initial clone of the new repository.

After a bit we should see `havel-east` ready and also resources created.

```
➜ candidate@cnpe7683:~$ flux get kustomizations
NAME            ...   READY   MESSAGE
havel-east      ...   True    Applied revision: main@sha1:ebc54518
havel-west      ...   True    Applied revision: main@sha1:a9333527

➜ candidate@cnpe7683:~$ k -n havel-east get pod
NAME      READY   STATUS    RESTARTS   AGE
cache-0   1/1     Running   0          2m54s
```

▼ Question 18 | Kyverno

Solve this question on: `ssh cnpe4328`

Kyverno is installed and should be used to mutate resources in *Namespace* `caribbean`. The Kyverno CLI is available via `kyverno`.

1. Create a *NamespacedMutatingPolicy* named `security-check` which:
   - Mutates *Pods* during `CREATE` and `UPDATE`
   - Adds the label `audit: pending` to the *Pods*, but only if the label does not already exist
2. Create two *Pods* named `test-pending` and `test-passed` with image `nginx:1-alpine`
3. Update the label on `test-passed` to `audit: passed`, Kyverno should not change it back

It's planned for the future that another service checks for *Pods* with label `audit: pending`, performs security checks, and updates the label value.

Answer:

Kyverno: Kubernetes-native policy engine that can validate, mutate, and generate resources using CEL expressions

NamespacedMutatingPolicy: Namespace-scoped mutation policy using CEL expressions and `ApplyConfiguration` patches

MutatingPolicy: Cluster-wide mutation policy, the cluster-scoped equivalent of *NamespacedMutatingPolicy*

CEL (Common Expression Language): Expression language used in Kubernetes for policy evaluation, replacing Kyverno's older template syntax

Step 1: Create the *NamespacedMutatingPolicy*

```
➜ ssh cnpe4328

➜ candidate@cnpe4328:~$ vim 18.yaml
```

```yaml
# cnpe4328:/home/candidate/18.yaml
apiVersion: policies.kyverno.io/v1
kind: NamespacedMutatingPolicy
metadata:
  name: security-check
  namespace: caribbean
spec:
  matchConstraints:
    resourceRules:
      - apiGroups: [""]
        apiVersions: ["v1"]
        operations: ["CREATE", "UPDATE"]
        resources: ["pods"]
  mutations:
    - patchType: ApplyConfiguration
      applyConfiguration:
        expression: |
          !has(object.metadata.labels) || !("audit" in object.metadata.labels) ?
          Object{
            metadata: Object.metadata{
              labels: Object.metadata.labels{
                audit: "pending"
              }
            }
          } : Object{}
```

The policy matches both `CREATE` and `UPDATE` operations. The CEL condition is essential because without it, every update to a *Pod* would reset the label back to `pending`.

The `!has(object.metadata.labels)` guards against a nil labels map, and `!("audit" in object.metadata.labels)` checks if the key is absent. If the label is missing, it returns an `Object` that adds the label via `ApplyConfiguration`. If the label already exists, it returns an empty `Object{}` which applies no changes.

```
→ candidate@cnpe4328:~$ k -f 18.yaml apply
namespacedmutatingpolicy.policies.kyverno.io/security-check created

→ candidate@cnpe4328:~$ k -n caribbean get namespacedmutatingpolicies
NAME             AGE   READY
security-check   12s
```

Step 2: Create *Pods*

Create two *Pods* without the label:

```
→ candidate@cnpe4328:~$ k -n caribbean run test-pending --image=nginx:1-alpine
pod/test-pending created

→ candidate@cnpe4328:~$ k -n caribbean run test-passed --image=nginx:1-alpine
pod/test-passed created
```

Verify both *Pods* received the label automatically:

```
→ candidate@cnpe4328:~$ k -n caribbean get pod --show-labels
NAME           READY   ...   LABELS
test-passed    1/1     ...   audit=pending,run=test-passed
test-pending   1/1     ...   audit=pending,run=test-pending
```

We see both *Pods* received the label `audit=pending` from our policy.

Step 3: Update Label and Verify

Update the label on `test-passed` to simulate the security service marking it as passed:

```
→ candidate@cnpe4328:~$ k -n caribbean label pod test-passed audit=passed --overwrite
pod/test-passed labeled
```

Verify Kyverno does not change the label back even though the `UPDATE` operation triggers the policy:

```
→ candidate@cnpe4328:~$ k -n caribbean get pod --show-labels
NAME           READY   STATUS    RESTARTS   AGE   LABELS
test-passed    1/1     Running   0          88s   audit=passed,run=test-passed
test-pending   1/1     Running   0          90s   audit=pending,run=test-pending
```

Now test the same for removing the label and should see that the label is added because it does not yet exist:

```
→ candidate@cnpe4328:~$ k -n caribbean label pod test-passed audit-
pod/test-passed unlabeled

→ candidate@cnpe4328:~$ k -n caribbean get pod --show-labels
NAME           READY   STATUS    RESTARTS   AGE   LABELS
test-passed    1/1     Running   0          61s   audit=pending,run=test-passed
test-pending   1/1     Running   0          64s   audit=pending,run=test-pending
```

▼ Question 19 | Crossplane

Solve this question on: `ssh cnpe3849`

Crossplane is installed. The platform team has created a *CompositeResourceDefinition* `redis.cache.killer.sh` and a partial *Composition* that uses native Kubernetes resources.

1. Create a *Redis* resource `cache` in *Namespace* `danau` with size `medium`
2. Extend the *Composition* in `/course/19/composition.yaml` to also create a *Service*:
   - Named `redis`
   - Mapping port `6379` to the *Pods* of the *StatefulSet*
   - Type `ClusterIP`
   - Follow the existing pattern for `patches` and `readinessChecks`
3. Verify the *Service* was added to the existing *Redis* resources

Answer:

Crossplane is a Kubernetes add-on that turns your cluster into a universal control plane (not to confuse with the K8s own control plane) for managing infrastructure via Kubernetes API. It can manage external resources (AWS, GCP, Azure) or cluster-internal ones (*Deployments*, *Services*).

CompositeResourceDefinition (XRD): Defines a custom API (CRD)

Composition: Maps how a composite resource provisions underlying managed resources. Uses Pipeline mode with composition functions

Composite Resource (XR): Instance of a custom API defined by an XRD, directly created by users

Investigate

Let's explore the existing Crossplane setup:

```
➜ ssh cnpe3849

➜ candidate@cnpe3849:~$ k get xrd
NAME                  ESTABLISHED   OFFERED   AGE
redis.cache.killer.sh   True                    3h31m

➜ candidate@cnpe3849:~$ k get compositions
NAME                XR-KIND   XR-APIVERSION               AGE
redis-composition   Redis     cache.killer.sh/v1alpha1    3h31m
```

The XRD defines the *Redis* API:

```
➜ candidate@cnpe3849:~$ cat /course/19/xrd.yaml
```

```yaml
# cnpe3849:/course/19/xrd.yaml
apiVersion: apiextensions.crossplane.io/v2
kind: CompositeResourceDefinition
metadata:
  name: redis.cache.killer.sh
spec:
  group: cache.killer.sh
```

```yaml
  names:
    kind: Redis
    plural: redis
  versions:
    - name: v1alpha1
      served: true
      referenceable: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                size: # users can set spec.size for their Redis resources
                  type: string
                  enum: ["small", "medium", "large"]
                  default: "small"
              required: []
```

The v2 XRD defines *Redis* as a namespaced resource that users create directly. Let's look at the existing *Composition*:

```
➜ candidate@cnpe3849:~$ cat /course/19/composition.yaml
```

```yaml
# cnpe3849:/course/19/composition.yaml
apiVersion: apiextensions.crossplane.io/v1
kind: Composition
metadata:
  name: redis-composition
spec:
  compositeTypeRef:
    apiVersion: cache.killer.sh/v1alpha1
    kind: Redis
  mode: Pipeline
  pipeline:
    - step: patch-and-transform
      functionRef:
        name: function-patch-and-transform
      input:
        apiVersion: pt.fn.crossplane.io/v1beta1
        kind: Resources
        resources:
          - name: statefulset
            base:
              apiVersion: apps/v1
              kind: StatefulSet
              metadata:
                name: redis
              spec:
                serviceName: redis
                replicas: 1
                selector:
                  matchLabels:
                    app: redis
                template:
                  metadata:
                    labels:
                      app: redis
                  spec:
                    containers:
                      - name: redis
                        image: redis:7-alpine
                        ports:
                          - containerPort: 6379
```

```
                    volumeMounts:
                      - name: data
                        mountPath: /data
                  volumes:
                    - name: data
                      emptyDir: {}
          patches:
            - fromFieldPath: metadata.namespace
              toFieldPath: metadata.namespace
          readinessChecks:
            - type: None
        - name: configmap
          base:
            apiVersion: v1
            kind: ConfigMap
            metadata:
              name: redis-config
            data:
              redis.conf: |
                maxmemory 128mb
                maxmemory-policy allkeys-lru
          patches:
            - fromFieldPath: metadata.namespace
              toFieldPath: metadata.namespace
          readinessChecks:
            - type: None
```

The *Composition* uses Pipeline mode with `function-patch-and-transform`. It composes native Kubernetes resources directly (*StatefulSet*, *ConfigMap*). Since v2 XRDs are namespaced, the patches copy `metadata.namespace` from the *Redis* resource to each composed resource.

What we really **need to understand here** is that under `base:` we can simply add our original K8s resources. Even if you have never worked with a *Composition* like this, just from studying it a little you should see the pattern and how to change or use it.

Step 1: Create Redis

Create a *Redis* resource:

```
➜ candidate@cnpe3849:~$ vim 19.yaml
```

```
# cnpe3849: /home/candidate/19.yaml
apiVersion: cache.killer.sh/v1alpha1
kind: Redis
metadata:
  name: cache
  namespace: danau
spec:
  size: medium
```

```
➜ candidate@cnpe3849:~$ k apply -f 19.yaml
redis.cache.killer.sh/cache created
```

Wait for the resources to be provisioned and verify:

```
➜ candidate@cnpe3849:~$ k -n danau get redis
NAME    SYNCED   READY   COMPOSITION          AGE
cache   True     True    redis-composition    29s

➜ candidate@cnpe3849:~$ k -n danau get sts,svc,cm,pod
```

```
NAME                        READY   AGE
statefulset.apps/redis      1/1     40s

NAME                            DATA   AGE
configmap/kube-root-ca.crt      1      3h35m
configmap/redis-config          1      40s

NAME          READY   STATUS    RESTARTS   AGE
pod/redis-0   1/1     Running   0          40s
```

The *Redis* resource creation caused Crossplane to create the *StatefulSet* and *ConfigMap*.

Step 2: Complete the Composition

Add a *Service* resource to the *Composition*. We can generate the *Service* YAML with dry-run:

```
➜ candidate@cnpe3849:~$ k create service clusterip redis --tcp=6379:6379 --dry-run=client -o yaml
apiVersion: v1
kind: Service
metadata:
  name: redis
spec:
  ports:
  - name: 6379-6379
    port: 6379
    targetPort: 6379
  selector:
    app: redis
  type: ClusterIP
```

This gives us the base *Service* spec. Following the existing pattern in the *Composition*, we need to:

1. Wrap it under `base:`
2. Add `patches:` to copy the namespace from the *Redis* resource
3. Add `readinessChecks:` with `type: None`

```
➜ candidate@cnpe3849:~$ vim /course/19/composition.yaml
```

```yaml
# cnpe3849:/course/19/composition.yaml
apiVersion: apiextensions.crossplane.io/v1
kind: Composition
metadata:
  name: redis-composition
spec:
  compositeTypeRef:
    apiVersion: cache.killer.sh/v1alpha1
    kind: Redis
  mode: Pipeline
  pipeline:
    - step: patch-and-transform
      functionRef:
        name: function-patch-and-transform
      input:
        apiVersion: pt.fn.crossplane.io/v1beta1
        kind: Resources
        resources:
          - name: statefulset
            base:
```

```
            apiVersion: apps/v1
            kind: StatefulSet
            metadata:
              name: redis
            spec:
...
        - name: configmap
          base:
            apiVersion: v1
            kind: ConfigMap
            metadata:
              name: redis-config
            data:
              redis.conf: |
                maxmemory 128mb
                maxmemory-policy allkeys-lru
          patches:
            - fromFieldPath: metadata.namespace
              toFieldPath: metadata.namespace
          readinessChecks:
            - type: None
        - name: service                         # ADD from here
          base:
            apiVersion: v1
            kind: Service
            metadata:
              name: redis
            spec:
              ports:
                - name: 6379-6379
                  port: 6379
                  targetPort: 6379
              selector:
                app: redis
              type: ClusterIP
          patches:
            - fromFieldPath: metadata.namespace
              toFieldPath: metadata.namespace
          readinessChecks:
            - type: None                        # ADD until here
```

Apply the updated *Composition*:

```
➜ candidate@cnpe3849:~$ k -f /course/19/composition.yaml diff
diff -u -N /tmp/LIVE-597703222/apiextensions.crossplane.io.v1.Composition..
...
-  generation: 1
+  generation: 2
   name: redis-composition
   resourceVersion: "163388"
   uid: bab9d24d-4e85-4c50-83fd-10525f345930
@@ -65,4 +65,22 @@
         patches:
         - fromFieldPath: metadata.namespace
           toFieldPath: metadata.namespace
+      - base:
+          apiVersion: v1
+          kind: Service
+          metadata:
+            name: redis
+          spec:
+            ports:
+            - name: 6379-6379
```

```
+              port: 6379
+              targetPort: 6379
+            selector:
+              app: redis
+            type: ClusterIP
+        name: service
+        patches:
+        - fromFieldPath: metadata.namespace
+          toFieldPath: metadata.namespace
+        readinessChecks:
+        - type: None
     step: patch-and-transform

➜ candidate@cnpe3849:~$ k apply -f /course/19/composition.yaml
composition.apiextensions.crossplane.io/redis-composition configured
```

Step 3: Verify *Service* Added

Crossplane continuously reconciles. The *Service* should be created automatically for the existing *Redis*:

```
➜ candidate@cnpe3849:~$ k -n danau get statefulset,service,configmap,pod
NAME                     READY   AGE
statefulset.apps/redis   1/1     3m13s

NAME            TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
service/redis   ClusterIP   10.99.175.171   <none>        6379/TCP   18s

NAME                         DATA   AGE
configmap/kube-root-ca.crt   1      3h38m
configmap/redis-config       1      3m13s

NAME          READY   STATUS    RESTARTS   AGE
pod/redis-0   1/1     Running   0          3m13s
```

The *Service* was added to the existing *Redis* resources without needing to recreate anything. This demonstrates Crossplane's declarative reconciliation.

▼ Question 20 | Linkerd, Gateway API

Solve this question on: `ssh cnpe4328`

Namespace `saltlake-app` is part of the Linkerd mesh:

1. Create two *Server* resources:
   - `frontend` for *Pod* label `app: frontend` on port `80`
   - `backend` for *Pod* label `app: backend` on port `80`
2. Fix existing *AuthorizationPolicy* `frontend-to-backend` to allow the `frontend` *Pods* to access the `backend` *Pods*
3. There are two backend versions available via *Services* `backend-v1` and `backend-v2`. Create an *HTTPRoute* (Gateway API) `backend-canary` for the `backend` *Service* that implements traffic splitting:

- 10% to `backend-v1`
- 90% to `backend-v2`

> Test connection for example with `kubectl -n saltlake-app exec deploy/frontend -c frontend -- curl backend`

Answer:

Linkerd: Lightweight service mesh for Kubernetes providing mTLS, observability, and traffic management

Server: Linkerd resource that defines a port on a set of *Pods* to apply authorization policies to

AuthorizationPolicy: Linkerd resource that defines which clients can access a *Server* based on mesh identity

HTTPRoute: Gateway API resource that defines HTTP routing rules, including traffic splitting for canary deployments

Investigate

Let's explore the current state of the environment:

```
➜ ssh cnpe4328

➜ candidate@cnpe4328:~$ k -n saltlake-app get deploy,svc,sa
NAME                              READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/backend-v1        1/1     1            1           2m7s
deployment.apps/backend-v2        1/1     1            1           2m6s
deployment.apps/frontend          1/1     1            1           2m7s

NAME                  TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)   AGE
service/backend       ClusterIP   10.97.76.5       <none>        80/TCP    2m6s
service/backend-v1    ClusterIP   10.101.179.49    <none>        80/TCP    2m4s
service/backend-v2    ClusterIP   10.101.129.196   <none>        80/TCP    2m3s
service/frontend      ClusterIP   10.98.57.167     <none>        80/TCP    2m6s

NAME                            SECRETS   AGE
serviceaccount/backend-sa       0         2m7s
serviceaccount/default          0         2m7s
serviceaccount/frontend-sa      0         2m7s

➜ candidate@cnpe4328:~$ k -n saltlake-app get pod
NAME                          READY   STATUS    RESTARTS   AGE
backend-v1-7544b6c79b-4cspn   2/2     Running   0          2m20s
backend-v2-94fdbff54-7ndg6    2/2     Running   0          2m20s
frontend-6d5cb8746c-kzwjv     2/2     Running   0          2m20s
```

The *Pods* show `2/2` containers because the Linkerd sidecar is injected in `saltlake-app` *Namespace*. Note the *ServiceAccounts* are named `frontend-sa` and `backend-sa`. There are two backend versions (`backend-v1` and `backend-v2`) and three *Services*: `backend` (selects both), `backend-v1`, and `backend-v2`.

Step 1: Create Server resources

Create *Server* resources for both `frontend` and `backend` on port 80 (as specified in the question) using the *Pod* labels:

```
→ candidate@cnpe4328:~$ k -n saltlake-app get pod --show-labels
NAME                            READY   STATUS    ...   LABELS
backend-v1-7544b6c79b-4cspn     2/2     Running   ...   app=backend,...
backend-v2-94fdbff54-7ndg6      2/2     Running   ...   app=backend,...
frontend-6d5cb8746c-kzwjv       2/2     Running   ...   app=frontend,...

→ candidate@cnpe4328:~$ vim 20.yaml
```

```yaml
# cnpe4328:/home/candidate/20.yaml
apiVersion: policy.linkerd.io/v1beta3
kind: Server
metadata:
  name: frontend
  namespace: saltlake-app
spec:
  podSelector:
    matchLabels:
      app: frontend
  port: 80
---
apiVersion: policy.linkerd.io/v1beta3
kind: Server
metadata:
  name: backend
  namespace: saltlake-app
spec:
  podSelector:
    matchLabels:
      app: backend
  port: 80
```

Both backend versions share the `app=backend` label, so one *Server* resource covers both.

```
→ candidate@cnpe4328:~$ k apply -f 20.yaml
server.policy.linkerd.io/frontend created
server.policy.linkerd.io/backend created

→ candidate@cnpe4328:~$ k -n saltlake-app get server
NAME       PORT   PROTOCOL   ACCESS POLICY
backend    80     unknown    deny
frontend   80     unknown    deny
```

Both *Servers* show `deny` access policy. There is an existing *AuthorizationPolicy* `frontend-to-backend` which should allow the `frontend` to access `backend`, but testing shows it's not working:

```
→ candidate@cnpe4328:~$ k -n saltlake-app exec deploy/frontend -c frontend -- curl -v backend
...
* Established connection to backend (10.97.76.5 port 80) from 10.32.0.7 port 40402
* using HTTP/1.x
> GET / HTTP/1.1
> Host: backend
> User-Agent: curl/8.17.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 403 Forbidden
< date: Sun, 18 Jan 2026 13:34:43 GMT
< content-length: 0
<
```

```
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
* Connection #0 to host backend:80 left intact
```

We can see `HTTP/1.1 403 Forbidden`, so we need to investigate the *AuthorizationPolicy*.

Step 2: Find and fix the AuthorizationPolicy issue

Let's examine the existing *AuthorizationPolicy*:

```
➜ candidate@cnpe4328:~$ k -n saltlake-app get authorizationpolicy frontend-to-backend -o yaml
apiVersion: policy.linkerd.io/v1alpha1
kind: AuthorizationPolicy
metadata:
...
  name: frontend-to-backend
  namespace: saltlake-app
spec:
  requiredAuthenticationRefs:
  - kind: ServiceAccount
    name: frontend
  targetRef:
    group: policy.linkerd.io
    kind: Server
    name: backend
```

The *AuthorizationPolicy* targets the `backend` *Server* and references a *ServiceAccount* named `frontend`. However, looking back at our investigation, the actual *ServiceAccount* used by the frontend *Deployment* is `frontend-sa`, not `frontend`.

The fix is to update the *AuthorizationPolicy* to reference the correct *ServiceAccount* name:

```
➜ candidate@cnpe4328:~$ k -n saltlake-app edit authorizationpolicy frontend-to-backend
```

```
# kubectl -n saltlake-app edit authorizationpolicy frontend-to-backend
apiVersion: policy.linkerd.io/v1alpha1
kind: AuthorizationPolicy
metadata:
...
  name: frontend-to-backend
  namespace: saltlake-app
spec:
  requiredAuthenticationRefs:
  - kind: ServiceAccount
    name: frontend-sa    # UPDATE: change from "frontend" to "frontend-sa"
  targetRef:
    group: policy.linkerd.io
    kind: Server
    name: backend
```

Now we test again:

```
→ candidate@cnpe4328:~$ k -n saltlake-app exec deploy/frontend -c frontend -- curl -s backend
{"name": "backend", "version": "v1"}

→ candidate@cnpe4328:~$ k -n saltlake-app exec deploy/frontend -c frontend -- curl -s backend
{"name": "backend", "version": "v2"}

→ candidate@cnpe4328:~$ k -n saltlake-app exec deploy/frontend -c frontend -- curl -s backend
{"name": "backend", "version": "v1"}

→ candidate@cnpe4328:~$ k -n saltlake-app exec deploy/frontend -c frontend -- curl -s backend
{"name": "backend", "version": "v2"}
```

Now we receive `HTTP/1.1 200 OK` responses. The `backend` *Service* selects both v1 and v2 *Pods*, so responses alternate randomly between them.

(Optional) Verify Linkerd mesh identities

We can also verify the mesh identities used for authorization:

```
→ candidate@cnpe4328:~$ linkerd identity -l app=frontend -n saltlake-app

POD frontend-6d5cb8746c-7jj8p (1 of 1)

Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number: 15 (0xf)
    Signature Algorithm: ECDSA-SHA256
        Issuer: CN=identity.linkerd.
        Validity
            Not Before: Jan 6 16:57:44 2026 UTC
            Not After : Jan 7 16:58:24 2026 UTC
        Subject: CN=frontend-sa.saltlake-app.serviceaccount.identity.linkerd.cluster.local
        Subject Public Key Info:
            Public Key Algorithm: ECDSA
                Public-Key: (256 bit)
                X:
                    68:6c:4d:c0:c2:9a:8c:c3:63:17:03:9a:3b:6a:6c:
                    77:ef:ac:3a:b0:12:0d:b0:f6:d5:53:06:58:c9:8f:
                    be:1d
                Y:
                    5a:77:99:ec:d1:0a:f7:7b:02:2d:07:63:3d:4c:e9:
                    be:fb:db:70:f8:8d:24:e9:f9:a3:f2:41:d2:9e:55:
                    18:27
                Curve: P-256
        X509v3 extensions:
            X509v3 Key Usage: critical
                Digital Signature, Key Encipherment
            X509v3 Extended Key Usage:
                TLS Web Server Authentication, TLS Web Client Authentication
            X509v3 Authority Key Identifier:
                keyid:59:43:62:B7:2D:C6:C4:E8:75:42:77:1E:89:01:46:51:AE:F8:4D:E3
            X509v3 Subject Alternative Name:
                DNS:frontend-sa.saltlake-app.serviceaccount.identity.linkerd.cluster.local

    Signature Algorithm: ECDSA-SHA256
         30:44:02:20:67:74:ae:dd:0f:7c:13:f0:91:ac:eb:d1:e3:16:
```

```
b7:c8:9e:fe:2d:df:76:5b:6c:d6:cd:10:b4:c0:4e:d7:66:39:
02:20:24:3e:ac:fc:89:5c:d5:ac:92:ee:88:21:94:44:6e:06:
7b:c5:b2:d3:58:57:32:3a:8b:d0:b3:8f:fa:1c:fc:ed
```

The `frontend-sa` identity shown in the certificate's Subject (`CN=frontend-sa.saltlake-app...`) matches what we configured in the *AuthorizationPolicy*, which is why the connection is now allowed.

Step 3: Create HTTPRoute for canary traffic splitting

Create an *HTTPRoute* that splits traffic to the `backend` *Service*: 10% to `backend-v1` and 90% to `backend-v2`:

```
➜ candidate@cnpe4328:~$ vim 20.yaml
```

```yaml
# cnpe4328:/home/candidate/20.yaml
...
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: backend-canary
  namespace: saltlake-app
spec:
  parentRefs:
    - name: backend
      kind: Service
      group: ""
  rules:
    - backendRefs:
        - name: backend-v1
          port: 80
          weight: 10
        - name: backend-v2
          port: 80
          weight: 90
```

The *HTTPRoute* uses:

- `parentRefs` to target the `backend` *Service* (the *Service* clients call)
- `backendRefs` define where traffic is routed:
    - 10% to `backend-v1`
    - 90% to `backend-v2`

Let's give it a try:

```
➜ candidate@cnpe4328:~$ k apply -f 20.yaml
server.policy.linkerd.io/frontend unchanged
server.policy.linkerd.io/backend unchanged
httproute.gateway.networking.k8s.io/backend-canary created

➜ candidate@cnpe4328:~$ k -n saltlake-app get httproute
NAME              HOSTNAMES   AGE
backend-canary                8s
```

Test the canary traffic splitting by making multiple requests:

```
➜ candidate@cnpe4328:~$ for i in {1..20}; do kubectl -n saltlake-app exec deploy/frontend -c frontend --
curl -s backend; done
```

```
{"name": "backend", "version": "v2"}
{"name": "backend", "version": "v1"}
{"name": "backend", "version": "v2"}
{"name": "backend", "version": "v2"}
{"name": "backend", "version": "v2"}
{"name": "backend", "version": "v2"}
{"name": "backend", "version": "v2"}
{"name": "backend", "version": "v2"}
{"name": "backend", "version": "v1"}
{"name": "backend", "version": "v2"}
{"name": "backend", "version": "v2"}
{"name": "backend", "version": "v2"}
{"name": "backend", "version": "v2"}
{"name": "backend", "version": "v1"}
{"name": "backend", "version": "v2"}
{"name": "backend", "version": "v2"}
{"name": "backend", "version": "v2"}
{"name": "backend", "version": "v2"}
{"name": "backend", "version": "v2"}
{"name": "backend", "version": "v2"}
```


The majority of responses come from `v2` (~90%), with occasional `v1` responses (~10%), confirming the canary traffic split is working.

Best of luck for your CNPE exam!

CNPE Tips Kubernetes 1.35

Success on the CNPE is all about knowing your way around the ecosystem. Here's how to prep.

Knowledge

General

- Study all topics as proposed in the curriculum until you feel comfortable with all
- Do both test sessions with this CNPE simulator. Understand the solutions and maybe try out other ways to achieve the same thing
- Be fast and breathe `kubectl`
- Learn and study the in-browser scenarios on
    - https://killercoda.com/killer-shell-cnpe
- Additionally it could help to do some CKAD and maybe CKA scenarios
    - https://killercoda.com/killer-shell-ckad
    - https://killercoda.com/killer-shell-cka
- Check the CNCF Platforms White Paper
- Reading into some content of the theoretical CNPA could be useful

Applications

You'll be working with various applications from the CNCF landscape. For some questions you can decide to use tool A or tool B, the UI or CLI. Applications you **might** have to work with:

- Argo
  - Argo CD
  - Argo Rollouts
  - Argo Workflows
- Crossplane
- Flagger
- Flux
- Gatekeeper
- Grafana
- Istio
- Jaeger
- Kyverno
- Linkerd
- OPA
- OpenCost
- OpenTelemetry
- Prometheus
- Tekton

> ℹ️ Verify the list here

Additional Applications

We would expand the list of applications with the following to be on the safe side:

- CloudNativePG
- Git
- Helm
- Kustomize
- Loki
- OpenTofu / Terraform

Kubernetes Resources

Among general Kubernetes knowledge you should be familiar with:

- Custom Resource Definitions
- StorageClass
- Persistent Volume and Persistent Volume Claim
- ResourceQuota

CNPE Exam Info

Read the Curriculum

https://github.com/cncf/curriculum

Read the Handbook

https://docs.linuxfoundation.org/tc-docs/certification/lf-handbook2

Read the important instructions

https://docs.linuxfoundation.org/tc-docs/certification/important-instructions-cnpe

Read the FAQ

https://docs.linuxfoundation.org/tc-docs/certification/frequently-asked-questions-cnpe

Kubernetes documentation

Get familiar with the Kubernetes documentation and be able to use the search. Allowed resources are:

- https://kubernetes.io/docs
- https://kubernetes.io/blog
- Task-specific documentation provided in the Quick Reference box. This includes links to documentation for various tools that might be needed to solve a task. The list of tools that may be included in the exam can be found in Important Instructions.

> ℹ️ Verify the list here

The Exam UI / Remote Desktop

The real exam, as well as the simulator, provides a Remote Desktop (XFCE) on Ubuntu/Debian. Coming from OSX/Windows there will be changes in copy&paste for example.

Official Information

ExamUI: Performance Based Exams

Lagging

There could be some lagging, definitely make sure you are using a good internet connection because your webcam and screen are transferring all the time.

Kubectl autocompletion and commands

The following are installed or pre-configured, verify the list here:

- `kubectl` with `k` alias and Bash autocompletion
- `yq` or YAML processing
- `curl` and `wget` for testing web services
- `man` and man pages for further documentation

> ℹ️ You're allowed to install tools, like `tmux` for terminal multiplexing or `jq` for JSON processing

Copy & Paste

Copy and pasting will work like normal in a Linux Environment:

What always works: copy+paste using right mouse context menu What works in Terminal: Ctrl+Shift+c and Ctrl+Shift+v What works in other apps like Firefox: Ctrl+c and Ctrl+v

Score

There are 15-20 questions in the exam. Your results will be automatically checked according to the handbook. If you don't agree with the results you can request a review by contacting the Linux Foundation Support.

Notepad & Flagging Questions

You can flag questions to return to later. This is just a marker for yourself and won't affect scoring. You also have access to a simple notepad in the browser which can be used to store any kind of plain text. It might make sense to use this and write down additional information about flagged questions. Instead of using the notepad you could also open Mousepad (XFCE application inside the Remote Desktop) or create a file with Vim.

VSCodium

You can use VSCodium to edit files and you can also use its terminal to run commands. You're not allowed to install any VSCodium extensions.

Servers

Each question needs to be solved on a specific instance other than your main terminal. You'll need to connect to the correct instance via ssh, the command is provided before each question.

PSI Bridge

The PSI Secure Browser will be installed by you at the moment you attend your real exam:

- It can be downloaded using the newest versions of Microsoft Edge, Safari, Chrome, or Firefox
- Multiple monitors will no longer be permitted
- Use of personal bookmarks will no longer be permitted

The new ExamUI includes improved features such as:

- A remote desktop configured with the tools and software needed to complete the tasks
- A timer that displays the actual time remaining (in minutes) and provides an alert with 30, 15, or 5 minute remaining
- The content panel remains the same (presented on the Left Hand Side of the ExamUI)

Terminal Handling

Bash Aliases

In the real exam, each question has to be solved on a different instance to which you connect via ssh. This means it's not advised to configure bash aliases because they wouldn't be available on the instances accessed by ssh.

Be fast

Use the `history` command to reuse already entered commands or use even faster history search through **Ctrl +r** .

If a command takes some time to execute, like sometimes `kubectl delete pod x`. You can put a task in the background using **Ctrl +z** and pull it back into foreground running command `fg`.

You can delete *pods* fast with:

```
k delete pod x --grace-period 0 --force
```

Vim

Be great with vim.

Settings

In case you face a situation where vim is not configured properly and you face for example issues with pasting copied content you should be able to configure via `~/.vimrc` or by entering manually in vim settings mode:

```
set tabstop=2
set expandtab
set shiftwidth=2
```

The `expandtab` option makes sure to use spaces for tabs.

Note that changes in `~/.vimrc` will not be transferred when connecting to other instances via ssh.

Toggle vim line numbers

When in `vim` you can press **Esc** and type `:set number` or `:set nonumber` followed by **Enter** to toggle line numbers. This can be useful when finding syntax errors based on line - but can be bad when wanting to mark&copy by mouse. You can also just jump to a line number with **Esc** `:22` + **Enter**.

Copy&Paste

Get used to copy/paste/cut with vim:

```
Mark lines: Esc+V (then arrow keys)
Copy marked lines: y
Cut marked lines: d
Paste lines: p or P
```

Indent multiple lines

To indent multiple lines press **Esc** and type `:set shiftwidth=2`. First mark multiple lines using `Shift v` and the up/down keys. Then to indent the marked lines press `>` or `<`. You can then press `.` to repeat the action.

Legal Notice / Impressum

Privacy Policy / Datenschutz

Terms / AGB

CONTENT

CKS

CKA

CKAD

CNPE

LFCS

LINKS

Killercoda

Kim Wuestkamp